Instituto Politécnico Nacional

UNIDAD PROFESIONAL INTERDISCIPLINARIA EN INGENIERÍA Y TECNOLOGÍAS AVANZADAS

Trabajo Terminal I

Control de posición de robot manipulador usando redes neuronales cuantizadas.

Que para obtener el título de

"Ingeniero en Mecatrónica"

Presentan:

Ramiro Mendoza Diaz

Asesores:

Dr. Jorge Alejandro Juárez Lora

Presidente del Jurado	**Profesor titular**
Dra. Blanca Rosa Briseño Tepepa	M. en C. Cecilia Fernández Nava

Índice

Resumen iv

Abstract iv

Índice de figuras vi

Índice de tablas viii

Nomenclatura ix

Simbología xi

Introducción 1

1. Marco de Referencia 12
- 1.1. Marco teórico . . . 12
  - 1.1.1. Modelado y Control de Robots Manipuladores . . . 12
  - 1.1.2. Redes Neuronales Artificiales para el Control Robótico . . . 17
  - 1.1.3. Determinismo Computacional y Tiempo Real Estricto en Robótica 18
  - 1.1.4. Estrategias de Optimización de Redes Neuronales . . . 19
- 1.2. Marco procedimental . . . 25
  - 1.2.1. Metodología mecatrónica . . . 25

2. Diseño del sistema 27
- 2.1. Diseño conceptual . . . 27
  - 2.1.1. Necesidades - requerimientos . . . 27
  - 2.1.2. Arquitectura funcional (funciones) . . . 29
  - 2.1.3. Arquitectura física (módulos) . . . 32
  - 2.1.4. Propuestas solución . . . 32
  - 2.1.5. Validación . . . 34
  - 2.1.6. Selección diseño conceptual . . . 36
- 2.2. Diseño detallado . . . 37
  - 2.2.1. Detalle módulo 1 (M_1) – Planta Robótica y Entorno Sim-to-Real 37
  - 2.2.2. Detalle módulo 2 (M_2) – Control Dinámico Analítico en Tiempo Real . . . 39
  - 2.2.3. Detalle módulo 3 (M_3) – Inteligencia Artificial y Motor Cuantizado 40
  - 2.2.4. Detalle módulo 4 (M_4) – Supervisión, Telemetría y Adquisición 44
  - 2.2.5. Integración del sistema mecatrónico . . . 45

3. Implementación del sistema 47
- 3.1. Implementación módulo 1 (M_1): Planta robótica y entorno Sim-to-Real 47
  - 3.1.1. Verificación (M_1) . . . 48
- 3.2. Implementación módulo 2 (M_2): Control dinámico analítico en tiempo real 48
  - 3.2.1. Verificación (M_2) . . . 50

3.3. Implementación módulo 3 (M_3): Inteligencia artificial y motor cuantizado 51
3.3.1. Verificación (M_3) . . . 57
3.4. Implementación módulo 4 (M_4): Supervisión, telemetría y adquisición . 61
3.4.1. Verificación (M_4) . . . 61
3.5. Integración del sistema mecatrónico . . . 63
3.5.1. Verificación de la integración del sistema . . . 64

4. Análisis de resultados **65**
4.1. Entorno de pruebas y condiciones experimentales (Sim-to-Real) . . . 65
4.2. Precisión dinámica y error de seguimiento articular . . . 66
4.3. Desempeño cronométrico, latencia de inferencia y presupuesto temporal a 500 Hz . . . 67
4.4. Eficiencia computacional y compresión del modelo neuronal . . . 70
4.5. Matriz consolidada de cumplimiento técnico de requerimientos . . . 71

Conclusiones **73**

Recomendaciones y trabajo a futuro **73**

Anexos **77**

Referencias **83**

Resumen

El despliegue de controladores avanzados basados en Redes Neuronales Profundas (DNNs) en la robótica de manipulación enfrenta restricciones severas de latencia y consumo energético impuestas por el cuello de botella de Von Neumann y la barrera de memoria en sistemas de cómputo embebidos. En lazos de control cerrados en tiempo real (100 Hz − 500 Hz), la evaluación continua de modelos densos en punto flotante (FP32) degrada la predictibilidad temporal e incrementa la potencia disipada. En este contexto, el presente reporte de Trabajo Terminal 1 aborda el diseño, implementación y verificación de un esquema mecatrónico híbrido de control de posición para un robot manipulador serial de 6 grados de libertad (Universal Robots UR3e), integrando dinámica analítica inversa con una Red Neuronal Cuantizada a enteros de 8 bits (QNN-INT8).

La metodología implementada articula una ley de control por par calculado (*Computed Torque Control*) donde el Algoritmo Recursivo de Newton-Euler (RNEA), programado en C++, compensa las no linealidades mecánicas nominales ($\mathcal{O}(n)$). Una red neuronal perceptrón multicapa (MLP), con 42 características que incluyen el par calculado por RNEA, se entrena para aproximar el torque total del controlador analítico con retroalimentación PD. Durante el control neuronal, la salida desnormalizada se aplica directamente tras su saturación. El modelo fue entrenado en PyTorch, exportado al estándar ONNX y cuantizado mediante Cuantización Dinámica Post-Entrenamiento (*Dynamic PTQ*) a INT8, ejecutándose dentro del middleware ROS 2 Jazzy Jalisco con el motor ONNX Runtime monohilo en C++. La evaluación experimental se llevó a cabo en un modelo simulado desarrollado en Gazebo Harmonic gobernado por la arquitectura modular de `ros2_control` a una frecuencia nominal de 500 Hz ($T_s = 2$ ms). La publicación de referencias y la recopilación de datos se realizaron a 100 Hz ($T_{datos} = 10$ ms), independientemente de la frecuencia interna del control.

Los resultados reportados indican una reducción del tamaño del modelo del 74,43 % y una latencia media de inferencia INT8 de 29,2 μs. El tiempo total del controlador neuronal queda pendiente de verificar; la inferencia aislada no constituye una garantía temporal. El RMSE de seguimiento fue de 0,063 rad para el controlador analítico, 0,092 rad para FP32 y 0,138 rad para INT8. RF-01, con criterio RMSE $<$ 0,05 rad, no se alcanzó. La implementación conserva asignaciones dinámicas y la saturación neuronal no representa una política completa de seguridad. Estos avances y limitaciones constituyen una referencia para TT2, donde deberán evaluarse las mejoras de precisión, las protecciones, el consumo eléctrico y la transferencia física.

Palabras clave: Redes Neuronales Cuantizadas, Robótica de Manipulación, Control por Par Calculado, Cuantización Dinámica, ROS 2, INT8, Sim-to-Real.

Abstract

The deployment of advanced controllers based on Deep Neural Networks (DNNs) in robotic manipulation faces severe latency and energy constraints imposed by the Von Neumann bottleneck and the memory wall in embedded computing platforms. In closed real-time control loops (100 Hz − 500 Hz), continuous evaluation of high-density floating-point models (FP32) degrades temporal predictability and escalates power dis-

sipation. In this context, this Term Project 1 report presents the design, implementation, and verification of a hybrid mechatronic position control scheme for a 6-DOF serial manipulator (Universal Robots UR3e), coupling analytical inverse dynamics with an 8-bit integer Quantized Neural Network (QNN-INT8).

The methodology integrates a Computed Torque Control (CTC) framework wherein the Recursive Newton-Euler Algorithm (RNEA), implemented in C++, compensates for nominal mechanical non-linearities with linear complexity ($\mathcal{O}(n)$). A multilayer perceptron (MLP), fed by 42 input features including the RNEA torque, is trained to approximate the total torque generated by the analytical controller with PD feedback. During neural control, the denormalized output is applied directly after saturation. The network was trained in PyTorch, exported to the ONNX standard, and compressed via dynamic Post-Training Quantization (Dynamic PTQ) to INT8, running within the ROS 2 Jazzy Jalisco middleware via the single-threaded C++ ONNX Runtime engine. Experimental verification was conducted on a simulated model in Gazebo Harmonic managed by the `ros2_control` framework at a nominal closed-loop frequency of 500 Hz ($T_s = 2$ ms). Trajectory references and experimental data were published and collected at 100 Hz ($T_{data} = 10$ ms), independently of the internal control frequency.

Reported results indicate a 74,43 % reduction in model size and an INT8 mean inference latency of 29,2 μs. The total neural controller execution time remains to be verified; isolated inference latency does not establish a timing guarantee. Tracking RMSE was 0,063 rad for the analytical controller, 0,092 rad for FP32, and 0,138 rad for INT8. RF-01, requiring RMSE $<$ 0,05 rad, was not met. The implementation retains dynamic allocations, and neural output saturation does not constitute a complete safety policy. These advances and limitations provide a baseline for TT2, where accuracy improvements, protection mechanisms, electrical power consumption, and physical transfer must be evaluated.

Keywords: Quantized Neural Networks, Robotic Manipulation, Computed Torque Control, Dynamic Quantization, ROS 2, INT8, Sim-to-Real.

Índice de figuras

1. Arquitectura CPU–memoria como contexto conceptual de la cuantización e indicadores observados en TT1: tamaño del archivo ONNX y latencia media de inferencia bajo la configuración experimental. Se señalan las verificaciones pendientes de duración del controlador, jitter y consumo eléctrico. 3
2. Esquema conceptual del Algoritmo Recursivo de Newton–Euler (RNEA): propagación hacia adelante ($i = 1 \dots n$), balance de esfuerzos hacia atrás ($i = n \dots 1$) y proyección del momento sobre el eje articular. El cronometraje anotado corresponde al controlador RNEA + PD integrado. . . 15
3. Control por par calculado como referencia teórica: el PD genera una corrección de aceleración antes de la dinámica inversa. El desacoplamiento corresponde al caso ideal de modelo exacto. La nota inferior distingue esta formulación de RNEA + PD en torque implementado en TT1. . . 16
4. Flujos de cuantización: PTQ dinámica utilizada en TT1, con pesos cuantizados fuera de línea y parámetros de activación calculados durante la inferencia, frente a QAT como propuesta para TT2, con cuantización aparente y estimador STE. Los resultados de tamaño de archivo corresponden exclusivamente a la campaña PTQ reportada. 24
5. Marco metodológico ágil *Scrum* adaptado a la ingeniería mecatrónica del proyecto, integrando de forma iterativa el desarrollo de dinámica analítica, simulación del gemelo digital, pipeline de inteligencia artificial cuantizada y evaluación de controladores en C++. Las duraciones son orientativas y la transferencia física se contempla para TT2. 27
6. Arquitectura funcional del controlador neuronal de torque total: RNEA como entrada de la red, desnormalización y saturación de la salida aplicada al UR3e virtual. Se distinguen el control interno nominal a 500 Hz, las referencias y datos a 100 Hz y la transferencia física prevista para TT2. El límite uniforme de ±55 N·m es provisional para simulación. . . 31
7. Predictor de torque total `UR3eNetwork`: (a) topología, ReLU y 307,590 parámetros; (b1, c1) formatos FP32 e INT8 en complemento a dos; (b2, c2) ejemplos conceptuales de resolución numérica. Las curvas no son registros experimentales y sus recuadros presentan métricas reportadas por separado. La salida es estandarizada y el torque físico se obtiene tras desnormalizarla. En FP32 se distingue la fracción $m = 0.f$ del significando $1 + m = 1.f$. 43
8. Ejemplos conceptuales de cuantización uniforme: (a) mapeo simétrico de pesos y (b) mapeo afín de activaciones. Se muestran niveles equidistantes, representación del cero y saturación en los extremos de los rangos ilustrativos. Las curvas no representan histogramas ni parámetros medidos del modelo. 56

9. Distinción entre las tasas nominales de control, publicación de referencias y recopilación de datos. Los conteos calculados a partir de frecuencia y duración no sustituyen el número efectivo de observaciones del registro experimental. 60
10. Perfiles cinemáticos suaves de posición, velocidad y aceleración articular generados mediante interpolación polinomial quíntica de clase $\mathcal{C}^2$, evidenciando derivadas continuas y nulas en los extremos temporales. . . . 62

Índice de cuadros

1. Acrónimos y términos técnicos empleados en el proyecto. IX
2. Simbología matemática y magnitudes físicas del sistema mecatrónico. . XI
3. Requerimientos funcionales y no funcionales del sistema de control mecatrónico. 28
4. Matriz de selección conceptual de Pugh: ponderaciones y valoraciones de diseño, no resultados experimentales de los controladores. 36
5. Parámetros de Denavit-Hartenberg modificados del robot UR3e. 37
6. Topología de la red neuronal predictora de torque total (UR3eNetwork). 41
7. Comparativa de recursos entre el modelo original en FP32 y el modelo cuantizado en INT8. 42
8. Presupuesto temporal consolidado del lazo crítico de control mecatrónico a 500 Hz ($T_s = 2000\,\mu$s). 46
9. Métricas experimentales de verificación para el módulo M_1 (Planta simulada). 49
10. Métricas experimentales de verificación para el módulo M_2 (Control Dinámico Clásico). 51
11. Resultados de verificación de los modelos FP32 e INT8 (M_3): control interno nominal a 500 Hz y recopilación de datos a 100 Hz. 60
12. Métricas experimentales de verificación para el módulo M_4 (Telemetría y Adquisición). 63
13. Comparativa de latencia de inferencia en C++ (ONNX Runtime) con control interno nominal a 500 Hz y recopilación de datos a 100 Hz. . . . 68
14. Presupuesto temporal consolidado del ciclo de control de par neuronal a 500 Hz ($T_s = 2000\,\mu$s). 69
15. Matriz comparativa de verificación del sistema mecatrónico frente a los requerimientos de diseño. 72

Nomenclatura

En la Tabla 1 se presentan los acrónimos, términos técnicos y abreviaturas especializadas utilizadas a lo largo de este documento, ordenadas alfabéticamente.

Cuadro 1: Acrónimos y términos técnicos empleados en el proyecto.

Término / Sigla	**Significado y Descripción Técnica**
AI / IA	Inteligencia Artificial (*Artificial Intelligence*).
ANN	Red Neuronal Artificial (*Artificial Neural Network*).
CIM	Computación en Memoria (*Computing-In-Memory*).
CoM	Centro de Masa (*Center of Mass*) de un eslabón rígido.
CTC	Control por Par Computado (*Computed Torque Control*).
DH	Denavit-Hartenberg (convención cinemática modificada de parámetros geométricos).
DNN	Red Neuronal Profunda (*Deep Neural Network*).
FP32	Formato de punto flotante de precisión simple de 32 bits (estándar IEEE 754).
FPGA	Matriz de Puertas Lógicas Programable en Campo (*Field-Programmable Gate Array*).
GDL	Grados de Libertad (*Degrees of Freedom*, DoF).
INT8	Formato numérico entero con signo de 8 bits (rango $[-128, 127]$).
MAC	Operación de Multiplicación y Acumulación (*Multiply-Accumulate*).
MAE	Error Absoluto Medio (*Mean Absolute Error*).
MCAP	Formato de almacenamiento eficiente para grabación de datos y telemetría en ROS 2.
ML	Aprendizaje Automático (*Machine Learning*).
MLP	Perceptrón Multicapa (*Multilayer Perceptron*).
MSE	Error Cuadrático Medio (*Mean Squared Error*).
ONNX	Formato Abierto para Intercambio de Redes Neuronales (*Open Neural Network Exchange*).
PD / PID	Controlador Proporcional-Derivativo / Proporcional-Integral-Derivativo.
PINN	Red Neuronal Informada por la Física (*Physics-Informed Neural Network*).
PTQ	Cuantización Post-Entrenamiento (*Post-Training Quantization*).
QAT	Entrenamiento Consciente de Cuantización (*Quantization-Aware Training*).
QNN	Red Neuronal Cuantizada (*Quantized Neural Network*).

Continúa en la siguiente página...

Término / Sigla	Significado y Descripción Técnica
QoS	Calidad de Servicio (*Quality of Service*) en protocolos de transporte ROS 2.
RNEA	Algoritmo Recursivo de Newton-Euler (*Recursive Newton-Euler Algorithm*).
RMSE	Raíz del Error Cuadrático Medio (*Root Mean Square Error*).
ROS / ROS 2	Sistema Operativo de Robots (*Robot Operating System* versión 2).
RT	Tiempo Real (*Real-Time*).
SNN	Red Neuronal de Impulsos (*Spiking Neural Network*).
UR3e	Robot manipulador colaborativo industrial de 6 GDL de Universal Robots.
URDF	Formato Unificado de Descripción de Robots (*Unified Robot Description Format*).
VoBo	Visto Bueno de aprobación emitido por el jurado o asesores.

Simbología

En la Tabla 2 se detallan las variables matemáticas, operadores, vectores y matrices empleados en la formulación cinemática, dinámica y de control, especificando sus unidades en el Sistema Internacional (SI).

Cuadro 2: Simbología matemática y magnitudes físicas del sistema mecatrónico.

Símbolo	**Descripción Física o Matemática**	**Unidad (SI)**
Cinemática y Variables Articulares:		
$\boldsymbol{q} = [q_1, \ldots, q_6]^T$	Vector de posiciones articulares generalizadas	rad
$\dot{\boldsymbol{q}} = [\dot{q}_1, \ldots, \dot{q}_6]^T$	Vector de velocidades articulares	rad/s
$\ddot{\boldsymbol{q}} = [\ddot{q}_1, \ldots, \ddot{q}_6]^T$	Vector de aceleraciones articulares	rad/s^2
$\boldsymbol{q}_d, \dot{\boldsymbol{q}}_d, \ddot{\boldsymbol{q}}_d$	Trayectorias articular deseada (posición, velocidad y aceleración)	rad, rad/s, rad/s^2
$\boldsymbol{e} = \boldsymbol{q}_d - \boldsymbol{q}$	Vector de error de seguimiento de posición articular	rad
$\dot{\boldsymbol{e}} = \dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}}$	Vector de error de velocidad articular	rad/s
α_{i-1}	Parámetro DH modificado: ángulo de torsión de eslabón	rad
a_{i-1}	Parámetro DH modificado: longitud de eslabón	m
d_i	Parámetro DH modificado: desplazamiento en el eje articular	m
θ_i	Parámetro DH modificado: variable angular articular	rad
$\boldsymbol{R}_i^{i-1}$	Matriz de rotación del marco coordenado i respecto al marco $i-1$	Adimensional
$\boldsymbol{P}_i^{i-1}$	Vector de traslación del origen $i-1$ al origen i expresado en el marco i	m
$\boldsymbol{z}_0$	Vector unitario sobre el eje de rotación articular $[0,0,1]^T$	Adimensional
Cinemática Diferencial y Dinámica RNEA (Paso hacia Adelante):		
$\boldsymbol{\omega}_i$	Vector de velocidad angular del eslabón i expresado en su propio marco	rad/s
$\dot{\boldsymbol{\omega}}_i$	Vector de aceleración angular del eslabón i	rad/s^2
$\dot{\boldsymbol{v}}_i$	Vector de aceleración lineal del origen del marco coordenado i	m/s^2
$\dot{\boldsymbol{v}}_{c,i}$	Vector de aceleración lineal del centro de masa del eslabón i	m/s^2
$\boldsymbol{r}_{c,i}$	Vector de posición local del centro de masa del eslabón i	m
$\boldsymbol{g} = [0,0,9{,}81]^T$	Vector de aceleración de la gravedad terrestre	m/s^2

Continúa en la siguiente página...

Símbolo	Descripción Física o Matemática	Unidad (SI)
Dinámica Analítica y Esfuerzos RNEA (Paso hacia Atrás):		
m_i	Masa total del eslabón i	kg
$\boldsymbol{I}_i$	Tensor de inercia baricéntrico del eslabón i expresado en su marco local	$\text{kg} \cdot \text{m}^2$
$\boldsymbol{F}_i$	Fuerza inercial total que actúa sobre el eslabón i	N
$\boldsymbol{N}_i$	Momento inercial total que actúa sobre el eslabón i	$\text{N} \cdot \text{m}$
$\boldsymbol{f}_i$	Fuerza ejercida por el eslabón $i-1$ sobre el eslabón i	N
$\boldsymbol{n}_i$	Par/momento ejercido por el eslabón $i-1$ sobre el eslabón i	$\text{N} \cdot \text{m}$
$\boldsymbol{\tau} = [\tau_1, \ldots, \tau_6]^T$	Vector de pares motores netos generalizados aplicados a las articulaciones	$\text{N} \cdot \text{m}$
$\boldsymbol{\tau}_{rnea}$	Par dinámico de prealimentación calculado por RNEA	$\text{N} \cdot \text{m}$
$\boldsymbol{\tau}_{cmd}$	Par de comando total enviado a las interfaces de esfuerzo de los motores	$\text{N} \cdot \text{m}$
$\boldsymbol{M}(\boldsymbol{q})$	Matriz de masa e inercia del manipulador (6×6)	$\text{kg} \cdot \text{m}^2$
$\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})$	Matriz de términos centrífugos y de Coriolis (6×6)	$\text{N} \cdot \text{m} \cdot \text{s/rad}$
$\boldsymbol{g}(\boldsymbol{q})$	Vector de pares gravitacionales articulares (6×1)	$\text{N} \cdot \text{m}$
Control Automático y Parámetros de Redes Neuronales:		
$\boldsymbol{K}_p, \boldsymbol{K}_d, \boldsymbol{K}_i$	Matrices de ganancias de retroalimentación proporcional, derivativa e integral	$\text{N} \cdot \text{m/rad}, \text{N} \cdot \text{m} \cdot \text{s/rad}, \text{N} \cdot \text{m}/(\text{rad} \cdot \text{s})$
$\boldsymbol{a}_{cmd}$	Vector de aceleración de control auxiliar desacoplada	rad/s^2
$\boldsymbol{X} \in \mathbb{R}^{42}$	Vector de entrada a la red neuronal (7 subgrupos de 6 articulaciones)	Dimensiones mixtas
$\boldsymbol{y} \in \mathbb{R}^6$	Vector de salida predicho por la red (par estimado para 6 articulaciones)	$\text{N} \cdot \text{m}$
$\boldsymbol{\mu}_X, \boldsymbol{\sigma}_X$	Vectores de media y desviación estándar para normalización de entrada	Dimensiones mixtas
$\boldsymbol{\mu}_y, \boldsymbol{\sigma}_y$	Vectores de media y desviación estándar para desnormalización de salida	$\text{N} \cdot \text{m}$
S	Factor de escala de cuantización (*Scale factor*)	Adimensional
Z	Punto cero entero de compensación de cuantización (*Zero-point*)	Adimensional
q_{int8}	Valor entero cuantizado a 8 bits de un operando o peso	Adimensional
T_s	Periodo de muestreo del bucle de control de tiempo real (2 ms − 10 ms, equivalente a 100 Hz − 500 Hz)	s

Introducción

Los robots manipuladores son componentes esenciales en la automatización industrial, la cirugía asistida, la exploración espacial y una creciente variedad de aplicaciones de servicio, gracias a su capacidad para realizar tareas de manipulación con precisión y repetibilidad [1]. El control efectivo de estos sistemas es fundamental para su desempeño. Tradicionalmente, se han empleado técnicas de control clásico, como el control Proporcional-Integral-Derivativo (PID) o el control por par calculado, cuyo desempeño depende de la tarea, la sintonización y la precisión del modelo dinámico [1]. Sin embargo, la creciente demanda de mayor autonomía, adaptabilidad a entornos no estructurados y la capacidad para manejar interacciones complejas ha impulsado la adopción de enfoques basados en Inteligencia Artificial (IA), particularmente las Redes Neuronales Artificiales (ANNs) y las Redes Neuronales Profundas (DNNs) [2, 3].

A pesar de sus ventajas, la implementación de DNNs en sistemas robóticos conlleva desafíos significativos. Estos modelos suelen ser computacionalmente intensivos, caracterizados por un gran número de parámetros y una alta densidad de operaciones aritméticas, lo que se traduce en un considerable consumo energético y una fuerte demanda de recursos de memoria y procesamiento [4, 5]. Estas exigencias son particularmente problemáticas para las plataformas robóticas autónomas, que operan con fuentes de alimentación y recursos computacionales limitados[6, 7].

El interés de este estudio surge de la necesidad de reducir estos costes computacionales y energéticos. Las Redes Neuronales Cuantizadas (QNNs), que reducen la precisión numérica de los pesos y activaciones de una red, ofrecen una vía prometedora para comprimir modelos y acelerar la inferencia, con el potencial de disminuir drásticamente el consumo energético y el uso de memoria [5, 8].

Este trabajo busca aportar conocimiento práctico y cuantitativo sobre la viabilidad y los beneficios de utilizar QNNs para el control eficiente de robots manipuladores, ofreciendo una perspectiva con un enfoque en la eficiencia energética de diferentes métodos de control de sistemas robóticos.

Planteamiento del problema

El control dinámico de robots manipuladores es un componente fundamental para su implementación exitosa en la automatización industrial, la robótica de servicio y entornos de manufactura avanzada. A medida que las aplicaciones exigen trayectorias más veloces, mayor precisión y capacidad de adaptación ante entornos dinámicos e incertidumbres, las leyes de control se vuelven significativamente más complejas, transitando desde lazos lineales descentralizados hacia algoritmos no lineales basados en modelos dinámicos analíticos y aproximadores universales como las redes neuronales profundas [2, 3]. Sin embargo, esta sofisticación algorítmica conlleva un incremento sustancial en el costo computacional y en la demanda energética del sistema de procesamiento.

Dicha problemática se manifiesta formalmente a través de tres dimensiones interconectadas: la densidad masiva de álgebra matricial requerida a altas tasas de refresco, las restricciones estructurales impuestas por la arquitectura de computación subyacente (el cuello de botella de Von Neumann y la barrera de memoria), y la proyección insos-

tenible del consumo de potencia en plataformas embebidas autónomas. A continuación, se detallan cada uno de estos aspectos.

Multiplicación de matrices, procesamiento de señales y problemas de computación en tareas robóticas

Las multiplicaciones matriciales y las operaciones de multiplicación-acumulación (MAC) constituyen el núcleo operativo fundamental en la robótica moderna. Tanto el cálculo cinemático diferencial mediante operadores jacobianos, como la resolución de la dinámica inversa a través de las formulaciones de Euler-Lagrange o Newton-Euler, e incluso las arquitecturas avanzadas de control basadas en aprendizaje automático, dependen intrínsecamente del álgebra lineal de vectores y matrices multidimensionales [4, 1].

En manipuladores seriales industriales que operan en lazos de control cerrados en tiempo real (con frecuencias típicas de 100 Hz o superiores), estas evaluaciones algebraicas deben completarse de forma predecible y con holgura temporal dentro de una ventana de 10 ms. La repetición continua de estas operaciones sobre plataformas de procesamiento convencional no solo impone una severa cota superior a la frecuencia de muestreo alcanzable, sino que dispara el consumo energético global, una limitación directamente atribuible a la organización física de las arquitecturas de cómputo actuales.

Cuello de botella de Von Neumann y la barrera de memoria

La arquitectura Von Neumann constituye el paradigma computacional predominante en los sistemas embebidos e industriales contemporáneos, caracterizado por compartir un canal y espacio de memoria unificado tanto para las instrucciones del programa como para los datos manipulados [9, 10].

Si bien este modelo ha posibilitado la estandarización y evolución de la microelectrónica de propósito general, presenta restricciones estructurales críticas derivadas de la separación física entre la unidad central de procesamiento (CPU) y la memoria principal. La necesidad de transitar constantemente datos y coeficientes a través de un bus de ancho de banda acotado genera el fenómeno conocido como el *cuello de botella de Von Neumann* y la *barrera de memoria* (*memory wall*) [9, 10, 11]. En algoritmos con un tráfico masivo de operandos de alta densidad, como la inferencia de redes neuronales y el filtrado matricial en tiempo real, la CPU experimenta ciclos prolongados de inactividad mientras espera la entrega de datos, disipando energía estática considerable y degradando la latencia del bucle de control.

La Figura 1 relaciona el intercambio CPU–memoria, como contexto conceptual, con los indicadores observados de tamaño del archivo ONNX y latencia media de inferencia. La reducción de archivo no constituye una medición del tráfico del bus ni del consumo eléctrico. Se distinguen el control interno nominal a 500 Hz, las referencias y registros a 100 Hz y las verificaciones pendientes del controlador completo.

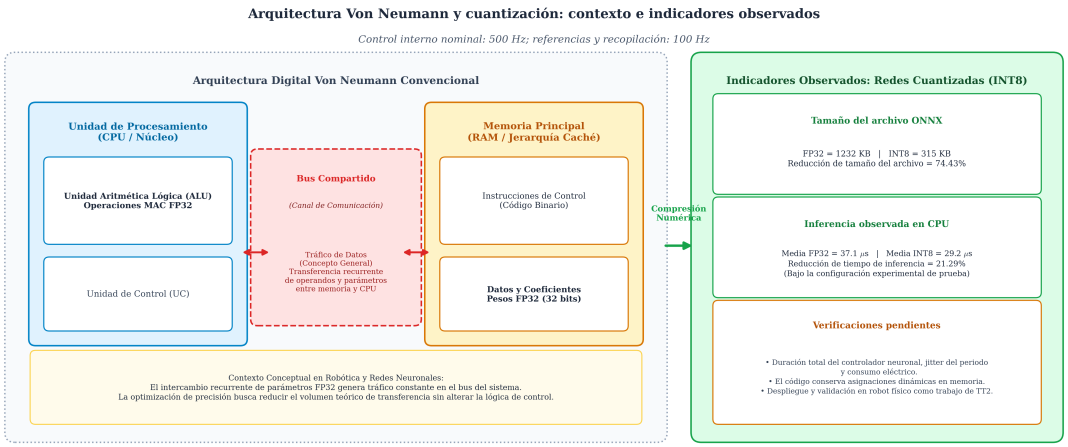


Figura 1: Arquitectura CPU–memoria como contexto conceptual de la cuantización e indicadores observados en TT1: tamaño del archivo ONNX y latencia media de inferencia bajo la configuración experimental. Se señalan las verificaciones pendientes de duración del controlador, jitter y consumo eléctrico.

Proyección a futuro de la problemática energética

La evolución de la robótica y la inteligencia artificial converge de forma inexorable hacia sistemas autónomos con capacidad de percepción multisensorial, toma de decisiones no estructuradas y compensación adaptativa en tiempo real. No obstante, conforme las arquitecturas de redes neuronales se vuelven más profundas y los sensores incrementan su tasa de muestreo, el costo computacional escala drásticamente, incrementando el consumo de potencia eléctrica a niveles incompatibles con plataformas robóticas móviles, exoesqueletos o robots manipuladores con fuentes de energía acotadas [6, 7, 12].

Este desafío evidencia la necesidad ineludible de explorar paradigmas computacionales optimizados, centrados no únicamente en la exactitud teórica del algoritmo, sino en la minimización del costo energético por ciclo de control mediante técnicas formales de compresión numérica y diseño de software determinista.

Justificación

Frente a las severas restricciones energéticas y de latencia impuestas por las plataformas de procesamiento digital estándar, se vuelve imperativo investigar métodos de control que optimicen simultáneamente la fidelidad dinámica y la eficiencia de cómputo. El presente Trabajo Terminal se enfoca en el estudio, diseño e implementación de un esquema de compensación dinámica basado en Redes Neuronales Cuantizadas (*Quantized Neural Networks*, QNNs) para el control articular de un robot manipulador industrial.

Las redes neuronales cuantizadas transforman las operaciones de punto flotante de alta precisión (típicamente FP32) en aritméticas enteras de precisión reducida (como INT8 o INT4). Esta discretización reduce de forma cuadrática o lineal el área de silicio y la energía consumida por cada operación aritmética, además de comprimir la huella de memoria requerida para el almacenamiento de los tensores de pesos y sesgos en

aproximadamente un 75 % respecto a sus contrapartes en punto flotante [13, 14, 5]. Si bien este mapeo discreto introduce un ruido de cuantización inherente sobre los pares de control generados, una adecuada estrategia de entrenamiento y calibración numérica busca limitar la degradación del desempeño; la estabilidad y el cumplimiento de tolerancias requieren evaluación específica.

Justificación mecatrónica

El desarrollo del presente proyecto es intrínsecamente mecatrónico, ya que no puede resolverse desde una perspectiva disciplinar aislada (únicamente mecánica, puramente electrónica, exclusivamente de control o exclusivamente computacional). La solución integra y articula de manera sinérgica los cuatro pilares fundamentales de la Ingeniería Mecatrónica:

1. **Sistemas Mecánicos (Cinemática y Dinámica del Manipulador):** El manipulador robótico industrial (Universal Robots UR3e) constituye una cadena cinemática abierta serial de 6 grados de libertad caracterizada por acoplamientos inerciales no lineales, efectos centrífugos, fuerzas de Coriolis y pares gravitacionales dependientes de la configuración. La mecánica impone las condiciones de frontera físicas del sistema: momentos de inercia baricéntricos, límites de esfuerzo por articulación, cuya identificación y aplicación se contemplan para la implementación física, y la necesidad de planificar trayectorias continuas de clase $\mathcal{C}^2$ con perfiles suaves de aceleración y jerk para proteger las transmisiones mecánicas armónicas (*harmonic drives*) frente a sobrecargas y fatiga.

2. **Sistemas Electrónicos y Cómputo Embebido:** Comprende los sensores articulares, las interfaces de comunicación y la arquitectura de procesamiento. La cuantización INT8 se estudia como alternativa para reducir el almacenamiento del modelo y evaluar la latencia de inferencia. Los efectos sobre tráfico de memoria, disipación térmica y despliegue en hardware reconfigurable requieren mediciones propias de la plataforma.

3. **Sistemas de Control Automático:** El control por par calculado (*Computed Torque Control*, CTC) emplea dinámica inversa y realimentación para regular el movimiento articular. Sus propiedades de desacoplamiento y estabilidad dependen de la formulación y de supuestos sobre el modelo, las ganancias y la ejecución. En este proyecto se configuró un periodo nominal de $T_s = 2$ ms (500 Hz); no se presenta como un requisito universal de CTC ni como una demostración de estabilidad del controlador implementado.

4. **Software e Inteligencia Artificial:** La arquitectura de software se basa en el estándar robótico contemporáneo ROS 2 Jazzy y el framework `ros2_control`. Incorpora el diseño, entrenamiento y optimización de redes neuronales profundas (MLP de 42 variables de entrada y 6 salidas de par articular) para la compensación de dinámicas no lineales complejas, el empleo de técnicas de cuantización numérica post-entrenamiento (*Dynamic PTQ* INT8 simétrica) y su ejecución en C++17

mediante el motor optimizado ONNX Runtime, con evaluación de su desempeño en el ciclo de control.

Sinergia e integración mecatrónica: La integración mecatrónica requiere considerar conjuntamente el modelo dinámico, el seguimiento articular y los recursos de cómputo. El periodo nominal de 2 ms es una decisión de configuración del proyecto. La precisión del modelo y los efectos de su compresión deben evaluarse junto con los tiempos de ejecución y las condiciones de operación de la planta.

Objetivos

Objetivo general

Evaluar el desempeño energético, de precisión y de recursos utilizados en la implementación de un control de lazo cerrado con una planta robótica con control tradicional y con una red neuronal cuantizada.

Objetivos específicos TT1

- Investigar el estado del arte de las redes neuronales cuantizadas (QNNs) aplicadas al control de sistemas robóticos, las metodologías de cuantización (Post-Training Quantization - PTQ y Quantization-Aware Training - QAT), y las técnicas para la estimación y comparación del consumo energético en plataformas de cómputo embebidas y tradicionales.
- Definir la arquitectura de una red neuronal artificial (ANN) adecuada para el control de posición de un robot manipulador, incluyendo el preprocesamiento de las señales de entrada necesarias para la red.
- Diseñar el sistema de control de posición basado en la ANN seleccionada (sin cuantizar) y establecer las métricas de rendimiento de control que servirán como línea base.
- Establecer una metodología detallada para la cuantización del modelo de red neuronal diseñado, seleccionando al menos dos niveles de precisión distintos (ej., INT6 y INT8) y las herramientas de software para llevar a cabo el proceso.
- Diseñar el plan experimental para la comparación energética y de rendimiento, definiendo los escenarios de prueba, las variables a medir (ej., latencia de inferencia, uso de memoria, consumo energético estimado/medido, precisión del control) y la configuración del entorno de pruebas (simulado y/o físico).
- Caracterizar y poner en marcha la plataforma robótica y su interfaz de control, para implementar y evaluar los diferentes esquemas de control.
- Escribir en el documento escrito dichos avances en la investigación.

Objetivos específicos TT2

- Implementar el lazo de control numérico tradicional (ej. PID) de posición del efector final, tanto en simulación numérica, como en físico, con la finalidad de tener un *benchmark* para comparar con la red neuronal quantizada, tanto en desempeño, recurso energético, etc.
- Entrenar, implementar y validar el sistema de control de posición basado en la red neuronal no cuantizada (ANN de punto flotante) en la plataforma robótica, evaluando su rendimiento de control y su perfil de consumo energético como segundo punto de referencia.

- Implementar los sistemas de control de posición basados en las redes neuronales cuantizadas (QNNs) para los niveles de precisión seleccionados en TT1, integrándolos con la plataforma robótica.
- Realizar estudio comparativo de ambas soluciones y evaluar el desempeño energético, de control, de recursos computacionales, y de recursos de hardware implementados.
- Analizar e interpretar los resultados experimentales para cuantificar las ventajas y desventajas de cada enfoque de control, con un énfasis particular en la eficiencia energética lograda por las QNNs y su relación con la precisión del control.
- Documentar la metodología, los resultados obtenidos, las conclusiones del estudio comparativo y las recomendaciones para futuras implementaciones de QNNs en control robótico.

Alcance y delimitación del proyecto

Para organizar el desarrollo y delimitar su evaluación, el desarrollo integral del proyecto se encuentra formalmente delimitado en dos etapas correspondientes a las asignaturas curriculares de Trabajo Terminal 1 (TT1) y Trabajo Terminal 2 (TT2). Esta división establece con precisión las fronteras operativas, el alcance de las evaluaciones y los entregables comprometidos:

Delimitación metodológica de Trabajo Terminal 1 (TT1)

En la presente etapa de TT1, la investigación y el desarrollo de ingeniería se concentran en el diseño conceptual y detallado de la arquitectura mecatrónica, el modelado matemático analítico del robot manipulador UR3e, y la validación algorítmica y computacional en un entorno de simulación física de alta fidelidad (*Sim-to-Real*) utilizando ROS 2 Jazzy, Gazebo Harmonic y el framework `ros2_control`.

Bajo este alcance, la evaluación de la eficiencia energética se formula a nivel de software y arquitectura de cómputo a través de indicadores cuantitativos indirectos (*proxies*):

- **Reducción del tamaño del modelo:** Cuantificación de la reducción del 74,43 % en el tamaño de los archivos ONNX utilizados en la campaña, de 1232 KB en FP32 a 315 KB en INT8. Esta comparación caracteriza almacenamiento en disco; no mide el tráfico del bus ni la memoria total utilizada durante la ejecución.
- **Evaluación de cuantización:** Despliegue del modelo INT8 mediante ONNX Runtime en C++17 y comparación de su tamaño de archivo y latencia de inferencia con FP32, sin atribuir instrucciones vectoriales específicas no verificadas.
- **Caracterización temporal en CPU:** Para INT8 se reportan una latencia media de inferencia de 29,2 μs y un percentil 95 de 31,8 μs. El tiempo medio de 151,2 μs corresponde al control analítico RNEA + PD. El tiempo total del controlador

neuronal y su margen respecto al periodo nominal de 2 ms quedan por verificar; estos intervalos no establecen garantías temporales absolutas.

La fase de TT1 comprende la integración del gemelo digital, la caracterización del desempeño de seguimiento articular y la estructuración del plan experimental comparativo. El requisito RF-01 de RMSE $< 0{,}05$ rad no se alcanzó en la campaña evaluada; sus resultados constituyen la referencia de desempeño y un objetivo de mejora para TT2.

Delimitación metodológica de Trabajo Terminal 2 (TT2)

La fase subsecuente de TT2 abarcará la transferencia, despliegue físico e instrumentación experimental sobre el manipulador robótico industrial Universal Robots UR3e disponible en las instalaciones de laboratorio. Los alcances específicos de dicha fase comprenden:

- **Medición eléctrica directa en banco de pruebas:** Instrumentación física con analizadores de potencia, osciloscopios y sondas de corriente para cuantificar directamente el consumo de potencia eléctrica activa (en Watts) y la energía acumulada (en Joules) en la etapa de cómputo y en los actuadores del robot.
- **Validación física experimental:** Ejecución comparativa sobre el robot físico bajo trayectorias estandarizadas para contrastar el error de seguimiento (RMSE articular y cartesiano), el comportamiento ante dinámicas no modeladas (fricción de Coulomb y Stribeck en las transmisiones mecánicas) y la estabilidad física entre el control clásico y el control neuronal cuantizado.
- **Optimización avanzada y hardware reconfigurable:** Exploración de técnicas de cuantización consciente del entrenamiento (*Quantization-Aware Training*, QAT) y el despliegue del motor de inferencia sobre plataformas de hardware reconfigurable (FPGA / SoC heterogéneo Zynq), evaluando la máxima eficiencia energética y paralelismo espacial alcanzable al filo del manipulador.

Antecedentes

El control de robots manipuladores constituye un campo maduro de la ingeniería mecatrónica, fundamentado históricamente en la modelación cinemática y dinámica analítica de cuerpos rígidos interconectados.

Técnicas de control clásico para robots manipuladores

Los esquemas de control para manipuladores seriales se clasifican formalmente según el espacio operacional en el que se formula la ley de control (espacio articular o espacio cartesiano de la tarea) y el nivel de dependencia respecto al modelo dinámico analítico de la planta.

La aproximación más extendida en la industria consiste en el control PID (Proporcional-Integral-Derivativo) descentralizado, donde cada eje articular es desacoplado y controlado como un sistema independiente de un solo grado de libertad mediante la realimentación del error articular [1]. La simplicidad algorítmica y la facilidad de implementación analógica o digital han posicionado al control PID como el estándar por antonomasia en actuadores industriales, especialmente cuando se carece de un modelo dinámico formal de la planta [1]. No obstante, en manipuladores seriales con acoplamientos inerciales pronunciados, el control descentralizado sufre degradaciones severas ante variaciones de configuración. Para mitigar estas desviaciones, la arquitectura se asocia habitualmente a términos de prealimentación (*feedforward*) cinemática y dinámica, delegando al lazo de realimentación lineal únicamente la compensación de errores transitorios y perturbaciones residuales no modeladas [1, 15].

Cuando se dispone del modelo paramétrico explícito del manipulador:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{\tau} \tag{1}$$

resulta viable implementar estrategias avanzadas de linealización por retroalimentación, destacando el control por par calculado (*Computed Torque Control*, CTC) o dinámica inversa. Bajo un modelo dinámico exacto y las hipótesis de la formulación, este método calcula los pares requeridos para cancelar las no linealidades de Coriolis, centrífugas y gravitatorias, transformando el manipulador multivariable no lineal en un conjunto de integradores dobles desacoplados sobre los cuales se prescribe una dinámica de error lineal estable de segundo orden [1, 15]. Asimismo, enfoques de control adaptativo y robusto permiten mitigar la degradación del desempeño ante cargas útiles variables o dinámicas de fricción inciertas [1].

Transición hacia el control basado en Inteligencia Artificial (IA)

A pesar del rigor matemático de los controladores clásicos, su eficacia operativa se degrada ante dinámicas no modeladas complejas, tales como la fricción no lineal de Coulomb y Stribeck en las cajas reductoras, flexibilidad en articulaciones o efectos térmicos durante ciclos prolongados de operación. La identificación paramétrica experimental de estas no linealidades resulta costosa y altamente específica para cada robot.

Estas limitaciones han motivado la adopción de aproximadores universales de funciones basados en inteligencia artificial, primordialmente Redes Neuronales Artificiales (ANNs) y Redes Neuronales Profundas (DNNs), capaces de reconstruir relaciones no lineales complejas directamente a partir de datos experimentales de telemetría articular [3, 16]. Las arquitecturas de múltiples capas ocultas permiten capturar dinámicas baricéntricas acopladas y fenómenos histeréticos que resultan intratables mediante formulaciones analíticas cerradas [16, 2]. Sin embargo, este poder expresivo demanda un volumen masivo de operaciones de multiplicación y acumulación (MAC), provocando una severa sobrecarga computacional que compromete la frecuencia de refresco en arquitecturas de cómputo en tiempo real deterministas [4, 7, 17]. Esta tensión entre la capacidad adaptativa del modelo y las restricciones temporales del hardware embebido impulsa la investigación en técnicas sistemáticas de compresión y cuantización de redes neuronales.

Estado del arte

La optimización computacional y energética de controladores inteligentes en robótica de manipulación ha experimentado un notable avance entre los años 2021 y 2025, impulsada por la necesidad de mitigar el consumo de potencia y satisfacer las estrictas restricciones de latencia temporal en sistemas embebidos. El estado del arte actual se estructura en torno a tres ejes fundamentales:

1. Compensación dinámica y control neuronal en manipuladores: Diversas investigaciones han explorado el modelado de dinámicas inversas mediante arquitecturas neuronales para asistir o sustituir a los algoritmos analíticos clásicos. Park et al. (2025) [3] evaluaron el diseño de redes neuronales con aprendizaje por refuerzo para manipuladores robóticos tridimensionales, evidenciando una destacada capacidad adaptativa frente a incertidumbres cinemáticas, pero señalando que la evaluación de redes densas en punto flotante impone cuellos de botella energéticos que restringen su viabilidad en controladores compactos. Por su parte, Zanatta et al. (2024) [16] investigaron redes neuronales bioinspiradas y neuromórficas orientadas a tareas robóticas de alta velocidad, identificando que el desafío crítico radica en preservar la estabilidad en lazo cerrado cuando la latencia de inferencia fluctúa respecto al reloj de muestreo. En el ámbito nacional, Juárez-Lora (2024) [2] analizó el comportamiento de modelos neuromórficos sobre hardware reconfigurable, resaltando que la sincronización temporal rígida ($T_s \leq 10$ ms) resulta indispensable para evitar derivas acumulativas y oscilaciones de par sobre los eslabones mecánicos.

2. Metodologías de cuantización y compresión para hardware embebido: La reducción del ancho de palabra en parámetros y activaciones representa una técnica de optimización para acelerar la inferencia en sistemas digitales. En el estudio panorámico de Liang et al. (2021) [5], se establece que la cuantización uniforme a 8 bits (INT8) reduce la huella de memoria en un factor de $4\times$ y el consumo por operación aritmética en más de un 60 % respecto a punto flotante estándar (FP32), manteniendo errores de degradación funcional marginales cuando se emplean algoritmos de calibración adecuados. Jiang et al. (2023) [14] propusieron métodos de poda y cuantización *single-shot* optimizados para hardware digital, demostrando que la aritmética entera reduce notablemente la disipación térmica y acelera el paso hacia adelante. Asimismo, Zhang et al. (2025) [13] desarrollaron esquemas de codiseño hardware/software para redes neuronales en el borde, abordando la relación entre cuantización y recursos de ejecución. La ausencia de asignaciones dinámicas depende de la implementación y debe verificarse por separado. Complementariamente, Rezk (2022) [18] formalizó la optimización multiobjetivo de precisión mixta mediante frentes de Pareto entre error de predicción y recursos de silicio.

3. Eficiencia energética y determinismo temporal en robótica: El impacto del costo computacional de la inteligencia artificial sobre la sostenibilidad de plataformas robóticas autónomas ha sido cuantificado por de Vries y Gao (2025) [7] y Domingo-Reguero et al. (2025) [12], quienes concluyen que el despliegue de redes neuronales densas sin optimizar agota aceleradamente fuentes energéticas acotadas y eleva el perfil térmico de los microprocesadores embebidos. En paralelo, Bartolozzi et al. (2022) [6] y Kudithipudi et al. (2025) [17] han enfatizado que la inteligencia embebida moderna

exige acoplar estrechamente la representación computacional con la dinámica física del mecanismo para superar la barrera impuesta por el cuello de botella de Von Neumann.

Síntesis y brecha de investigación identificada: A partir de la literatura analizada, se concluye que mientras existen abundantes desarrollos de redes profundas no optimizadas ejecutadas en estaciones de cómputo de alta gama, y paralelamente estudios teóricos de cuantización aplicados a conjuntos de datos estáticos de visión por computadora o procesamiento del lenguaje, se identifica una evidente brecha técnica en la integración y validación experimental de modelos neuronales cuantizados a 8 bits (INT8) desplegados directamente dentro del bucle de control en tiempo real ($100\ \text{Hz} - 500\ \text{Hz}$) de un robot manipulador industrial bajo estándares modernos como ROS 2 y ros2_control. El presente Trabajo Terminal aborda directamente dicha brecha mediante una arquitectura mecatrónica completa que integra el modelo del robot en simulación, la adquisición de datos normalizados, el entrenamiento de una red de 42 entradas, la cuantización dinámica post-entrenamiento (*Dynamic PTQ*) a INT8 y la verificación cuantitativa del compromiso entre precisión de seguimiento articular (RMSE) y eficiencia de cómputo en C++.

Organización del documento

El presente reporte de Trabajo Terminal 1 se estructura en cuatro capítulos fundamentales y las secciones finales de conclusiones y anexos:

- **Capítulo 1: Marco de Referencia.** Expone el marco teórico que sustenta la cinemática y dinámica analítica de manipuladores mediante el algoritmo recursivo de Newton-Euler (RNEA), los principios del control por par calculado, las bases de las redes neuronales artificiales, las formulaciones matemáticas de cuantización (PTQ y QAT) y el marco procedimental ágil (*Scrum*) adaptado a la ingeniería mecatrónica.

- **Capítulo 2: Diseño del Sistema.** Desarrolla la ingeniería conceptual y de detalle del sistema, abarcando la matriz de requerimientos técnicos, la arquitectura modular mecatrónica (M_1, M_2, M_3, M_4), las matrices de selección y ponderación (AHP/Pugh), la derivación del modelo cinemático y dinámico para el robot UR3e, la topología de la red neuronal predictora de torque total y el plan experimental de evaluación.

- **Capítulo 3: Implementación del Sistema.** Describe la materialización técnica y la verificación de pruebas funcionales de cada módulo en el entorno de simulación física (*Sim-to-Real*) bajo Gazebo Harmonic y ROS 2 Jazzy, la programación del solver RNEA en C++, el pipeline de telemetría con filtrado Savitzky-Golay, el entrenamiento de la red en PyTorch y su exportación a ONNX.

- **Capítulo 4: Análisis de Resultados.** Presenta la evaluación experimental cuantitativa del sistema en bucle cerrado a 500 Hz, contrastando la precisión de seguimiento articular (RMSE) entre control clásico, neuronal FP32 y neuronal

INT8, el presupuesto temporal y dispersión de latencia en tiempo real, la atenuación de la huella de memoria (74,43 %) y la matriz de cumplimiento técnico de requerimientos.

- **Conclusiones y Trabajo a Futuro.** Sintetiza los resultados y aportaciones alcanzadas en esta primera etapa e instaura la ruta crítica para la transferencia y validación sobre el brazo robótico físico en la fase subsecuente (TT2).
- **Anexos.** Integra la gestión y planeación de actividades en *Sprints* de Scrum, la infraestructura computacional y el desglose de recursos.

1. Marco de Referencia

1.1. Marco teórico

Este proyecto se fundamenta en la intersección de varias disciplinas clave: la robótica de manipulación, el control automático, la inteligencia artificial (específicamente las redes neuronales) y las técnicas de optimización de modelos para la eficiencia computacional y energética.

1.1.1. Modelado y Control de Robots Manipuladores

Los robots manipuladores son sistemas mecánicos articulados cuyo movimiento y comportamiento se describen mediante modelos matemáticos precisos.

Cinemática La cinemática estudia la geometría pura del movimiento del manipulador con respecto a un sistema de coordenadas inercial de referencia, prescindiendo de las fuerzas y momentos que originan el desplazamiento. La descripción espacial del efector final respecto a la base se define formalmente en el grupo especial euclidiano $SE(3) = \mathbb{R}^3 \times SO(3)$.

La **cinemática directa** establece la relación funcional entre las coordenadas generalizadas articulares $\boldsymbol{q} = [q_1, q_2, \ldots, q_n]^T \in \mathbb{R}^n$ y la pose (posición cartesiana y orientación) del efector final mediante la función no lineal $\boldsymbol{x} = f(\boldsymbol{q})$. Para manipuladores seriales de cadena abierta, esta relación se parametriza sistemáticamente mediante la convención de **Denavit-Hartenberg modificada (Craig)** [19], en la cual el sistema de coordenadas del marco i se fija rígidamente al eslabón i, y la transformación homogénea respecto al marco previo $i-1$ depende de cuatro parámetros geométricos: la longitud del eslabón a_{i-1}, el ángulo de torsión α_{i-1}, el desplazamiento articular d_i y el ángulo de rotación θ_i (siendo variable $\theta_i = q_i$ para pares de revolución). La matriz de transformación homogénea $\boldsymbol{T}_i^{i-1}(q_i) \in SE(3)$ adopta la forma:

$$\boldsymbol{T}_i^{i-1}(q_i) = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_{i-1} \\ \sin\theta_i \cos\alpha_{i-1} & \cos\theta_i \cos\alpha_{i-1} & -\sin\alpha_{i-1} & -d_i \sin\alpha_{i-1} \\ \sin\theta_i \sin\alpha_{i-1} & \cos\theta_i \sin\alpha_{i-1} & \cos\alpha_{i-1} & d_i \cos\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2}$$

La pose acumulada del extremo del robot respecto a la base inercial se obtiene mediante el producto encadenado de matrices de transformación consecutivas:

$$\boldsymbol{T}_n^0(\boldsymbol{q}) = \boldsymbol{T}_1^0(q_1)\boldsymbol{T}_2^1(q_2)\cdots\boldsymbol{T}_n^{n-1}(q_n) = \begin{bmatrix} \boldsymbol{R}_n^0(\boldsymbol{q}) & \boldsymbol{p}_n^0(\boldsymbol{q}) \\ \boldsymbol{0}_{1\times 3} & 1 \end{bmatrix} \tag{3}$$

donde $\boldsymbol{R}_n^0(\boldsymbol{q}) \in SO(3)$ describe la orientación ortonormal del efector final y $\boldsymbol{p}_n^0(\boldsymbol{q}) \in \mathbb{R}^3$ define su posición tridimensional.

Por su parte, la **cinemática diferencial** vincula las velocidades articulares $\dot{\boldsymbol{q}} \in \mathbb{R}^n$ con las velocidades lineales $\dot{\boldsymbol{p}}_e$ y angulares $\boldsymbol{\omega}_e$ del efector final mediante el operador **Jacobiano geométrico** $\boldsymbol{J}(\boldsymbol{q}) \in \mathbb{R}^{6\times n}$ [1]:

$$\boldsymbol{v}_e = \begin{bmatrix} \dot{\boldsymbol{p}}_e \\ \boldsymbol{\omega}_e \end{bmatrix} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}} = \begin{bmatrix} \boldsymbol{J}_v(\boldsymbol{q}) \\ \boldsymbol{J}_\omega(\boldsymbol{q}) \end{bmatrix} \dot{\boldsymbol{q}} \tag{4}$$

donde cada columna $\boldsymbol{J}_i(\boldsymbol{q})$ para una articulación de revolución i se define vectorialmente como $\boldsymbol{J}_i(\boldsymbol{q}) = [\boldsymbol{z}_{i-1} \times (\boldsymbol{p}_e - \boldsymbol{p}_{i-1}); \boldsymbol{z}_{i-1}]$, siendo $\boldsymbol{z}_{i-1}$ el vector unitario del eje de rotación y $\boldsymbol{p}_{i-1}$ la posición del origen del marco $i-1$. Las configuraciones articulares $\boldsymbol{q}^*$ en las cuales $\det(\boldsymbol{J}(\boldsymbol{q}^*)) = 0$ (o $\text{rango}(\boldsymbol{J}(\boldsymbol{q}^*)) < 6$) constituyen **singularidades cinemáticas**, en las que el robot pierde la capacidad de desplazarse instantáneamente en ciertas direcciones del espacio cartesiano y las demandas de velocidad articular ante trayectorias continuas tienden teóricamente a infinito.

La **cinemática inversa** consiste en determinar el vector articular $\boldsymbol{q}$ requerido para alcanzar una pose cartesiana deseada $\boldsymbol{x}_d \in SE(3)$, es decir, $\boldsymbol{q} = f^{-1}(\boldsymbol{x}_d)$. Para robots antropomórficos y cobots de 6 grados de libertad como el UR3e, la existencia de simetrías geométricas permite resolver la cinemática inversa de manera analítica en forma cerrada, arrojando hasta 8 configuraciones articulares posibles para una misma pose cartesiana [19, 1].

Dinámica El modelo dinámico de un robot manipulador relaciona los pares/fuerzas articulares aplicados $\boldsymbol{\tau} \in \mathbb{R}^n$ con las posiciones $\boldsymbol{q}$, velocidades $\dot{\boldsymbol{q}}$ y aceleraciones $\ddot{\boldsymbol{q}}$ articulares. La ecuación general de la dinámica en espacio de estados para un manipulador rígido de n grados de libertad se expresa tradicionalmente mediante la formulación de Lagrange-Euler [15, 1]:

$$\boldsymbol{\tau} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) \tag{5}$$

donde:

- $\boldsymbol{M}(\boldsymbol{q}) \in \mathbb{R}^{n\times n}$ es la matriz de masa e inercia del manipulador, simétrica y estrictamente definida positiva ($\boldsymbol{M}(\boldsymbol{q}) = \boldsymbol{M}^T(\boldsymbol{q}) > 0$).
- $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) \in \mathbb{R}^{n\times n}$ es la matriz de efectos centrífugos y de Coriolis, la cual satisface la propiedad fundamental de que la matriz $\dot{\boldsymbol{M}}(\boldsymbol{q}) - 2\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ es antisimétrica (*skew-symmetric*).
- $\boldsymbol{g}(\boldsymbol{q}) \in \mathbb{R}^n$ es el vector de fuerzas y pares generados por la atracción gravitatoria sobre los eslabones.

Si bien la formulación explícita de Lagrange-Euler permite el análisis analítico formal de estabilidad, el cálculo simbólico de $\boldsymbol{M}(\boldsymbol{q})$ y $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ presenta una complejidad computacional de orden $\mathcal{O}(n^4)$ o $\mathcal{O}(n^3)$, lo cual resulta potencialmente costoso según la formulación y la implementación. Por esta razón, para el cálculo en tiempo real de la dinámica inversa se utiliza el **Algoritmo Recursivo de Newton-Euler (RNEA)**, desarrollado originalmente por Luh, Walker y Paul [1, 15]. El RNEA reduce la complejidad a un orden lineal $\mathcal{O}(n)$, propagando las cinemáticas desde la base hasta el efector final y los balances de fuerza en sentido inverso.

Bajo la convención de parámetros Denavit-Hartenberg modificados (Craig), el algoritmo se divide en dos fases iterativas:

1. Propagación Cinemática Hacia Adelante ($i = 1, \ldots, n$):
Partiendo de las condiciones iniciales en la base inmóvil ($\boldsymbol{\omega}_0 = \mathbf{0}$, $\dot{\boldsymbol{\omega}}_0 = \mathbf{0}$), y considerando la aceleración gravitacional simulada en el marco inercial $\dot{\boldsymbol{v}}_0 = [0, 0, 9{,}81]^T$ m/s^2 para incorporar la gravedad intrínsecamente sin cálculos adicionales, se propagan las velocidades y aceleraciones hacia el extremo del robot:

$$\boldsymbol{\omega}_i = (\boldsymbol{R}_i^{i-1})^T \boldsymbol{\omega}_{i-1} + \dot{q}_i \boldsymbol{z}_0 \tag{6}$$

$$\dot{\boldsymbol{\omega}}_i = (\boldsymbol{R}_i^{i-1})^T \dot{\boldsymbol{\omega}}_{i-1} + \left((\boldsymbol{R}_i^{i-1})^T \boldsymbol{\omega}_{i-1}\right) \times (\dot{q}_i \boldsymbol{z}_0) + \ddot{q}_i \boldsymbol{z}_0 \tag{7}$$

$$\dot{\boldsymbol{v}}_i = (\boldsymbol{R}_i^{i-1})^T \dot{\boldsymbol{v}}_{i-1} + \dot{\boldsymbol{\omega}}_i \times \boldsymbol{P}_i^{i-1} + \boldsymbol{\omega}_i \times \left(\boldsymbol{\omega}_i \times \boldsymbol{P}_i^{i-1}\right) \tag{8}$$

donde $\boldsymbol{R}_i^{i-1}$ es la matriz de orientación del marco i respecto a $i-1$, $\boldsymbol{P}_i^{i-1}$ es el vector de traslación del origen $i-1$ al i expresado en el marco i, y $\boldsymbol{z}_0 = [0, 0, 1]^T$ define el eje unitario de rotación articular.

2. Balance Dinámico Hacia Atrás ($i = n, \ldots, 1$):
Calculada la cinemática de cada eslabón, se obtiene la aceleración lineal en su respectivo centro de masa $\boldsymbol{r}_{c,i}$:

$$\dot{\boldsymbol{v}}_{c,i} = \dot{\boldsymbol{v}}_i + \dot{\boldsymbol{\omega}}_i \times \boldsymbol{r}_{c,i} + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \boldsymbol{r}_{c,i}) \tag{9}$$

Aplicando las leyes de Newton y Euler, se determinan la fuerza inercial neta $\boldsymbol{F}_i$ y el momento inercial baricéntrico $\boldsymbol{N}_i$ sobre cada eslabón:

$$\boldsymbol{F}_i = m_i \dot{\boldsymbol{v}}_{c,i} \tag{10}$$

$$\boldsymbol{N}_i = \boldsymbol{I}_i \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times (\boldsymbol{I}_i \boldsymbol{\omega}_i) \tag{11}$$

donde m_i y $\boldsymbol{I}_i$ representan la masa y el tensor de inercia baricéntrico del eslabón i. Finalmente, el balance de fuerzas $\boldsymbol{f}_i$ y momentos $\boldsymbol{n}_i$ transmitidos a través de la articulación se propaga recursivamente desde el último eslabón hasta la base:

$$\boldsymbol{f}_i = \boldsymbol{R}_{i+1}^i \boldsymbol{f}_{i+1} + \boldsymbol{F}_i \tag{12}$$

$$\boldsymbol{n}_i = \boldsymbol{N}_i + \boldsymbol{R}_{i+1}^i \boldsymbol{n}_{i+1} + \boldsymbol{r}_{c,i} \times \boldsymbol{F}_i + \boldsymbol{P}_{i+1}^i \times \left(\boldsymbol{R}_{i+1}^i \boldsymbol{f}_{i+1}\right) \tag{13}$$

asumiendo que sobre el efector final no actúan fuerzas externas no compensadas ($\boldsymbol{f}_{n+1} = \mathbf{0}, \boldsymbol{n}_{n+1} = \mathbf{0}$). El par dinámico que debe suministrar el actuador de la articulación i corresponde a la proyección del momento $\boldsymbol{n}_i$ sobre el eje de rotación:

$$\tau_{rnea,i} = \boldsymbol{n}_i^T \boldsymbol{z}_0 \tag{14}$$

Esta formulación recursiva se evalúa en el nodo de control en C++, configurado nominalmente a 500 Hz ($T_s = 2$ ms). Su salida proporciona el término dinámico del controlador maestro RNEA + PD y una característica de entrada para el predictor neuronal de torque total; no acredita un desacoplamiento experimental de la planta.

La Figura 2 resume conceptualmente las dos pasadas de RNEA: propagación cinemática desde la base y transmisión de fuerzas y momentos desde el extremo. Se ilustra el caso de base fija y sin carga externa; las magnitudes de cada operación deben expresarse en marcos compatibles. La nota de 151.2 μs corresponde al cronometraje integrado RNEA + PD y no a una medición aislada del algoritmo.

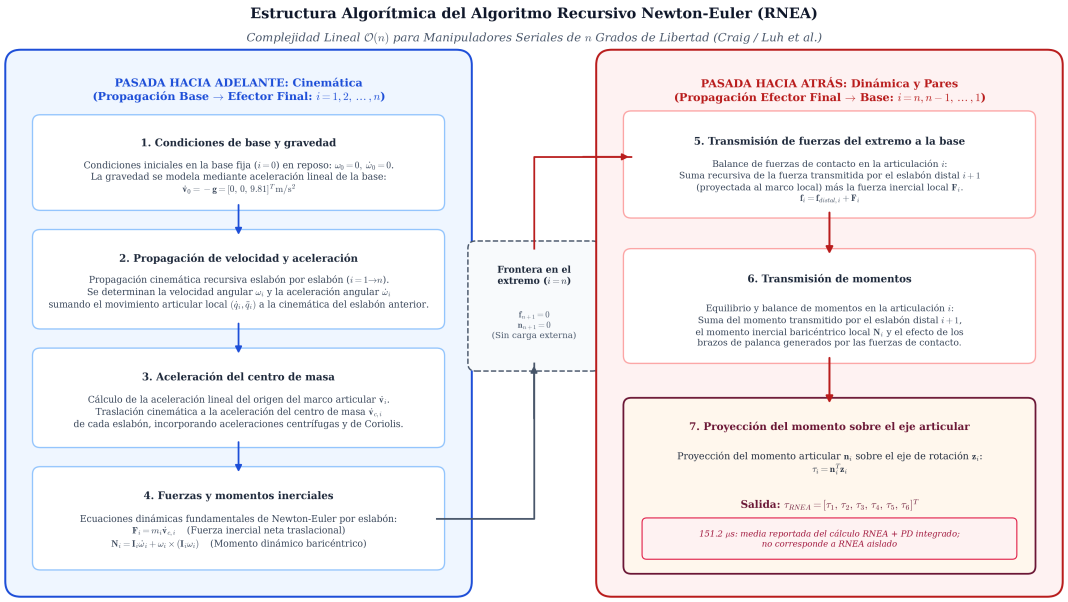


Figura 2: Esquema conceptual del Algoritmo Recursivo de Newton–Euler (RNEA): propagación hacia adelante ($i = 1 \ldots n$), balance de esfuerzos hacia atrás ($i = n \ldots 1$) y proyección del momento sobre el eje articular. El cronometraje anotado corresponde al controlador RNEA + PD integrado.

Estrategias de Control Clásicas

- **Control PID Independiente por Articulación:** Para cada articulación i, la ley de control es:

$$\tau_i(t) = K_{p_i} e_i(t) + K_{i_i} \int_0^t e_i(\lambda) d\lambda + K_{d_i} \frac{de_i(t)}{dt} \tag{15}$$

donde $e_i(t) = q_{d_i}(t) - q_i(t)$ es el error de la articulación i, y K_p, K_i, K_d son las ganancias proporcional, integral y derivativa respectivamente. A menudo se complementa con un término de prealimentación (*feedforward*) τ_{ff_i} para mejorar el seguimiento de trayectorias[1]:

$$\tau_i = \tau_{ff_i} + K_{p_i} e_i + K_{i_i} \int e_i dt + K_{d_i} \dot{e}_i \tag{16}$$

- **Control por Par Calculado (*Computed Torque Control*):** Esta técnica utiliza el modelo dinámico inverso para linealizar y desacoplar la planta robótica mediante retroalimentación de estados. Si se dispone de un modelo dinámico estimado $\hat{\boldsymbol{M}}(\boldsymbol{q})$, $\hat{\boldsymbol{c}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$, y $\hat{\boldsymbol{g}}(\boldsymbol{q})$, la ley de control por par computado se define como:

$$\boldsymbol{\tau} = \hat{\boldsymbol{M}}(\boldsymbol{q})\boldsymbol{a}_{cmd} + \hat{\boldsymbol{c}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) + \hat{\boldsymbol{g}}(\boldsymbol{q}) \tag{17}$$

donde $\boldsymbol{a}_{cmd}$ es el vector de aceleración de control sintetizado mediante una ley lineal proporcional-derivativa sobre el error de seguimiento $\boldsymbol{e} = \boldsymbol{q}_d - \boldsymbol{q}$:

$$\boldsymbol{a}_{cmd} = \ddot{\boldsymbol{q}}_d + \boldsymbol{K}_d(\dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}}) + \boldsymbol{K}_p(\boldsymbol{q}_d - \boldsymbol{q}) \tag{18}$$

siendo $\boldsymbol{K}_p, \boldsymbol{K}_d$ matrices diagonales definidas positivas para la realimentación de aceleración. Bajo un modelo exacto y sin fricción no modelada, la sustitución de la ley teórica en la dinámica del robot conduce a:

$$\ddot{\boldsymbol{e}} + \boldsymbol{K}_d\dot{\boldsymbol{e}} + \boldsymbol{K}_p\boldsymbol{e} = \boldsymbol{0}. \tag{19}$$

Esta ecuación describe el caso ideal de la formulación teórica, no una estabilidad demostrada del controlador del proyecto. La implementación TT1 suma la realimentación PD en unidades de torque a RNEA evaluado con la aceleración deseada, conforme a la Ecuación (39); no introduce el término PD dentro de $\boldsymbol{a}_{cmd}$. Por ello, sus ganancias y resultados no se equiparan directamente con la ley teórica anterior.

En la Figura 3 se esquematiza el lazo cerrado de control por par calculado, como referencia teórica. La arquitectura implementada se presenta en la Figura 6; no debe identificarse con este esquema de realimentación de aceleración.

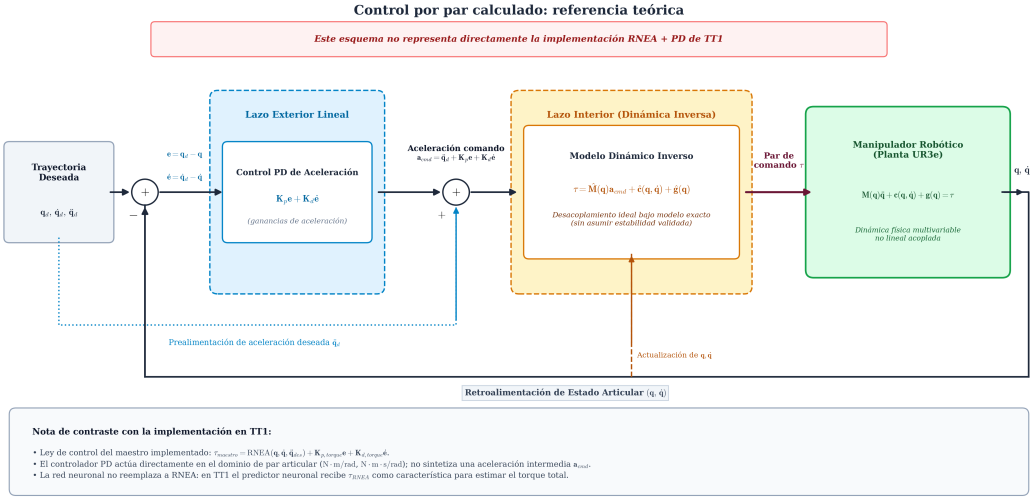


Figura 3: Control por par calculado como referencia teórica: el PD genera una corrección de aceleración antes de la dinámica inversa. El desacoplamiento corresponde al caso ideal de modelo exacto. La nota inferior distingue esta formulación de RNEA + PD en torque implementado en TT1.

1.1.2. Redes Neuronales Artificiales para el Control Robótico

Las Redes Neuronales Artificiales (ANN), y sus variantes profundas (DNN), representan una familia de modelos computacionales inspirados en la interconexión sináptica biológica, capaces de aprender relaciones no lineales complejas directamente a partir de datos experimentales. En el ámbito del control robótico, su aplicación resulta especialmente provechosa para modelar dinámicas físicas difíciles de parametrizar analíticamente, tales como fricciones no lineales en los reductores armónicos, flexibilidades estructurales, efectos térmicos y perturbaciones mecánicas acopladas [3, 16].

Fundamentación Teórica y Teorema de Aproximación Universal: La justificación formal para emplear redes neuronales como aproximadores de dinámica robótica descansa en el **Teorema de Aproximación Universal**, demostrado por Cybenko (1989) [20] para funciones de activación sigmoideas y generalizado por Hornik et al. (1989) [21] para funciones de activación continuas no lineales en arquitecturas multicapa. Este teorema establece que cualquier función continua $f : \mathcal{K} \subset \mathbb{R}^d \to \mathbb{R}^m$ definida sobre un conjunto compacto $\mathcal{K}$ puede ser aproximada uniformemente con cualquier grado de exactitud deseado $\epsilon > 0$ mediante una red neuronal hacia adelante (*feedforward*) con un número finito de neuronas ocultas:

$$\sup_{\boldsymbol{x}\in\mathcal{K}} \|f(\boldsymbol{x}) - \hat{f}(\boldsymbol{x};\boldsymbol{\theta})\| < \epsilon \tag{20}$$

donde $\boldsymbol{\theta}$ representa el conjunto de parámetros entrenables (pesos y sesgos). En TT1 se utiliza la red como aproximador del torque total del controlador maestro sobre el dominio de datos disponible. El teorema expresa una capacidad de aproximación bajo sus hipótesis; no demuestra por sí solo la precisión obtenida ni la estabilidad del sistema en lazo cerrado.

Estructura del Perceptrón Multicapa y Funciones de Activación: En un perceptrón multicapa (*Multilayer Perceptron*, MLP) de L capas, la propagación hacia adelante en la capa $l \in \{1, \ldots, L\}$ se rige por la combinación afín de sus entradas y una transformación no lineal:

$$\boldsymbol{z}^{[l]} = \boldsymbol{W}^{[l]}\boldsymbol{a}^{[l-1]} + \boldsymbol{b}^{[l]} \tag{21}$$

$$\boldsymbol{a}^{[l]} = \sigma(\boldsymbol{z}^{[l]}) \tag{22}$$

donde $\boldsymbol{a}^{[0]} = \boldsymbol{x} \in \mathbb{R}^{d_{in}}$ es el vector de características de entrada, $\boldsymbol{W}^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$ es la matriz de pesos sinápticos, $\boldsymbol{b}^{[l]} \in \mathbb{R}^{n_l}$ es el vector de sesgo, y $\sigma(\cdot)$ es la función de activación elemento a elemento. Para las capas ocultas intermedias, la función predominante es la Unidad Lineal Rectificada (*Rectified Linear Unit*, ReLU):

$$\mathrm{ReLU}(z) = \mathrm{máx}(0, z) \tag{23}$$

por su cálculo elemento a elemento mediante un umbral y su derivada constante en la región activa ($z > 0$). Para la capa de salida ($l = L$), se emplea una función de activación lineal ($\boldsymbol{a}^{[L]} = \boldsymbol{z}^{[L]}$), permitiendo predecir pares totales continuos $\hat{\boldsymbol{\tau}} \in \mathbb{R}^n$ en cualquier cuadrante de operación.

Función de Pérdida y Optimización Estocástica (Adam): El ajuste de los parámetros $\boldsymbol{\theta} = \{\boldsymbol{W}^{[l]}, \boldsymbol{b}^{[l]}\}_{l=1}^{L}$ se lleva a cabo mediante entrenamiento supervisado minimizando la discrepancia entre el par predicho $\hat{\boldsymbol{\tau}}$ y el torque total objetivo $\boldsymbol{\tau}^*$ generado por el controlador maestro. Para ello se emplea la función de pérdida del Error Cuadrático Medio (*Mean Squared Error*, MSE):

$$\mathcal{L}_{MSE}(\boldsymbol{\theta}) = \frac{1}{6N} \sum_{k=1}^{N} \|\boldsymbol{y}^{*(k)} - \hat{\boldsymbol{y}}(\boldsymbol{x}^{(k)}; \boldsymbol{\theta})\|_2^2 \tag{24}$$

Aquí N es el número de muestras del lote; $\boldsymbol{y}^*$ y $\hat{\boldsymbol{y}}$ son las seis salidas objetivo y predichas en la escala estandarizada empleada en entrenamiento. El promedio incluye muestras y salidas, conforme a `nn.MSELoss()`. La actualización de pesos se ejecuta empleando el optimizador **Adam** (*Adaptive Moment Estimation*) [22], el cual calcula estimaciones móviles del primer momento (media) y segundo momento no centrado (varianza no centrada) del gradiente $\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$:

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1)\boldsymbol{g}_t, \quad \boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2)\boldsymbol{g}_t^2 \tag{25}$$

con factores de corrección de sesgo $\hat{\boldsymbol{m}}_t = \boldsymbol{m}_t/(1 - \beta_1^t)$ y $\hat{\boldsymbol{v}}_t = \boldsymbol{v}_t/(1 - \beta_2^t)$, actualizando los parámetros mediante:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon} \hat{\boldsymbol{m}}_t \tag{26}$$

donde α es la tasa de aprendizaje ($\alpha = 10^{-3}$ por defecto), $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, y $\epsilon = 10^{-8}$ evita divisiones por cero.

1.1.3. Determinismo Computacional y Tiempo Real Estricto en Robótica

En el control automático de robots manipuladores, el cumplimiento temporal influye en el comportamiento del lazo cerrado. Los requisitos de tiempo real estricto (*Hard Real-Time*) exigen respetar los plazos especificados; el efecto de un incumplimiento depende del sistema y de su política de fallo. Los tiempos observados en TT1 no acreditan ese tipo de garantía temporal. La tasa configurada en `controller_manager` establece la frecuencia nominal del ciclo de lectura, actualización y escritura; su configuración no demuestra el cumplimiento de todos los plazos.

Para un periodo de muestreo nominal T_s, el presupuesto de cómputo se expresa como una condición de diseño:

$$t_{ciclo} = t_{lectura} + t_{rnea} + t_{nn} + t_{escritura} + t_{overhead} \leq T_s \tag{27}$$

La duración de cómputo t_{ciclo} debe distinguirse del intervalo entre inicios de actualizaciones del controlador. Para caracterizar el ***jitter* del periodo**, se define:

$$T_k = t_k - t_{k-1}, \qquad j_k = T_k - T_s \tag{28}$$

donde t_k es la marca temporal de inicio de la actualización k, T_k es el intervalo efectivo entre actualizaciones, T_s es el periodo nominal y j_k es su desviación temporal, expresada en segundos. Su evaluación requiere una serie de marcas temporales del lazo interno

obtenidas con una misma referencia de reloj, identificando si corresponde a tiempo de simulación o de ejecución en el equipo. La desviación estándar y el máximo absoluto de j_k describen aspectos distintos de la regularidad del periodo; no deben confundirse con la dispersión del tiempo de inferencia ni con diferencias entre duraciones de cómputo consecutivas.

En este proyecto se conserva el criterio de diseño $\max_k |j_k| < 100\,\mu$s sobre la serie de actualizaciones evaluada, como requisito pendiente de verificación. No constituye un umbral universal de estabilidad ni un resultado alcanzado en TT1. Los cronómetros disponibles caracterizan intervalos de cómputo; la recopilación de datos a 100 Hz tampoco demuestra la regularidad de todas las actualizaciones internas a 500 Hz. Sin una serie temporal trazable del lazo interno, no se asigna un valor experimental al jitter del periodo.

Para reducir fuentes de variación temporal se plantean tres directrices de diseño de software, cuyo cumplimiento debe verificarse en la implementación:

1. **Cero Asignación Dinámica en el Lazo Crítico:** Prohibición estricta de llamadas al asignador del sistema operativo (`malloc`, `free`, `new`, `delete`) o redimensionamiento de contenedores (`std::vector::resize`) dentro de la función periódica `update()`. Como directriz, se propone preasignar la memoria de tensores y estructuras de cálculo durante la fase de configuración del ciclo de vida (`on_configure()`).

2. **Separación de tareas y comunicación:** Se propone separar las tareas de registro del ciclo crítico y utilizar mecanismos de intercambio adecuados entre callbacks y control. El uso de `RealtimeBuffer` para recibir referencias no demuestra por sí solo ausencia de bloqueos, inversión de prioridades ni publicación asíncrona de telemetría.

3. **Reducción del tamaño de almacenamiento:** La cuantización puede reducir el tamaño de los tensores de pesos. La residencia en caché, el tráfico de memoria y su efecto temporal dependen de la plataforma y del runtime; requieren medición específica.

1.1.4. Estrategias de Optimización de Redes Neuronales

La alta demanda computacional y energética de las DNN, detallada en secciones previas, se convierte en el principal problema a tratar en este proyecto. El objetivo general es reducir la complejidad del modelo (en términos de parámetros y operaciones), el tamaño en memoria, la latencia de inferencia y el consumo energético, manteniendo al mismo tiempo la precisión de la tarea lo más alta posible. Las principales estrategias incluyen la poda de redes, el desarrollo de arquitecturas eficientes, el uso de paradigmas alternativos como las redes neuronales de impulsos, y la cuantización de redes neuronales, siendo esta última el foco central de este proyecto.

Poda de Redes (*Neural Pruning*) La poda de redes neuronales se basa en la observación de que muchos modelos de DNNs están sobreparametrizados, conteniendo

un gran número de pesos o neuronas que contribuyen poco o nada al rendimiento final [5]. El objetivo de la poda es identificar y eliminar estos elementos redundantes.

Tipos de Poda:

- **Poda No Estructurada (*Unstructured Pruning*):** Consiste en eliminar pesos individuales de la red. Se establece un criterio de importancia (saliencia) para cada peso w_{ij}, y aquellos por debajo de un umbral son eliminados (su valor se fija a cero). Formalmente, si $\mathcal{T}$ es el umbral:

$$w'_{ij} = \begin{cases} w_{ij} & \text{si saliencia}(w_{ij}) > \mathcal{T} \\ 0 & \text{en caso contrario} \end{cases} \tag{29}$$

- **Poda Estructurada (*Structured Pruning*):** En lugar de eliminar pesos individuales, se eliminan grupos de pesos o estructuras enteras, como neuronas completas, filtros convolucionales completos o incluso capas. Por ejemplo, se puede calcular la importancia de un filtro convolucional (ej., sumando las magnitudes de sus pesos o usando su norma L1/L2) y eliminar los filtros menos importantes. Esto resulta en redes más pequeñas y con una estructura regular, lo que facilita la aceleración en hardware estándar, ya que las dimensiones de las matrices y tensores se reducen directamente [5].

Proceso de Poda: La poda puede realizarse de forma iterativa: se entrena la red, se podan los elementos menos importantes, y luego se re-entrena (*fine-tuning*) la red podada para recuperar la precisión que se haya podido perder. Este ciclo de podar y re-entrenar puede repetirse varias veces. Alternativamente, existen métodos de poda *one-shot* que intentan realizar la poda en una sola etapa, a veces durante el mismo entrenamiento o inmediatamente después, como la propuesta de "*Single-shot pruning and quantization*" de Jiang [14].

Redes Neuronales de Impulsos (*Spiking Neural Networks* - SNN) Las SNN representan un paradigma de computación neuronal alternativo, inspirado más directamente en el funcionamiento de las neuronas biológicas. Se las considera la "tercera generación" de redes neuronales y se caracterizan por procesar información mediante secuencias de eventos discretos en el tiempo, denominados impulsos (*spikes*).

Modelos Neuronales de SNN: A diferencia de las neuronas de las ANN que calculan una salida analógica (o un valor de activación continuo), las neuronas en las SNN integran las señales de entrada (*spikes* ponderados de otras neuronas) a lo largo del tiempo, y generan un *spike* de salida solo cuando su potencial de membrana alcanza un umbral específico.

- ***Leaky Integrate-and-Fire* (LIF):** Es uno de los modelos más simples y computacionalmente eficientes. El potencial de membrana $v(t)$ de una neurona LIF evo-

luciona según la ecuación diferencial:

$$\tau_m \frac{dv(t)}{dt} = -(v(t) - v_{rest}) + R_{in} I(t) \tag{30}$$

donde τ_m es la constante de tiempo de la membrana, v_{rest} es el potencial de reposo, R_{in} es la resistencia de entrada, e $I(t)$ es la corriente sináptica total de entrada. Cuando $v(t)$ alcanza el umbral v_{th}, la neurona emite un *spike* y $v(t)$ se reinicia (ej., a $v_{reset} < v_{th}$) y puede entrar en un periodo refractario.

- **Modelo de Izhikevich:** Es un modelo bidimensional que puede reproducir una rica variedad de patrones de disparo observados en neuronas biológicas (ej., *regular spiking*, *bursting*, *chattering*, *fast spiking*, etc.) con solo cuatro parámetros ajustables (a, b, c, d) y un par de ecuaciones diferenciales acopladas:

$$\frac{dv}{dt} = 0{,}04v^2 + 5v + 140 - u + I \tag{31}$$

$$\frac{du}{dt} = a(bv - u) \tag{32}$$

Con la condición de reseteo post-impulso: si $v(t) \geq v_{peak}$ (usualmente 30 mV), entonces $v(t) \leftarrow c$ y $u(t) \leftarrow u(t) + d$. La variable u representa una variable de recuperación de la membrana. Este modelo es utilizado, por ejemplo, en la tesis del Dr. Juárez Lora para la implementación de una plataforma neuromórfica [2].

Las SNN ofrecen un gran potencial para la eficiencia energética, especialmente cuando se implementan en *hardware* neuromórfico dedicado (ej., Loihi de Intel, TrueNorth de IBM, SpiNNaker) que está diseñado para manejar la comunicación basada en eventos y la computación dispersa en el tiempo y el espacio [23].

Redes Neuronales Cuantizadas (*Quantized Neural Networks* - QNN) La cuantización es una técnica de compresión y optimización que reduce la precisión numérica utilizada para representar los pesos $\boldsymbol{w}$ y/o las activaciones $\boldsymbol{a}$ de una red neuronal. Esta es la estrategia de optimización central de este proyecto.

Principios Fundamentales y Proceso de Cuantización: El objetivo es transformar los parámetros y activaciones, típicamente almacenados como números de punto flotante de 32 bits (FP32), a representaciones de enteros de baja precisión (ej., INT8, INT4) o formatos de punto fijo. Una cuantización **uniforme afín** (asimétrica) de un valor real x a un entero cuantizado x_q de k bits se define mediante una escala $S \in \mathbb{R}^+$ y un punto cero $Z \in \mathbb{Z}$ (que es un entero de k bits y representa el mapeo del valor real 0,0):

$$x_q = \text{clamp}\left(\text{round}\left(\frac{x}{S}\right) + Z, \quad N_{min}^{(k)}, N_{max}^{(k)}\right) \tag{33}$$

donde round$(\cdot)$ es el redondeo al entero más cercano, y clamp$(\cdot, N_{min}^{(k)}, N_{max}^{(k)})$ asegura que x_q permanezca dentro del rango representable por k bits (ej., $[0, 2^k - 1]$ para k bits sin signo, o $[-2^{k-1}, 2^{k-1} - 1]$ para k bits con signo). La escala S determina el tamaño del

paso de cuantización, y Z permite que el rango de cuantización no sea necesariamente simétrico alrededor del cero real. La operación inversa (de-cuantización) para obtener una aproximación de x es:

$$\hat{x} = S(x_q - Z) \tag{34}$$

En la cuantización **uniforme simétrica**, el punto cero Z se asume como 0 (o el punto cero de la representación entera es el cero real), y el mapeo es simétrico alrededor del origen. La escala S se determina a partir del valor absoluto máximo del rango. La determinación de los parámetros S y Z es crucial:

- ***Min-Max Quantization*:** $S = \frac{\text{máx}(x_{tensor}) - \text{mín}(x_{tensor})}{N_{max}^{(k)} - N_{min}^{(k)}}$ y Z se deriva para alinear el cero.
- Otros métodos buscan minimizar el error de cuantización, por ejemplo, minimizando la divergencia KL entre la distribución de los valores FP32 y los valores cuantizados, o utilizando percentiles para ser robustos a outliers.

La cuantización puede aplicarse con diferente granularidad:

- **Por Tensor:** Un único par (S, Z) para todo un tensor (ej., todos los pesos de una capa).
- **Por Canal:** Un par (S, Z) diferente para cada canal de salida de un filtro convolucional o para cada fila/columna de una matriz de pesos. Generalmente ofrece mejor precisión que la cuantización por tensor, especialmente para capas con distribuciones de pesos muy variables entre canales.

Tipos de Datos de Baja Precisión y Formatos de Punto Fijo: Mientras que INT8 es un objetivo común y bien soportado, la investigación explora precisiones aún menores como INT4, INT2 (ternario: {-1, 0, 1}) o INT1 (binario: {-1, 1} o {0, 1}) [24]. La representación de **punto fijo** $Q(I, F)$ denota un número con I bits para la parte entera y F bits para la parte fraccionaria (más un bit de signo si es necesario). El valor real es $X_{int} \cdot 2^{-F}$, donde X_{int} es la representación entera. La cuantización moderna a menudo prefiere el formalismo de escala y punto cero para enteros, ya que es más flexible y se adapta mejor a las capacidades del *hardware*.

***Post-Training Quantization* (PTQ):** PTQ se aplica a un modelo ya entrenado en FP32, sin reentrenamiento específico de cuantización. Se distinguen dos modalidades:

- **PTQ estática:** Utiliza datos representativos para estimar los rangos de activaciones mediante calibración y fijar sus parámetros de cuantización antes del despliegue.
- **PTQ dinámica:** Cuantiza los pesos fuera de línea y calcula los parámetros de cuantización de activaciones durante la inferencia. Esta modalidad se empleó en TT1 mediante `quantize_dynamic`; no se realizó una calibración estática de activaciones como etapa de ese flujo.

Para mejorar la precisión de PTQ, especialmente con pocos bits, se utilizan técnicas como:

- ***Bias Correction*:** Se ajustan los vectores de sesgo de las capas para compensar el error medio introducido por la cuantización de los pesos, que puede desplazar la media de las activaciones.
- ***Cross-Layer Equalization* (CLE):** Se re-escalan los pesos entre capas consecutivas para igualar sus rangos dinámicos antes de la cuantización, lo que puede reducir el error de cuantización global. Métodos como AdaRound ajustan los pesos de forma más sofisticada para minimizar el error de redondeo [5, 18].

***Quantization-Aware Training* (QAT):** QAT simula el proceso de cuantización durante la fase de entrenamiento (o más comúnmente, durante una fase de re-entrenamiento o *fine-tuning* a partir de un modelo FP32 pre-entrenado). Esto se logra insertando nodos de ***Fake Quantization* (FQ)** en el grafo computacional, típicamente después de los pesos y de las activaciones.

- ***Forward Pass*:** En un nodo FQ, la entrada x (FP32) se cuantiza a k bits (simulando el efecto de S y Z) y luego se de-cuantiza inmediatamente para obtener $\hat{x}$ (FP32 con valores discretos). Este $\hat{x}$ se pasa a la siguiente operación.

$$\hat{x}_{FQ} = S\left(\text{clamp}\left(\text{round}\left(\frac{x}{S}\right) + Z, N_{min}^{(k)}, N_{max}^{(k)}\right) - Z\right) \tag{35}$$

- ***Backward Pass*:** El gradiente de la función de pérdida con respecto a $\hat{x}_{FQ}$ debe propagarse hacia x. Dado que la función $round(\cdot)$ tiene gradiente cero casi en todas partes, se utiliza el ***Straight-Through Estimator* (STE)**, que simplemente copia el gradiente: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial \hat{x}_{FQ}}$. Es decir, se asume $\frac{\partial \hat{x}_{FQ}}{\partial x} \approx 1$.

QAT permite que los pesos del modelo se ajusten durante el entrenamiento para compensar la pérdida de precisión introducida por la cuantización, resultando generalmente en una mayor exactitud final en comparación con PTQ, especialmente para cuantización a muy bajos bits [5, 18]. La Figura 4 contrasta la PTQ dinámica aplicada en TT1 con el flujo QAT propuesto para TT2. La primera calcula los parámetros de activación durante la inferencia; la segunda introduce cuantización aparente y estimación STE durante el entrenamiento. QAT no se evaluó en TT1 y su posible beneficio requiere pruebas posteriores.

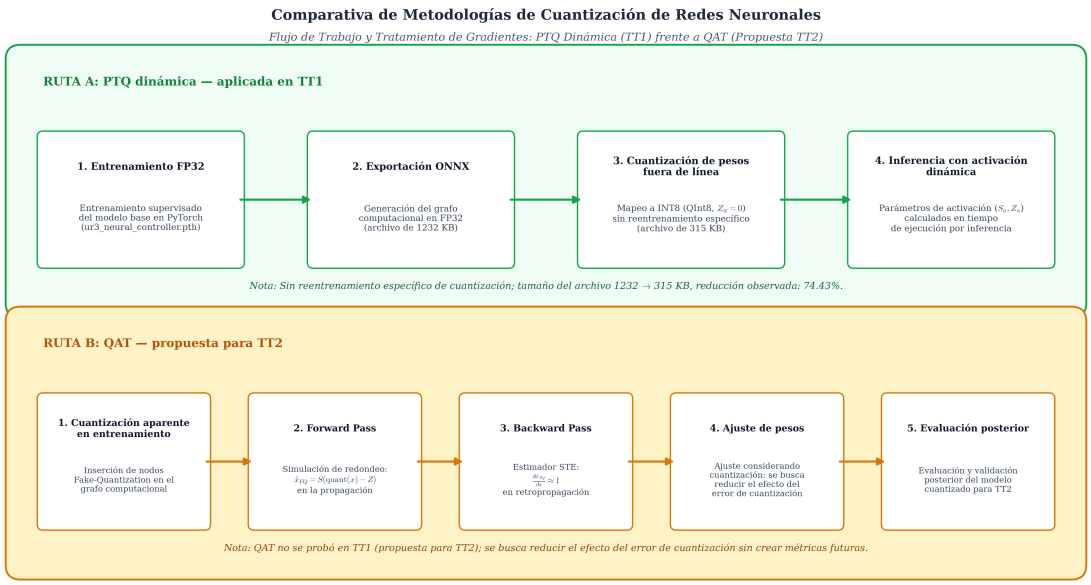


Figura 4: Flujos de cuantización: PTQ dinámica utilizada en TT1, con pesos cuantizados fuera de línea y parámetros de activación calculados durante la inferencia, frente a QAT como propuesta para TT2, con cuantización aparente y estimador STE. Los resultados de tamaño de archivo corresponden exclusivamente a la campaña PTQ reportada.

Cuantización de Precisión Mixta (*Mixed-Precision Quantization*): No todas las capas de una red neuronal tienen la misma sensibilidad a la cuantización. Algunas pueden tolerar precisiones muy bajas (ej., INT4) con poca degradación, mientras que otras (ej., la primera o la última capa) pueden requerir mayor precisión (ej., INT8 o incluso FP16) para mantener el rendimiento general del modelo. La cuantización de precisión mixta asigna diferentes números de bits a diferentes capas o tensores, buscando un óptimo global entre compresión/eficiencia y precisión. La determinación de la política de asignación de bits es un problema complejo. Los enfoques incluyen:

- Métodos basados en análisis de sensibilidad heurístico.
- Técnicas de búsqueda automatizada como AutoML o aprendizaje por refuerzo, donde un agente aprende a asignar precisiones.
- Optimización multiobjetivo, como en MOHAQ de **Rezk (2022)**[18], que utiliza algoritmos genéticos para encontrar soluciones en el frente de Pareto de error vs. recursos de hardware.

Impacto en Operaciones Aritméticas y Eficiencia: La cuantización transforma las operaciones de punto flotante en operaciones enteras. Consideremos la multiplicación de matrices o convoluciones, que son sumas de productos: $y = \sum w_i x_i + b$. Si w_i y x_i son cuantizados a k_w y k_x bits respectivamente, con parámetros (S_w, Z_w) y (S_x, Z_x):

$$S_y(y_q - Z_y) \approx \sum_i [S_w(w_{q_i} - Z_w)][S_x(x_{q_i} - Z_x)] + S_b(b_q - Z_b) \tag{36}$$

Desarrollando y agrupando las escalas:

$$y_q \approx Z_y + \frac{S_w S_x}{S_y} \sum_i (w_{q_i} - Z_w)(x_{q_i} - Z_x) + \frac{S_b}{S_y}(b_q - Z_b) \tag{37}$$

La suma $\sum_i (w_{q_i} - Z_w)(x_{q_i} - Z_x)$ involucra productos y acumulaciones de enteros, que se realizan típicamente en una precisión mayor (ej., INT32). El factor de re-escala $M = \frac{S_w S_x}{S_y}$ (y similar para el sesgo) se calcula offline y puede aproximarse por un producto de un entero y una potencia de dos para ser implementado con multiplicaciones enteras y bit-shifts, evitando operaciones de punto flotante en tiempo de inferencia. Las operaciones de normalización de batch (Batch Normalization), si presentes, a menudo se "pliegan" en los pesos y sesgos de las capas lineales o convolucionales precedentes antes de la cuantización, eliminando la necesidad de calcularlas durante la inferencia cuantizada. Los beneficios son una reducción significativa en el tamaño del modelo (ej., INT8 vs FP32 es una reducción de aproximadamente 4 veces), menor ancho de banda de memoria, y aceleración de cómputo debido a que las operaciones enteras son inherentemente más rápidas y menos costosas energéticamente que las de punto flotante en la mayoría de los procesadores y hardware dedicado.

1.2. Marco procedimental

1.2.1. Metodología mecatrónica

Para la gestión y desarrollo de este proyecto de investigación, que inherentemente combina elementos de diseño de software (controladores neuronales, cuantización), integración de hardware (plataforma robótica, sistema de cómputo) y experimentación (simulación y/o pruebas físicas), se ha optado por adoptar y adaptar el marco de trabajo ágil ***Scrum***. Esta elección se fundamenta en la flexibilidad de *Scrum* para manejar la incertidumbre inherente a la investigación, su énfasis en la entrega incremental de valor y la colaboración continua, son aspectos importantes para un proyecto de con un componente significativo de desarrollo tecnológico y evaluación comparativa [25].

Scrum, originalmente se crea para el desarrollo de software, pese a eso se ha aplicado al desarrollo de proyectos de ingeniería [26]. Su naturaleza iterativa e incremental permite la validación y verificación constante del progreso, un principio fundamental en el diseño mecatrónico.

Eventos de *Scrum* Adaptados al Proyecto Los eventos de *Scrum* proporcionarán la estructura para la inspección y adaptación regular:

- ***Sprint*:** Serán iteraciones de duración fija (ej., 2 a 4 semanas) durante las cuales se trabajará para completar un conjunto de objetivos del *Product Backlog* y crear un Incremento utilizable o evaluable del proyecto. La duración se definirá al inicio de cada fase (TT1 y TT2).
- **Planificación del *Sprint* (*Sprint* Planning):** Al inicio de cada *Sprint*, se seleccionarán los ítems de mayor prioridad del *Product Backlog* que se pueden

completar en el *Sprint* y definirá un Objetivo del *Sprint* (*Sprint goal*). Se desglosarán las tareas necesarias (*Sprint Backlog*).

- **Reuniones de Avance (Adaptación del *Daily Scrum*):** Se realizarán reuniones periódicas (ej., semanales) con los asesores para inspeccionar el progreso hacia el Objetivo del *Sprint*, discutir los trabajos completados, los que están en curso, los problemas encontrados y planificar los siguientes pasos. Esto reemplaza al *Daily Scrum* en un contexto de trabajo individual supervisado.
- **Revisión del *Sprint* (*Sprint Review*):** Al final de cada *Sprint*, se presentará el Incremento logrado (ej., un módulo de software funcional, resultados de simulación, un avance en la investigación documentado) a los asesores. Se discutirá el trabajo completado y se recogerá retroalimentación que podrá influir en el *Product Backlog* y en la planificación de *Sprints* futuros.
- **Retrospectiva del *Sprint* (*Sprint* Retrospective):** Después de la Revisión del *Sprint*, se realizará una reflexión sobre el proceso seguido durante el *Sprint*: qué funcionó bien, qué se podría mejorar y qué acciones concretas se tomarán para optimizar la eficiencia y efectividad en el siguiente *Sprint*.

Artefactos de *Scrum* Adaptados al Proyecto Los artefactos de *Scrum* proporcionarán transparencia y oportunidades para la inspección y adaptación:

- ***Product Backlog*:** Será una lista ordenada y dinámica de todo lo que se necesita para el proyecto. Incluirá los objetivos específicos de TT1 y TT2, desglosados en tareas de investigación, diseño, desarrollo, experimentación y documentación.
- ***Sprint Backlog*:** Es el conjunto de ítems del *Product Backlog* seleccionados para un *Sprint*, más un plan para entregar el Incremento y alcanzar el Objetivo del *Sprint*.
- **Incremento:** Es la suma de todos los ítems del *Product Backlog* completados durante un *Sprint* y todos los *Sprints* anteriores. Al final de cada *Sprint*, el nuevo Incremento debe ser *utilizable* o **evaluable** en el contexto del proyecto, lo que significa que representa un avance tangible y verificable hacia los objetivos generales.

Esta adopción de la metodología *Scrum* proporciona una planificación estructurada pero flexible, fomentando el progreso constante, la adaptación de los hallazgos y la entrega de resultados de forma incremental. En la Figura 5 se ilustra el ciclo iterativo completo del marco metodológico adaptado, mostrando la articulación entre el backlog de requerimientos técnicos, los sprints mecatrónicos multidisciplinarios y la generación progresiva de incrementos evaluables en simulación y controladores en C++.

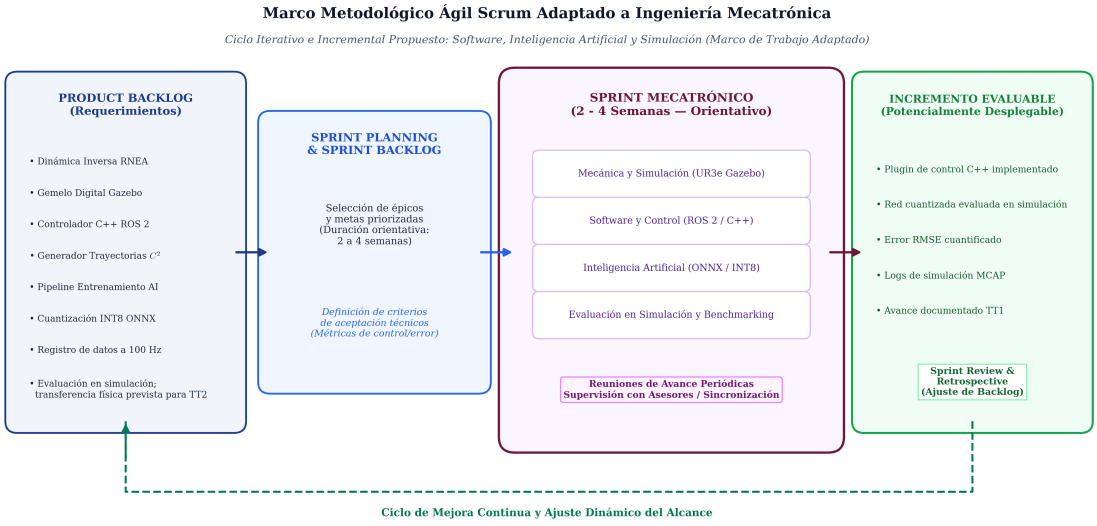


Figura 5: Marco metodológico ágil *Scrum* adaptado a la ingeniería mecatrónica del proyecto, integrando de forma iterativa el desarrollo de dinámica analítica, simulación del gemelo digital, pipeline de inteligencia artificial cuantizada y evaluación de controladores en C++. Las duraciones son orientativas y la transferencia física se contempla para TT2.

2. Diseño del sistema

2.1. Diseño conceptual

El diseño conceptual representa la fase inicial del desarrollo mecatrónico en la cual se traducen las necesidades del problema en especificaciones técnicas formales y se definen las alternativas de solución bajo el marco metodológico ágil adoptado (*Scrum*). En este proyecto, la arquitectura se concibe para evolucionar de manera incremental a través de *Sprints* multidisciplinarios, manteniendo una rigurosa separación de responsabilidades y facilitando la validación temprana mediante simulación física de alta fidelidad, preparando la posterior evaluación de transferencia a la plataforma física.

2.1.1. Necesidades - requerimientos

Con el propósito de garantizar un control articular preciso, estable y computacionalmente eficiente sobre el manipulador robótico de 6 grados de libertad (GDL), se establecieron los requerimientos de ingeniería que rigen el diseño del sistema. Estos se dividen en requerimientos funcionales (RF), que describen las operaciones y capacidades dinámicas que el sistema debe ejecutar, y requerimientos no funcionales (RNF), que imponen restricciones de tiempo real, seguridad, determinismo y compatibilidad con estándares industriales de robótica y aprendizaje profundo. La Tabla 3 sintetiza estos requerimientos con sus correspondientes métricas y criterios de aceptación.

Cuadro 3: Requerimientos funcionales y no funcionales del sistema de control mecatrónico.

ID	Requerimiento	Descripción Técnica	Criterio / Métrica
Requerimientos Funcionales (RF)			
RF-01	Cinemática de 6 GDL	Control articular coordinado de las 6 articulaciones rotacionales del robot en espacio articular ($\boldsymbol{q} \in \mathbb{R}^6$).	Grados de libertad: 6. Error de seguimiento evaluado mediante RMSE $< 0{,}05$ rad.
RF-02	Bucle de control en tiempo real	Ejecución periódica determinista del lazo cerrado de control a una tasa mínima de 100 Hz, con escalabilidad a 500 Hz.	Periodo $T_s \leq 10$ ms (diseño nominal $T_s = 2$ ms).
RF-03	Predicción de torque total	Aproximación neuronal del torque total del controlador analítico con PD, utilizando RNEA como característica de entrada.	Salida total de la red, desnormalizada y saturada, aplicada a los actuadores.
RF-04	Cuantización a INT8	Compresión de pesos y activaciones a precisión entera de 8 bits (INT8) mediante cuantización post-entrenamiento (PTQ).	Reducción de tamaño $\geq 50\,\%$ (alcanzado: $74.4\,\%$).
RF-05	Límites de esfuerzo	Definición y aplicación de límites de comando por articulación durante la implementación física. En TT1 se utiliza una saturación uniforme provisional para simulación.	$\lvert\tau_i\rvert \leq \tau_{max,i}$, con $\tau_{max,i}$ por verificar en TT2; valor provisional TT1: 55 N·m.
RF-06	Trayectorias suaves (C^2)	Generación de perfiles de referencia articular con continuidad de posición, velocidad y aceleración continuas mediante polinomios de 5to orden.	Continuidad matemática C^2, continuidad cinemática C^2.
Requerimientos No Funcionales (RNF)			
RNF-01	Paridad Sim-to-Real	Compatibilidad nativa con ROS 2 Jazzy y el marco estándar `ros2_control`, permitiendo intercambiar el gemelo digital por el robot físico con adaptaciones que deberán verificarse en TT2.	Soporte para el driver `Universal_Robots_ROS2_Driver` y `gz_ros2_control`.
RNF-02	Determinismo temporal C++	Exclusión total de asignaciones dinámicas de memoria (`malloc`, `new`) en el método cíclico `update()` y uso de buffers no bloqueantes.	Jitter del periodo: $\max_k \lvert j_k\rvert < 100\,\mu$s (por verificar); uso de `RealtimeBuffer`.
RNF-03	Interoperabilidad de ML	Adopción del estándar abierto ONNX (*Open Neural Network Exchange*) y motor de inferencia C++ monohilo para minimizar fuentes de latencia variable y fluctuaciones de jitter.	ONNX IR v9 / Opset 9, ejecución monohilo con ONNX Runtime.
RNF-04	Modularidad para plataforma neutra	Aislamiento funcional de los algoritmos para posibilitar su migración y benchmarking en hardware neutro (Microcontrolador vs. FPGA) en TT2.	Arquitectura desacoplada en librerías independientes y plugins.

2.1.2. Arquitectura funcional (funciones)

En congruencia con el marco ágil de desarrollo (*Scrum*), la arquitectura funcional de este sistema se concibe como un **pipeline desacoplado de procesamiento de datos y servicios en tiempo real**. Esta estructura permite que cada servicio opere como un incremento de software modular, verificable de forma independiente y coordinado a través de interfaces estandarizadas.

RF-03 se precisa en esta revisión para corresponder a la arquitectura utilizada en los ensayos y confirmada por el autor: predicción del torque total con RNEA como entrada. La descripción previa de una suma de RNEA y un residual neuronal no correspondía al controlador evaluado. Esta corrección no modifica el protocolo aprobado ni supone un cambio de implementación.

La descomposición funcional del sistema se estructura en los siguientes cinco servicios esenciales:

1. **Servicio de Planificación y Generación de Trayectorias:** Encargado de sintetizar las trayectorias articulares de referencia deseadas $(\boldsymbol{q}_d(t), \dot{\boldsymbol{q}}_d(t), \ddot{\boldsymbol{q}}_d(t))$. Emplea formulaciones polinomiales de quinto grado [27] para obtener continuidad de posición, velocidad y aceleración (C^2) bajo las condiciones de frontera definidas. Esta propiedad matemática no verifica la vida útil de las transmisiones.

2. **Servicio de Supervisión y Bucle Crítico en Tiempo Real:** Núcleo central del controlador. En cada periodo de muestreo ($T_s = 2$ ms), este servicio adquiere el estado cinemático real del manipulador $(\boldsymbol{q}, \dot{\boldsymbol{q}})$ a través de las interfaces de hardware (`hardware_interface::HW_IF_POSITION` y `HW_IF_VELOCITY`), calcula el vector de error articular $\boldsymbol{e} = \boldsymbol{q}_d - \boldsymbol{q}$ y su derivada $\dot{\boldsymbol{e}}$, y evalúa analíticamente las ecuaciones recursivas de Newton-Euler (RNEA) [28] para obtener el vector de pares nominales $\boldsymbol{\tau}_{RNEA}(\boldsymbol{q}, \dot{\boldsymbol{q}}, \ddot{\boldsymbol{q}}_d)$.

3. **Servicio de Inferencia Inteligente Cuantizada:** Construye un vector de características normalizado de 42 dimensiones que condensa el estado cinemático actual, el estado deseado y la predicción física analítica. Mediante el motor de inferencia optimizado de ONNX Runtime ejecutado en modo monohilo dentro del espacio de memoria de C++, este servicio predice el torque total mediante el modelo cuantizado dinámicamente a INT8 [29], restituye la escala física de las magnitudes y aplica un esquema de saturación provisional acotado a $\pm 55{,}0\ \mathrm{N \cdot m}$.

4. **Capa de Abstracción de Hardware:** Mediante `ros2_control` se envían comandos de esfuerzo a la planta simulada [30]. La modularidad facilita organizar el controlador, pero no demuestra disponibilidad de una interfaz equivalente ni ejecución de la misma ley de torque en el robot físico.

5. **Servicio de Publicación de Datos y Registro:** El controlador maestro publica desde `update()` los vectores de estado, referencias y pares a 100 Hz hacia el sistema de registro en formato MCAP (`ros2 bag`). Estos datos se procesan posteriormente para el entrenamiento y la evaluación. El `RealtimeBuffer` del código

disponible se utiliza para recibir referencias, no para trasladar esta publicación a un hilo asíncrono; su impacto temporal requiere medición.

En la Figura 6 se ilustra el diagrama funcional del pipeline de datos y bucles de control mecatrónico.

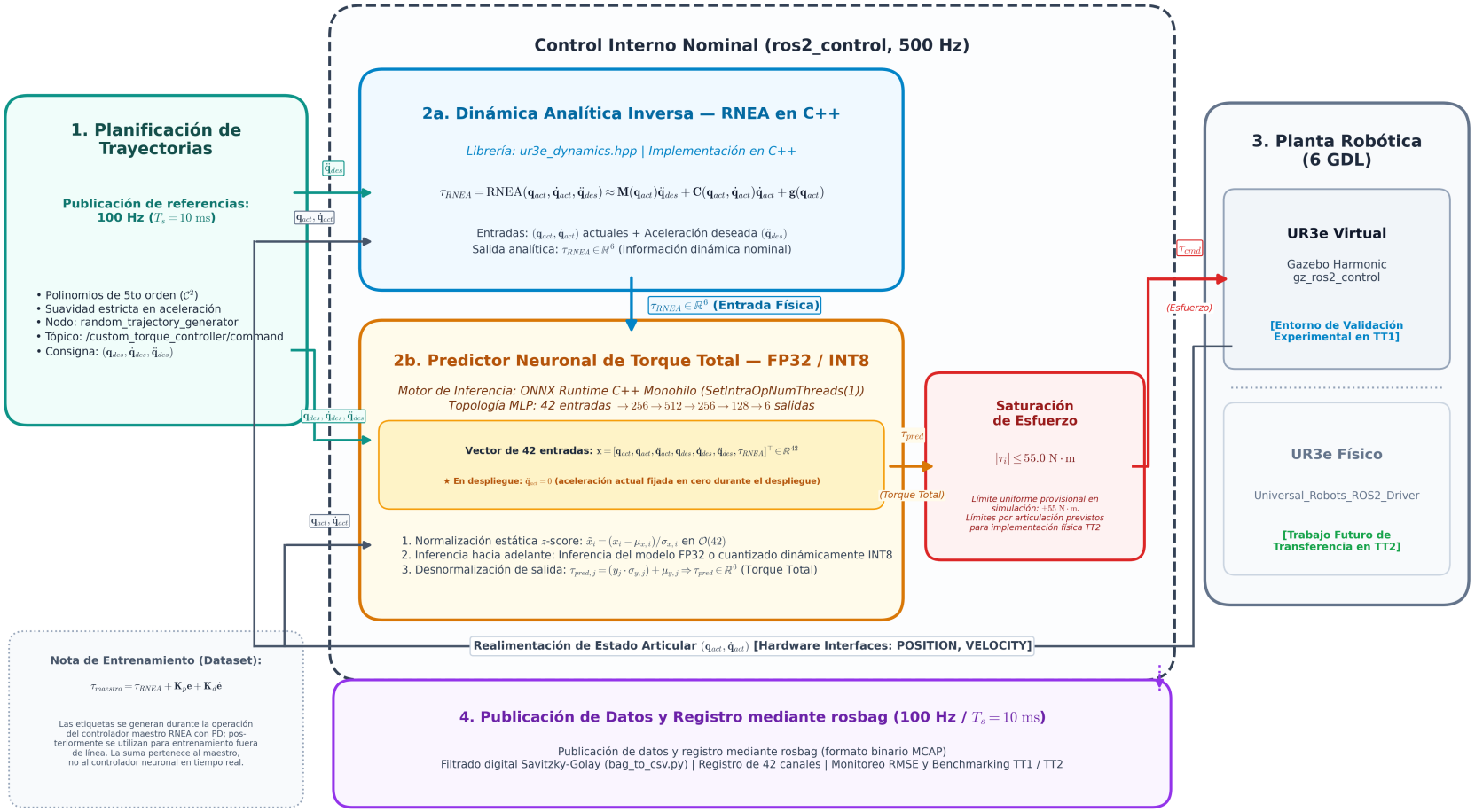


Figura 6: Arquitectura funcional del controlador neuronal de torque total: RNEA como entrada de la red, desnormalización y saturación de la salida aplicada al UR3e virtual. Se distinguen el control interno nominal a 500 Hz, las referencias y datos a 100 Hz y la transferencia física prevista para TT2. El límite uniforme de ±55 N·m es provisional para simulación.

2.1.3. Arquitectura física (módulos)

Para habilitar un desarrollo ágil por *Sprints* y facilitar la integración mecatrónica incremental, el sistema se divide físicamente en cuatro módulos cohesivos y desacoplados (M_1 a M_4). Esta división organiza los avances de simulación de TT1 y el trabajo de integración física previsto para TT2:

- **Módulo 1 (M_1) – Planta Robótica y Entorno Sim-to-Real:** Comprende el manipulador robótico de 6 GDL Universal Robots UR3e (alcance de 500 mm, carga útil nominal de 3 kg) [31] y su contraparte virtual de alta fidelidad. En TT1, la planta se modela rigurosamente en el simulador Gazebo Harmonic acoplado con el plugin `gz_ros2_control`, utilizando los descriptores cinemáticos y dinámicos oficiales (URDF/Xacro). Este entorno reproduce las inercias, masas y límites articulares reales del UR3e, sin que las interfaces simuladas demuestren compatibilidad directa con el manipulador físico de TT2.

- **Módulo 2 (M_2) – Control Dinámico Analítico y Lógica de Tiempo Real:** Contiene la formulación analítica de la dinámica inversa implementada en C++17 mediante la librería de encabezados `ur3e_dynamics.hpp`. Este módulo computa el algoritmo RNEA [28] para proporcionar el término de dinámica inversa del controlador analítico y las características de entrada neuronal. Asimismo, aloja el controlador clásico por par computado (`ComputedTorqueController`) [32] que sirve como referencia (*baseline*) experimental.

- **Módulo 3 (M_3) – Inteligencia Artificial y Motor de Cuantización:** Abarca el ciclo de vida completo de la red neuronal predictora de torque total: desde la arquitectura de perceptrón multicapa (MLP) diseñada y entrenada en PyTorch (42 entradas, capas densas intermedias de 256, 512, 256 y 128 neuronas con activaciones ReLU, y 6 salidas de par), hasta su exportación al formato estándar ONNX y su posterior cuantización dinámica post-entrenamiento (PTQ) a enteros de 8 bits (INT8) mediante `quantize_model.py` [29]. Este módulo integra el plugin de control `NeuralTorqueController`, el cual aloja una sesión persistente de la API C++ de ONNX Runtime configurada en modo monohilo (`SetIntraOpNumThreads(1)`) para reducir la sobrecarga de ejecución; las latencias observadas dependen de la CPU y del sistema operativo.

- **Módulo 4 (M_4) – Supervisión, Telemetría y Adquisición:** Integra el software de generación de trayectorias aleatorias y deterministas con polinomios de quinto orden (`random_trajectory_generator.py`) [27], el subsistema de registro de telemetría de alta frecuencia mediante `ros2 bag` en almacenamiento binario estructurado MCAP, y las rutinas de procesamiento y filtrado digital (`bag_to_csv.py`) basadas en el filtro de Savitzky-Golay [33] para la estimación suave de velocidades y aceleraciones a partir de datos experimentales.

2.1.4. Propuestas solución

En la concepción mecatrónica del sistema se analizaron diversas alternativas tecnológicas tanto a nivel de componentes individuales como de integración global del

sistema.

Alternativas por Componente

1. **Entorno de Planta y Simulación (M_1):**
   - *Opción 1.1 (Simulación Cinemática en MATLAB):* Entorno accesible para validar cinemática teórica, pero carente de dinámica multicuerpo realista y sin compatibilidad nativa con bucles de tiempo real en C++.
   - *Opción 1.2 (CoppeliaSim con API Remota):* Simulación física versátil, pero su comunicación basada en sockets TCP/IP externos induce fluctuaciones de latencia inaceptables para control a 500 Hz.
   - *Opción 1.3 (Gazebo Harmonic con `gz_ros2_control`):* Integración nativa con la pila de control de ROS 2 en memoria compartida, resolución de contactos y total paridad de interfaces con el robot físico real.
2. **Estrategia de Control Dinámico (M_2):**
   - *Opción 2.1 (PID Articular Independiente Desacoplado):* Sencillo de implementar, pero incapaz de compensar el fuerte acoplamiento inercial, los efectos de Coriolis y la gravedad a velocidades moderadas o altas.
   - *Opción 2.2 (Control por Par Computado Analítico RNEA):* Linealiza la dinámica en todo el espacio de trabajo; no obstante, es vulnerable a incertidumbres paramétricas, fricciones no modeladas y envejecimiento mecánico.
   - *Opción 2.3 (Control Adaptable no Lineal Clásico):* Requiere derivar y calcular en línea matrices de regresores analíticos sumamente costosas computacionalmente para 6 GDL.
3. **Motor de Compensación Inteligente y Despliegue (M_3):**
   - *Opción 3.1 (Inferencia en Python mediante Nodos ROS):* Inviable para el lazo crítico de control debido a las interrupciones del recolector de basura (*Garbage Collector*) y el bloqueo global de intérprete (GIL), generando retardos estocásticos de decenas de milisegundos.
   - *Opción 3.2 (Red Neuronal FP32 en C++):* Despliegue en punto flotante para establecer una referencia de tamaño y latencia de inferencia en la plataforma evaluada.
   - *Opción 3.3 (Red Neuronal INT8 con ONNX Runtime C++):* Cuantización dinámica para reducir el tamaño del modelo y evaluar la latencia de inferencia, sin establecer una cota temporal garantizada.

Propuestas de Solución Integrada del Sistema A partir del análisis de componentes y del propósito global del proyecto, se definieron tres propuestas de integración conceptual:

Propuesta A (Control Clásico Desacoplado en Entorno Estándar): Consiste en la implementación de controladores PID individuales por articulación ejecutados en una arquitectura computacional genérica. Las trayectorias se generan de manera trapezoidal y no se incorpora ningún tipo de compensación analítica ni neuronal. Representa el esquema industrial más elemental y sirve como límite inferior de comparación.

Propuesta B (Plataforma Neutra de Comparación Hardware – Microcontrolador vs. FPGA): Esta propuesta surge como la plataforma de evaluación hardware concebida para realizar una comparación científica neutra y cuantitativa entre el paradigma de control clásico y el control inteligente. En este esquema, el control clásico PID se implementa en un microcontrolador o procesador embebido convencional, mientras que la inferencia de la red neuronal se acelera en hardware dedicado dentro de una tarjeta FPGA (aprovechando el paralelismo espacial y operaciones aritméticas de enteros de bajo consumo). Esto permite medir objetivamente la latencia, el consumo de potencia eléctrica y la ocupación de recursos de silicio (LUTs, DSPs, memoria) bajo condiciones experimentales idénticas. Debido a que en la fase TT1 no se contaba con la planta física confirmada y la implementación directa en FPGA requería desviar recursos del modelado cinemático-dinámico, esta propuesta se establece formalmente como el marco experimental objetivo a consumar en la fase TT2.

Propuesta C (Arquitectura Híbrida Cuantizada en Sim-to-Real – Solución Seleccionada para TT1): Plantea una arquitectura híbrida mecatrónica desarrollada en ROS 2 Jazzy y C++17, donde el algoritmo analítico RNEA proporciona una característica de entrada y una red neuronal cuantizada a INT8 desplegada mediante ONNX Runtime monohilo predice el torque total del controlador maestro. La salida de la red se desnormaliza y satura antes de aplicarse a los actuadores. El bucle se ejecuta dentro del marco `ros2_control` gobernando el gemelo digital del UR3e en Gazebo Harmonic, con trayectorias quínticas continuas (C^2) y telemetría en formato MCAP. Esta propuesta permite evaluar la implementación en simulación durante TT1. La transferencia al manipulador físico y la instrumentación de una plataforma de comparación deberán evaluarse en TT2.

2.1.5. Validación

La validación conceptual de la arquitectura mecatrónica seleccionada (Propuesta C) se sustenta en tres análisis analíticos y de ingeniería que confirman su viabilidad para cumplir los requerimientos antes de su despliegue físico en TT2:

1. **Contrato de Aprendizaje y Aplicación del Torque Total:** La dinámica articular de un manipulador rígido de 6 GDL viene dada por:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) + \boldsymbol{\tau}_f(\dot{\boldsymbol{q}}) = \boldsymbol{\tau} \tag{38}$$

   Para el aprendizaje supervisado se utiliza el torque total del controlador maestro:

$$\boldsymbol{y} = \boldsymbol{\tau}_{maestro} = \boldsymbol{\tau}_{RNEA}(\boldsymbol{q},\dot{\boldsymbol{q}},\ddot{\boldsymbol{q}}_d) + \boldsymbol{K}_p\boldsymbol{e} + \boldsymbol{K}_d\dot{\boldsymbol{e}} \tag{39}$$

donde $\boldsymbol{y} \in \mathbb{R}^6$ es el objetivo de entrenamiento en N·m, $\boldsymbol{e} = \boldsymbol{q}_d - \boldsymbol{q}$ y $\dot{\boldsymbol{e}} = \dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}}$ son los errores de posición y velocidad, y $\boldsymbol{K}_p$, $\boldsymbol{K}_d$ son las matrices diagonales de ganancias. El vector de características $\boldsymbol{x}$ incluye $\boldsymbol{\tau}_{RNEA}$, como se define en la Ecuación (57). Durante la operación neuronal se aplica:

$$\boldsymbol{\tau}_{cmd} = \mathrm{sat}_{55}(\hat{\boldsymbol{\tau}}_{\boldsymbol{\theta}}(\boldsymbol{x})) \tag{40}$$

donde $\hat{\boldsymbol{\tau}}_{\boldsymbol{\theta}}$ es la salida neuronal desnormalizada a N·m, $\boldsymbol{\theta}$ son los parámetros entrenados y sat_{55} limita cada componente a ± 55 N·m, conforme a la implementación disponible. No se suma RNEA nuevamente a la salida. La tarea corresponde a imitación supervisada de torque total; la presencia de RNEA como entrada no demuestra identificación de fricción ni de dinámica no modelada. La estabilidad y la generalización requieren evaluación en lazo cerrado.

2. **Presupuesto Temporal de Tiempo Real (*Timing Budget*):** Para una frecuencia de control objetivo de $f_s = 500$ Hz, el periodo máximo disponible por ciclo es $T_s = 2000\,\mu$s. Las mediciones de perfilado temporal en C++ arrojan los siguientes tiempos de ejecución en CPU monohilo:

   - Adquisición y lectura de interfaces de estado (`ros2_control`): $\approx 10{,}2\,\mu$s.
   - Referencia del módulo de control analítico RNEA + PD: $\approx 151{,}2\,\mu$s; no corresponde a RNEA aislado ni a un subproceso del controlador neuronal.
   - Normalización de 42 características con escaladores precalculados: $\approx 5{,}1\,\mu$s.
   - Inferencia de red neuronal cuantizada INT8 en ONNX Runtime: $\approx 29{,}2\,\mu$s.
   - Desnormalización, saturación de comando y escritura de comandos de par: $\approx 19{,}7\,\mu$s.

   El tiempo total del controlador neuronal debe medirse de forma independiente. El presupuesto previo sumaba una referencia RNEA + PD como si fuera RNEA aislado; por ello no se utiliza para acreditar margen temporal ni ausencia de incumplimientos del plazo.

3. **Preservación de Rango Dinámico en INT8:** Al aplicar normalización estándar con media cero y desviación unitaria (z-score) a las 42 entradas y 6 salidas en el pipeline de entrenamiento:

$$\tilde{x}_i = \frac{x_i - \mu_{x_i}}{\sigma_{x_i}}, \quad \forall i \in \{1, \ldots, 42\} \tag{41}$$

   las distribuciones estadísticas quedan concentradas en el intervalo $[-3, 3]$ con una probabilidad superior al 99,7 %. La cuantización dinámica uniforme a 8 bits provee $2^8 = 256$ niveles de discretización lineal, lo cual otorga una resolución efectiva por bit suficiente para representar los pares con un error de redondeo del orden de 10^{-3} N · m, un valor varios órdenes de magnitud menor que la fricción estática de las articulaciones del UR3e.

2.1.6. Selección diseño conceptual

Para formalizar la toma de decisiones ingenieriles y determinar la idoneidad técnica de la arquitectura para la fase TT1, se empleó una matriz de selección conceptual de Pugh ponderada [34]. En este método, los criterios de evaluación reflejan los factores críticos de éxito del proyecto (viabilidad analítica, precisión dinámica, eficiencia computacional y preparación para hardware real en TT2).

Se asignó un peso relativo normalizado ($w_i \in [0,1]$ con $\sum w_i = 1{,}0$) a cada criterio en función de su criticidad mecatrónica. Como solución de referencia (*Datum*, con puntaje base 0), se seleccionó la **Propuesta B (Plataforma Neutra / FPGA)**, dado que representa el marco comparativo ideal proyectado. Las alternativas se calificaron con una escala relativa: +2 (significativamente mejor que la referencia), +1 (mejor), 0 (equivalente), −1 (inferior) y −2 (significativamente inferior).

En la Tabla 4 se detallan los resultados de la evaluación conceptual.

Cuadro 4: Matriz de selección conceptual de Pugh: ponderaciones y valoraciones de diseño, no resultados experimentales de los controladores.

Criterio de Evaluación	**Peso (w_i)**	**Propuesta A (PID Clásico)**	**Propuesta B (Plat. Neutra)**	**Propuesta C (Híbrida INT8)**
1. Viabilidad de validación algorítmica y desriesgo previo en TT1	0.25	0	0 (Datum)	+2
2. Preparación modular para transferencia física (TT2)	0.20	0	0 (Datum)	+2
3. Potencial para estudiar dinámica no lineal	0.20	-2	0 (Datum)	+1
4. Eficiencia de cómputo y reducción de huella de memoria (INT8)	0.15	+1	0 (Datum)	+1
5. Potencial de integración en un ciclo periódico	0.10	0	0 (Datum)	+1
6. Simplicidad de desarrollo e integración en fase TT1	0.10	+1	0 (Datum)	+1
Puntaje Total Ponderado ($\sum w_i s_i$)	**1.00**	**-0.15**	**0.00**	**+1.45**

Justificación de la Selección: Como se aprecia en la matriz de Pugh, la **Propuesta C (Arquitectura Híbrida Cuantizada en Sim-to-Real)** obtiene la calificación más alta (+1.45), por encima de tanto al PID clásico (-0.15) como a la línea base de la Propuesta B (0.00).

Las razones técnicas determinantes para esta elección en la fase TT1 radican en:

1. **Evaluación de Viabilidad Integral:** Permite evaluar en simulación el pipeline de predicción de torque total con RNEA como entrada de la red cuantizada, antes del despliegue físico.

2. **Preparación para la Transferencia Física:** La separación modular facilita estudiar la reutilización de componentes en TT2. La interfaz de comando del robot y las adaptaciones requeridas deberán verificarse antes del despliegue.

3. **Base de Comparación para TT2:** Los modelos y controladores evaluados en simulación ofrecen una referencia inicial para comparar plataformas. Su validación numérica, física y temporal deberá completarse antes de atribuir resultados al hardware embebido.

2.2. Diseño detallado

En esta sección se desarrolla la especificación cuantitativa, matemática y algorítmica de cada uno de los cuatro módulos mecatrónicos concebidos (M_1 a M_4), así como la arquitectura de su integración global. Para cada módulo se detallan las ecuaciones de diseño, el flujo de señales, los algoritmos implementados y su correspondiente validación analítica y funcional previa a la transferencia física en TT2.

2.2.1. Detalle módulo 1 (M_1) – Planta Robótica y Entorno Sim-to-Real

El módulo M_1 comprende la planta física objetivo y su réplica virtual en simulación. Como efector y estructura de manipulación se utiliza el robot industrial colaborativo Universal Robots UR3e [31], un manipulador serial antropomórfico de 6 articulaciones rotacionales concebido para tareas de alta repetibilidad (±0,03 mm) con un radio de alcance máximo de 500 mm y una capacidad de carga nominal de 3 kg.

Parámetros Cinemáticos y Dinámicos del Manipulador Para el cálculo de la cinemática directa y el posterior modelado dinámico analítico, se adoptó la convención de Denavit-Hartenberg modificada propuesta por Craig [19]. En la Tabla 5 se detallan los parámetros geométricos del manipulador implementados en C++, validados previamente con respecto al árbol cinemático oficial de Universal Robots.

Cuadro 5: Parámetros de Denavit-Hartenberg modificados del robot UR3e.

Articulación (i)	$\boldsymbol{\alpha_{i-1}}$ [rad]	$\boldsymbol{a_{i-1}}$ [m]	$\boldsymbol{d_i}$ [m]	$\boldsymbol{\theta_i}$ [rad]	**Rango Operacional [rad]**
1 (Base / Hombro Pan)	0	0	0,15185	θ_1	$[-2\pi, 2\pi]$
2 (Hombro Lift)	$\pi/2$	0	0	θ_2	$[-2\pi, 2\pi]$
3 (Codo)	0	−0,24355	0	θ_3	$[-2\pi, 2\pi]$
4 (Muñeca 1)	0	−0,21320	0,13105	θ_4	$[-2\pi, 2\pi]$
5 (Muñeca 2)	$\pi/2$	0	0,08535	θ_5	$[-2\pi, 2\pi]$
6 (Muñeca 3)	$-\pi/2$	0	0,09210	θ_6	$[-2\pi, 2\pi]$

A partir de los parámetros de la Tabla 5, la matriz homogénea de transformación entre el marco $i-1$ y el marco i se define como:

$$\boldsymbol{T}_i^{i-1} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_{i-1} \\ \sin\theta_i\cos\alpha_{i-1} & \cos\theta_i\cos\alpha_{i-1} & -\sin\alpha_{i-1} & -d_i\sin\alpha_{i-1} \\ \sin\theta_i\sin\alpha_{i-1} & \cos\theta_i\sin\alpha_{i-1} & \cos\alpha_{i-1} & d_i\cos\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{42}$$

En cuanto a las propiedades inerciales, los eslabones presentan una distribución de masa total de 8,81 kg, distribuida según el vector baricéntrico:

$$\boldsymbol{m} = [2{,}00,\ 3{,}44,\ 1{,}44,\ 0{,}87,\ 0{,}80,\ 0{,}26]^T \text{ kg} \tag{43}$$

Los centros de masa locales ($\boldsymbol{r}_{c_i}$) de cada eslabón medidos respecto a su marco articular corresponden a:

$$\begin{aligned} \boldsymbol{r}_{c_1} &= [0{,}000, 0{,}000, -0{,}020]^T \text{ m}, & \boldsymbol{r}_{c_2} &= [-0{,}11355, 0{,}000, 0{,}11570]^T \text{ m}, \\ \boldsymbol{r}_{c_3} &= [-0{,}16320, 0{,}000, 0{,}02380]^T \text{ m}, & \boldsymbol{r}_{c_4} &= [0{,}000, -0{,}010, 0{,}000]^T \text{ m}, \\ \boldsymbol{r}_{c_5} &= [0{,}000, 0{,}010, 0{,}000]^T \text{ m}, & \boldsymbol{r}_{c_6} &= [0{,}000, 0{,}000, -0{,}020]^T \text{ m} \end{aligned} \tag{44}$$

Los tensores de inercia baricéntricos se modelan en su componente principal diagonal con valores nominales $\boldsymbol{I}_{c_i} \approx \text{diag}(0{,}01, 0{,}01, 0{,}01)$ kg $\cdot$ m^2, suficientes para capturar el momento de inercia dominante en manipuladores compactos.

Entorno de Simulación Física y Paridad Sim-to-Real La planta virtual se configuró en el motor de física multicuerpo Gazebo Harmonic. A través de la interfaz `gz_ros2_control`, el simulador expone dos interfaces de lectura de estado por cada articulación: posición angular (`HW_IF_POSITION`) y velocidad angular (`HW_IF_VELOCITY`), así como una interfaz de comando de bajo nivel en esfuerzo/torque (`HW_IF_EFFORT`).

El uso de interfaces estándar de `ros2_control` organiza la integración en simulación. La compatibilidad con el driver del robot físico, especialmente la disponibilidad de comandos de torque, debe comprobarse antes de reutilizar el controlador en TT2. No se considera demostrada una transferencia mediante cambios exclusivos del archivo de lanzamiento.

Validación (M_1) La validación cinemática del módulo M_1 se efectuó contrastando las matrices de transformación homogénea derivadas de la Ecuación (42) frente a las lecturas del árbol de transformaciones dinámicas de ROS 2 (`/tf`). Para configuraciones singulares (brazo extendido horizontalmente con $\boldsymbol{q} = [0, 0, 0, 0, 0, 0]^T$), el efector final alcanza una posición cartesiana de $x = a_2 + a_3 = -0{,}45675$ m, con un error cuadrático medio respecto a la cinemática analítica reportado como inferior a 10^{-6} m. La saturación del controlador neuronal limita sus salidas finitas al intervalo configurado. No se presenta una validación de seguridad ante colisiones o singularidades ni de los límites físicos por articulación.

2.2.2. Detalle módulo 2 (M_2) – Control Dinámico Analítico en Tiempo Real

El módulo M_2 implementa en tiempo real la solución de la dinámica inversa del robot basada en las leyes clásicas de Newton y Euler mediante la formulación recursiva de Luh, Walker y Paul (RNEA) [28]. Su propósito primario es calcular de manera exacta el vector de pares articulares $\boldsymbol{\tau}_{RNEA} \in \mathbb{R}^6$ requerido para que el robot siga una trayectoria con aceleraciones $\ddot{\boldsymbol{q}}_d$ compensando las inercias, fuerzas centrífugas, de Coriolis y gravitacionales.

Formulación Analítica Recursiva de Newton-Euler El algoritmo se divide en dos etapas recursivas:

1. **Pasada hacia adelante (Propagación Cinemática, $i = 1 \dots 6$):** Se calcula la velocidad y aceleración de cada eslabón desde la base inmóvil hasta el efector final:

$$\boldsymbol{\omega}_i = \boldsymbol{R}_i^{i-1}\boldsymbol{\omega}_{i-1} + \dot{q}_i \boldsymbol{z}_0 \tag{45}$$
$$\dot{\boldsymbol{\omega}}_i = \boldsymbol{R}_i^{i-1}\dot{\boldsymbol{\omega}}_{i-1} + (\boldsymbol{R}_i^{i-1}\boldsymbol{\omega}_{i-1}) \times (\dot{q}_i \boldsymbol{z}_0) + \ddot{q}_i \boldsymbol{z}_0 \tag{46}$$
$$\dot{\boldsymbol{v}}_i = \boldsymbol{R}_i^{i-1}\dot{\boldsymbol{v}}_{i-1} + \dot{\boldsymbol{\omega}}_i \times \boldsymbol{P}_i + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \boldsymbol{P}_i) \tag{47}$$
$$\dot{\boldsymbol{v}}_{c_i} = \dot{\boldsymbol{v}}_i + \dot{\boldsymbol{\omega}}_i \times \boldsymbol{r}_{c_i} + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \boldsymbol{r}_{c_i}) \tag{48}$$

   donde $\boldsymbol{z}_0 = [0, 0, 1]^T$ es el eje de revolución articular, y se establece como condición de frontera en la base una aceleración ficticia vertical $\dot{\boldsymbol{v}}_0 = [0, 0, 9{,}81]^T$ m/s^2 para incorporar la gravedad sin costo adicional.

2. **Pasada hacia atrás (Balance Inercial de Fuerzas y Momentos, $i = 6 \dots 1$):** Desde el efector hasta la base se evalúan las fuerzas y momentos netos:

$$\boldsymbol{F}_i = m_i \dot{\boldsymbol{v}}_{c_i} \tag{49}$$
$$\boldsymbol{N}_i = \boldsymbol{I}_{c_i}\dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times (\boldsymbol{I}_{c_i}\boldsymbol{\omega}_i) \tag{50}$$
$$\boldsymbol{f}_i = \boldsymbol{R}_{i+1}^{i}\boldsymbol{f}_{i+1} + \boldsymbol{F}_i \tag{51}$$
$$\boldsymbol{n}_i = \boldsymbol{R}_{i+1}^{i}\boldsymbol{n}_{i+1} + \boldsymbol{P}_{i+1} \times (\boldsymbol{R}_{i+1}^{i}\boldsymbol{f}_{i+1}) + \boldsymbol{N}_i + \boldsymbol{r}_{c_i} \times \boldsymbol{F}_i \tag{52}$$

   Finalmente, el par del actuador en la articulación i resulta de proyectar el momento $\boldsymbol{n}_i$ sobre el eje de rotación:

$$\tau_{RNEA,i} = \boldsymbol{n}_i^T \boldsymbol{z}_0 \tag{53}$$

Implementación en C++ El método `rnea` se implementó en `ur3e_dynamics.hpp` utilizando Eigen. La versión disponible utiliza contenedores dinámicos para parte de sus datos; no se presenta como una implementación libre de asignaciones. El Listing 1 muestra el núcleo recursivo del cálculo.

```
// Pasada hacia adelante: propagacion cinematica (eslabones 0 a 5)
for (int i = 0; i < 6; ++i) {
    int idx = i + 1, prev = i;
    Eigen::Matrix3d Rt = R_store[idx].transpose();
```

```
    Eigen::Vector3d z_axis = z0 * dq(i), z_accel = z0 * ddq(i);
    w[idx] = Rt * w[prev] + z_axis;
    dw[idx] = Rt * dw[prev] + (Rt * w[prev]).cross(z_axis) + z_accel;
    dv[idx] = Rt * dv[prev] + dw[idx].cross(P) + w[idx].cross(w[idx].
    cross(P));
}
// Pasada hacia atras: balance de fuerzas y momentos (eslabones 5 a 0)
for (int i = 5; i >= 0; --i) {
    int idx = i + 1;
    Eigen::Vector3d dv_c = dv[idx] + dw[idx].cross(com[i]) + w[idx].
    cross(com[i]);
    Eigen::Vector3d F_in = mass[i] * dv_c;
    Eigen::Vector3d N_in = inertia[i] * dw[idx] + w[idx].cross(inertia
    [i] * w[idx]);
    f[idx] = (i == 5) ? F_in : R_store[idx+1] * f[idx+1] + F_in;
    n[idx] = (i == 5) ? N_in + com[i].cross(F_in) :
             R_store[idx+1] * n[idx+1] + P_store[idx+1].cross(R_store[
    idx+1] * f[idx+1]) + N_in + com[i].cross(F_in);
    tau(i) = n[idx].dot(z0);
}
```

Listing 1: Núcleo recursivo de cálculo analítico de dinámica inversa RNEA en C++ (`ur3e_dynamics.hpp`).

Lógica de Control por Par Computado (CTC) El módulo aloja además el controlador `ComputedTorqueController` en `ur3_custom_controller.cpp`. La ley de control por par computado (CTC) [32, 19] articula la compensación analítica y una retroalimentación de error proporcional-derivativo:

$$\boldsymbol{\tau}_{cmd} = \boldsymbol{\tau}_{RNEA}(\boldsymbol{q}, \dot{\boldsymbol{q}}, \ddot{\boldsymbol{q}}_d) + \boldsymbol{K}_p(\boldsymbol{q}_d - \boldsymbol{q}) + \boldsymbol{K}_d(\dot{\boldsymbol{q}}_d - \dot{\boldsymbol{q}}) \tag{54}$$

con matrices de ganancia diagonales definidas experimentalmente en:

$$\boldsymbol{K}_p = \text{diag}(100{,}0,\ 100{,}0,\ 100{,}0,\ 30{,}0,\ 30{,}0,\ 10{,}0) \tag{55}$$
$$\boldsymbol{K}_d = \text{diag}(10{,}0,\ 10{,}0,\ 10{,}0,\ 2{,}0,\ 2{,}0,\ 0{,}5) \tag{56}$$

Validación (M_2) Para la evaluación estática del módulo M_2 se reportó un par analítico de hombro de aproximadamente $-18{,}7$ N·m en la configuración horizontal indicada, con un error angular inferior a 0,002 rad. Este resultado describe un caso de simulación y no valida por sí solo todos los parámetros inerciales ni la implementación RNEA. La latencia aislada de RNEA debe caracterizarse por separado del cronometraje disponible del controlador analítico con PD; no se establece una cota garantizada de 50 μs.

2.2.3. Detalle módulo 3 (M_3) – Inteligencia Artificial y Motor Cuantizado

El módulo M_3 aproxima el torque total del controlador maestro definido en la Ecuación (39). El módulo M_2 proporciona el cálculo RNEA incluido entre las entradas. Las etiquetas no corresponden a un residual obtenido restando RNEA ni a una medición independiente de fricción, flexibilidad o incertidumbre paramétrica.

Dataset y Vector de Características Para el entrenamiento de la red se conformó un dataset estructurado (`dataset_ur3_final_1.csv`) donde cada muestra consiste en un vector de entrada $\boldsymbol{x} \in \mathbb{R}^{42}$ y un vector objetivo de par $\boldsymbol{y} = \boldsymbol{\tau}_{cmd} \in \mathbb{R}^6$. Las 42 variables de entrada agrupan 7 vectores articulares de 6 dimensiones cada uno:

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{q}_{act}^\top & \dot{\boldsymbol{q}}_{act}^\top & \ddot{\boldsymbol{q}}_{act}^\top & \boldsymbol{q}_{des}^\top & \dot{\boldsymbol{q}}_{des}^\top & \ddot{\boldsymbol{q}}_{des}^\top & \boldsymbol{\tau}_{RNEA}^\top \end{bmatrix}^T \in \mathbb{R}^{42} \tag{57}$$

La inclusión explícita de $\boldsymbol{\tau}_{RNEA}$ proporciona información del modelo analítico para la regresión del torque total. El entrenamiento utiliza directamente las columnas `tau_cmd` como objetivo, sin restar las columnas `tau_rnea`. En despliegue se mantiene el cálculo RNEA para construir la entrada, y la salida neuronal desnormalizada se aplica según la Ecuación (40). Por tanto, la red no sustituye la evaluación de RNEA ni implementa un compensador residual aditivo.

Topología del Perceptrón Multicapa (MLP) y Entrenamiento La red neuronal se diseñó en PyTorch (`train_pytorch.py`) bajo una arquitectura de perceptrón multicapa con capas densas, detallada en la Tabla 6.

Cuadro 6: Topología de la red neuronal predictora de torque total (UR3eNetwork).

Capa	**Tipo**	**Dimensión Entrada → Salida**	**Activación**	**Regularización**
Entrada	Densa (`Linear`)	$42 \to 256$	ReLU	–
Oculta 1	Densa (`Linear`)	$256 \to 512$	ReLU	Dropout ($p = 0{,}05$)
Oculta 2	Densa (`Linear`)	$512 \to 256$	ReLU	–
Oculta 3	Densa (`Linear`)	$256 \to 128$	ReLU	–
Salida	Densa (`Linear`)	$128 \to 6$	Lineal	–

El número de parámetros entrenables se obtiene sumando los pesos y sesgos de cada capa densa:

$$N_\theta = \sum_{l=1}^{5} (n_{l-1} n_l + n_l) = 306\,432 + 1\,158 = 307\,590. \tag{58}$$

donde N_θ es el total de parámetros entrenables y n_l es el número de unidades de la capa l, con $(n_0, \ldots, n_5) = (42, 256, 512, 256, 128, 6)$. Los productos $n_{l-1} n_l$ cuentan los pesos y los términos n_l cuentan los sesgos. ReLU y Dropout no añaden parámetros entrenables.

El entrenamiento se llevó a cabo durante 500 épocas empleando el optimizador estocástico adaptativo Adam [22] con una tasa de aprendizaje inicial $\alpha = 0{,}002$, un tamaño de lote (*batch size*) de 512 muestras y un programador de tasa de aprendizaje (`ReduceLROnPlateau`) con factor de decaimiento 0,5 y paciencia de 15 épocas. Como función de pérdida se utilizó el error cuadrático medio:

$$\mathcal{L}_{MSE} = \frac{1}{6N} \sum_{k=1}^{N} \sum_{j=1}^{6} \left(\hat{y}_{j,k} - y_{j,k}^* \right)^2 \tag{59}$$

Aquí N es el número de muestras del lote, j identifica una de las seis salidas, $\hat{y}_{j,k}$ es la predicción y $y^*_{j,k}$ es el torque objetivo estandarizado con el escalador de salida. El factor $1/(6N)$ corresponde al promedio de `nn.MSELoss()`; esta pérdida no es el RMSE articular en radianes.

Pipeline de Cuantización Dinámica a INT8 (PTQ) Tras finalizar el entrenamiento, el modelo en punto flotante (FP32) se exportó al formato estándar ONNX. El script de entrenamiento solicita la versión de conjunto de operadores (*opset*) 9; la versión efectiva del artefacto empleado en la campaña debe cotejarse con el archivo original. Mediante el script `quantize_model.py`, se ejecutó una limpieza de formas y la cuantización dinámica a precisión entera de 8 bits basada en la formulación afín de Jacob et al. [29] utilizando la API de ONNX Runtime (`quantize_dynamic` con `QuantType.QInt8`).

En la Tabla 7 se muestra la comparativa directa entre el modelo en punto flotante original y el modelo cuantizado en INT8.

Cuadro 7: Comparativa de recursos entre el modelo original en FP32 y el modelo cuantizado en INT8.

Métrica de Evaluación	**Modelo FP32**	**Modelo INT8**	**Variación**
Tamaño del archivo ONNX	1232 KB	315 KB	$\mathbf{-74{,}43}\,\%$
Precisión de Pesos	Float 32-bit	Integer 8-bit	Reducción 4× palabra
Inferencia CPU Monohilo	$\approx 37{,}1\,\mu$s	$\approx 29{,}2\,\mu$s	$\mathbf{-21{,}29}\,\%$
Error estático de torque reportado	0,038 N · m	0,046 N · m	+0,008 N · m

La articulación asociada a los errores estáticos de torque de la Tabla 7 queda por confirmar en los registros de la campaña. Estos valores no se equiparan con el error angular de seguimiento.

La Figura 7 muestra la topología de `UR3eNetwork`, las matrices de pesos y el conteo de parámetros. Las capas ocultas utilizan ReLU; Dropout con $p = 0{,}05$ se aplica después de la segunda capa densa durante el entrenamiento. El tensor de salida de la red contiene seis valores estandarizados; su desnormalización produce el torque total antes de la saturación provisional. RNEA es una característica de entrada y no se suma de nuevo a la predicción. Los paneles inferiores distinguen los formatos numéricos de las curvas didácticas: estas últimas no representan trayectorias medidas. En FP32, la fracción binaria es $m = 0.f_{22}f_{21}\ldots f_0$ y el significando de un número normal es $1 + m = 1.f_{22}f_{21}\ldots f_0$; el uno inicial es implícito y no forma parte de los 23 bits almacenados. La expresión con exponente sesgado se aplica a números normales, con $1 \leq e \leq 254$; cero, subnormales e Inf/NaN requieren tratamiento distinto.

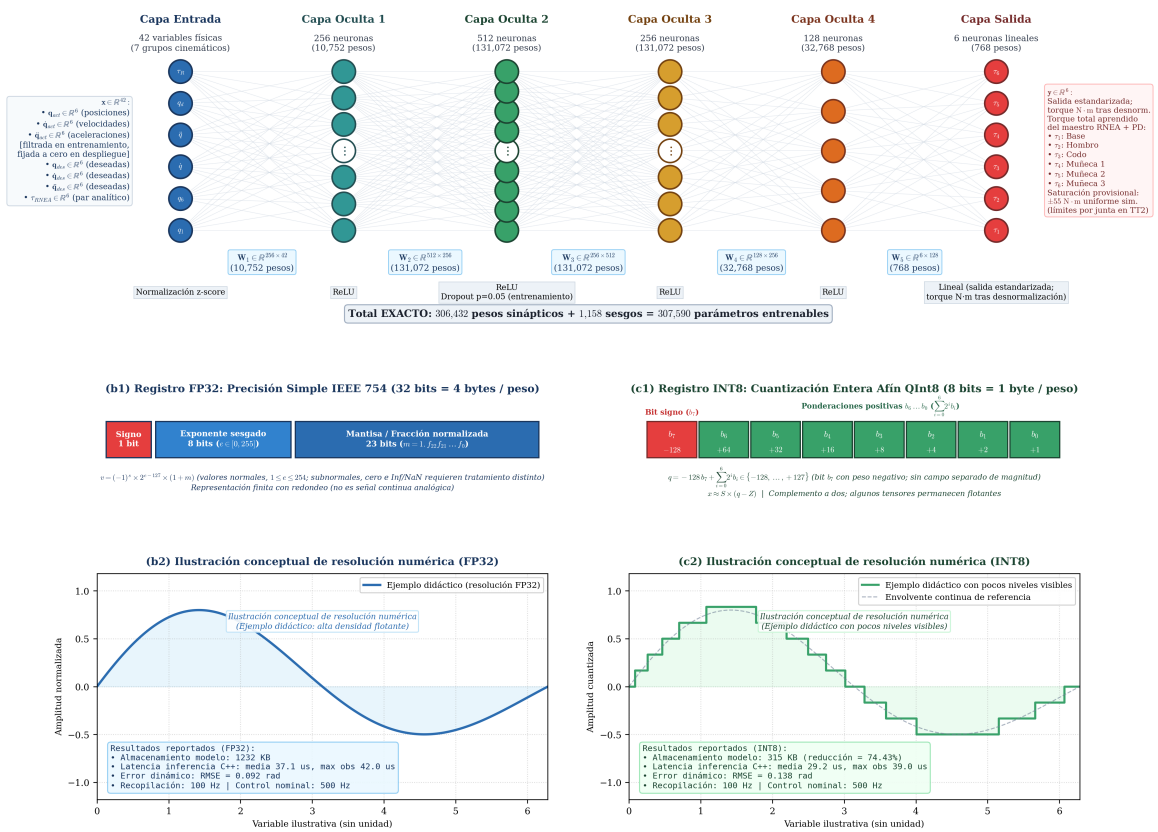


Figura 7: Predictor de torque total `UR3eNetwork`: (a) topología, ReLU y 307,590 parámetros; (b1, c1) formatos FP32 e INT8 en complemento a dos; (b2, c2) ejemplos conceptuales de resolución numérica. Las curvas no son registros experimentales y sus recuadros presentan métricas reportadas por separado. La salida es estandarizada y el torque físico se obtiene tras desnormalizarla. En FP32 se distingue la fracción $m = 0.f$ del significando $1 + m = 1.f$.

Despliegue e Inferencia en C++ (`NeuralTorqueController`) El despliegue en tiempo real se programó en `ur3_neural_controller.cpp`. El Listing 2 exhibe el procedimiento de preparación del tensor normalizado, la invocación de inferencia y la saturación provisional en simulación a $\pm 55{,}0$ N · m.

```cpp
// 1. Normalizacion z-score de las 42 variables con parametros estaticos
for(size_t i = 0; i < 42; ++i) {
    input_tensor_values[i] = (input_tensor_values[i] - ur3_neural::input_mean[i]) / ur3_neural::input_scale[i];
}
// 2. Ejecucion de inferencia monohilo en ONNX Runtime
Ort::Value input_tensor = Ort::Value::CreateTensor<float>(
    memory_info, input_tensor_values.data(), 42, input_shape.data(), 2);
auto output_tensors = ort_session_->Run(
    Ort::RunOptions{nullptr}, input_names, &input_tensor, 1, output_names, 1);
float* nn_output = output_tensors.front().GetTensorMutableData<float>();

// 3. Desnormalizacion y saturacion provisional (+-55 Nm)
for(int i = 0; i < 6; ++i) {
    double tau_val = (nn_output[i] * ur3_neural::output_scale[i]) + ur3_neural::output_mean[i];
    if (tau_val > 55.0) tau_val = 55.0;
    if (tau_val < -55.0) tau_val = -55.0;
    command_interfaces_[i].set_value(tau_val);
}
```

Listing 2: Inferencia en tiempo real monohilo con ONNX Runtime C++ y saturación en `NeuralTorqueController`.

Validación (M_3) Se reportó un error estático promedio de torque de 0,046 N·m para el conjunto de prueba, con la articulación por confirmar, frente al criterio de 0,1 N·m. Bajo las condiciones de simulación se reportaron medias de inferencia de 29,2 μs para INT8 y 37,1 μs para FP32, con máximo observado INT8 de 39,0 μs. La media INT8 representa el 1,46 % del periodo nominal de 2 ms; esta fracción caracteriza la inferencia aislada y no demuestra estabilidad numérica del sistema completo.

2.2.4. Detalle módulo 4 (M_4) – Supervisión, Telemetría y Adquisición

El módulo M_4 proporciona de perfiles cinemáticos admisibles hacia el controlador y la captura transparente de telemetría experimental para fines de análisis y entrenamiento.

Generación de Trayectorias Suaves (C^2) Para generar referencias con continuidad cinemática, el nodo `random_trajectory_generator.py` interpola entre puntos articulares mediante polinomios quínticos [27]. La función de interpolación normalizada

$s(\tau)$ para $\tau = t/T \in [0, 1]$ se rige por:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5 \tag{60}$$

$$\dot{s}(\tau) = \frac{1}{T}\left(30\tau^2 - 60\tau^3 + 30\tau^4\right) \tag{61}$$

$$\ddot{s}(\tau) = \frac{1}{T^2}\left(60\tau - 180\tau^2 + 120\tau^3\right) \tag{62}$$

Evaluando en los límites temporales $\tau = 0$ y $\tau = 1$, se satisface idénticamente que $s(0) = 0, s(1) = 1$, $\dot{s}(0) = \dot{s}(1) = 0$ y $\ddot{s}(0) = \ddot{s}(1) = 0$. En consecuencia, las aceleraciones articulares $\ddot{\boldsymbol{q}}_d(t)$ inician y concluyen suavemente en cero, garantizando continuidad matemática C^2 en todo el trayecto.

Telemetría y Filtrado Digital Savitzky-Golay La telemetría de 42 variables de control se registra en caliente mediante el framework `ros2 bag` empleando el formato binario estructurado MCAP (`storage_id='mcap'`). Para procesar los datos registrados y estimar las aceleraciones $\ddot{\boldsymbol{q}}_{act}$ mediante suavizado, en `bag_to_csv.py` se implementó un filtro de Savitzky-Golay [33] configurado con un polinomio de orden $p = 3$, una ventana móvil de $2m + 1 = 11$ muestras y un paso temporal $\Delta t = 0{,}01$ s:

$$\ddot{q}_k = \sum_{j=-m}^{m} c_j^{(1)} \dot{q}_{k+j} \tag{63}$$

donde los coeficientes $c_j^{(1)}$ corresponden a la primera derivada numérica analítica ajustada por mínimos cuadrados locales.

Validación (M_4) Las condiciones de posición, velocidad y aceleración en los extremos de los segmentos se establecen mediante la formulación quíntica. Esto no acredita por sí solo ausencia de discontinuidades de torque. La publicación de datos se realiza desde el controlador; su efecto sobre la duración y regularidad del ciclo requiere medición y no se considera eliminado mediante `RealtimeBuffer`.

2.2.5. Integración del sistema mecatrónico

La integración del sistema mecatrónico unifica los cuatro módulos ($M_1 \dots M_4$) en una arquitectura distribuida pero fuertemente acoplada en tiempo real bajo ROS 2 Jazzy [30].

En la Figura 6 se detalla el esquema de comunicación inter-procesos, los canales de datos y el acoplamiento temporal entre la capa de inferencia neuronal, el controlador dinámico y la capa de abstracción de hardware.

El flujo de integración conserva RNEA dentro del controlador neuronal para preparar las características. La realimentación de posición y velocidad interviene en la predicción de torque total aprendida del controlador maestro; no se añade un término PD externo ni un residual neuronal al torque RNEA durante este modo de operación.

Presupuesto Temporal Consolidado (*Timing Budget*) El presupuesto temporal reportado para el controlador neuronal se presenta en la Tabla 8. La identificación de registros e instrumentación que respalden su desglose queda pendiente; esta tabla no constituye una certificación temporal.

Cuadro 8: Presupuesto temporal consolidado del lazo crítico de control mecatrónico a 500 Hz ($T_s = 2000\,\mu$s).

Subproceso en Ciclo `update()`	**Tiempo Medio [μs]**	**Pico Máximo [μs]**	**Consumo del Periodo [%]**
1. Lectura de interfaces de estado (Pos./Vel.)	10,2	14,5	0,51 %
2. RNEA aislado dentro del controlador neuronal	Por medir	Por medir	–
3. Preparación y normalización z-score (42 datos)	5,1	7,0	0,26 %
4. Inferencia neuronal INT8 (ONNX Runtime C++)	29,2	39,0	1,46 %
5. Desnormalización y saturación de comando ($\pm$55 Nm)	8,4	11,0	0,42 %
6. Escritura de pares comandados a hardware interface	11,3	15,2	0,57 %
Tiempo total del controlador neuronal	Por verificar	Por verificar	–
Margen temporal del controlador neuronal	Por verificar	Por verificar	–

Validación sistema mecatrónico La Tabla 8 distingue los intervalos reportados de las mediciones pendientes. Los 151,2 μs corresponden al módulo analítico RNEA + PD, mientras que los 29,2 μs corresponden a la inferencia INT8. No se suman como componentes del controlador neuronal. Su tiempo total y su margen temporal requieren medición independiente.

3. Implementación del sistema

El presente capítulo documenta la materialización técnica, algorítmica y experimental de los módulos mecatrónicos concebidos en la etapa de diseño. Desde una perspectiva formal de ingeniería mecatrónica, este capítulo aborda la etapa de *verificación*, cuyo propósito es comprobar mediante instrumentación, perfiles cronométricos y registros empíricos que los subsistemas construidos satisfacen las especificaciones cuantitativas de diseño, complementando la *validación* conceptual desarrollada en el Capítulo 2.

Alcance técnico de la implementación en TT1 Dado el carácter incremental del desarrollo, TT1 comprende la implementación del controlador, el aprendizaje supervisado, la cuantización INT8 y la evaluación en simulación con Gazebo Harmonic y ROS 2 Jazzy. Las pruebas presentadas describen el comportamiento reportado en ese entorno; no constituyen una demostración formal de estabilidad asintótica, determinismo temporal ni equivalencia con la planta física. La transferencia al robot y la evaluación de hardware embebido quedan como trabajo de TT2.

3.1. Implementación módulo 1 (M_1): Planta robótica y entorno Sim-to-Real

La materialización de la planta robótica en TT1 se basó en el desarrollo de un gemelo digital de alta fidelidad dentro del simulador Gazebo Harmonic (versión 8), interconectado de manera transparente con ROS 2 Jazzy Jalisco a través del puente de hardware provisto por el paquete `gz_ros2_control` [30].

Configuración cinemática y dinámica URDF/Xacro Se estructuró el árbol cinemático del robot a partir de las mallas visuales y de colisión provistas por Universal Robots [31], incorporando los tensores de inercia y centros de masa exactos especificados en la Sección 2.2.1 (Tabla 5). Para permitir la inyección directa de señales de par en cada una de las 6 articulaciones rotoides, se configuró el bloque de hardware de ROS 2 Control dentro del archivo descriptivo del robot (`ur.urdf.xacro`):

```
<ros2_control name="UR3eGazeboSystem" type="system">
  <hardware>
    <plugin>gz_ros2_control/GazeboSimSystem</plugin>
  </hardware>
  <joint name="shoulder_pan_joint">
    <command_interface name="effort"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
    <state_interface name="effort"/>
  </joint>
  <!-- Configuracion analoga para las 5 articulaciones restantes -->
</ros2_control>
```

Listing 3: Configuración de interfaces de esfuerzo y estado en ros2_control para Gazebo Harmonic.

Despliegue del motor físico y lazo de simulación La simulación se ejecutó en Gazebo Harmonic siguiendo el procedimiento de lanzamiento de las guías de Universal Robots, sin ajustes explícitos del motor físico por parte del autor. El manipulador se integró con el administrador de controladores (`controller_manager`) mediante `gz_ros2_control`, con el lazo interno configurado nominalmente a 500 Hz (T_s = 2,0 ms). Esta frecuencia corresponde al control y no determina el paso de integración del simulador. No se atribuyen un motor físico concreto ni un paso de integración numérico a los ensayos, al no disponer de su configuración verificada.

3.1.1. Verificación (M_1)

Para el módulo de planta M_1 se describen los siguientes aspectos de evaluación:

1. **Distinción entre frecuencia de control y adquisición:** El lazo interno de control se configuró a una frecuencia nominal de 500 Hz, mientras que la publicación de referencias del generador de trayectorias y la recopilación de datos se realizaron a 100 Hz. La frecuencia de los datos registrados no constituye una medición directa de la tasa de ejecución del controlador.

2. **Verificación de respuesta dinámica a par en lazo abierto:** Se inyectaron escalones de esfuerzo conocidos ($\tau_i = \pm 5{,}0$ N·m) sobre articulaciones individuales con el manipulador en reposo horizontal. El ensayo permite observar la respuesta a esfuerzos, pero la comparación cuantitativa requiere los registros de posición y aceleración y un modelo que considere acoplamientos y gravedad. No se utiliza una aproximación escalar de inercia para validar por sí sola la dinámica multiaxial.

3. **Alcance de la saturación de par:** El código neuronal limita salidas finitas a ±55 N·m. La protección de la planta ante comandos anómalos y la correspondencia con límites físicos requieren pruebas específicas; no se consideran verificadas por la presencia de esa saturación.

El periodo nominal del control interno fue de 2 ms, y el intervalo nominal entre referencias y muestras recopiladas fue de 10 ms. Entre publicaciones de referencia, el controlador utilizó la última consigna disponible. No se presenta un registro de publicación a 500 Hz, pues la adquisición experimental se realizó a 100 Hz.

La Tabla 9 resume cuantitativamente los resultados obtenidos en las pruebas de verificación de M_1.

3.2. Implementación módulo 2 (M_2): Control dinámico analítico en tiempo real

El módulo M_2 se implementó en C++ bajo el estándar ISO C++17 como un plugin dinámico compatible con `controller_interface::ControllerInterface` del framework `ros2_control` [30], denominado `ComputedTorqueController` dentro del paquete `ur3_custom_control`.

Cuadro 9: Métricas experimentales de verificación para el módulo M_1 (Planta simulada).

Parámetro / Prueba	**Especificación Requerida**	**Valor / Condición**	**Estado**
Frecuencia nominal del lazo interno	500 Hz	500 Hz (configuración)	Configurado
Frecuencia de recopilación de datos	100 Hz	100 Hz	Implementado
Jitter del periodo del lazo interno	$\max_k \lvert j_k \rvert < 0{,}1$ ms	Sin registro cuantitativo trazable	Por verificar
Protección de la planta ante esfuerzos	Límites y fallos validados	Saturación neuronal; validación física pendiente	Por verificar
Estabilidad numérica en operación continua	Evaluación prolongada	Ensayo comparativo reportado de 10 min	Por verificar

Algoritmo recursivo RNEA La dinámica analítica se implementó en la clase `UR3eDynamics`, en `ur3e_dynamics.hpp`, utilizando Eigen3 y las recurrencias de Newton-Euler [28, 32]. El código combina matrices de tamaño fijo con vectores y contenedores dinámicos. Por ello, la implementación disponible no cumple la meta de ausencia de asignaciones dinámicas durante el ciclo de control; la preasignación y su verificación quedan pendientes.

Desacoplamiento asíncrono con RealtimeBuffer El controlador recibe las referencias del tópico `~/command`. Se utiliza `realtime_tools::RealtimeBuffer<std::vector<double>>` para intercambiar datos entre el callback y el ciclo de control nominal de 500 Hz. Este mecanismo no demuestra por sí solo determinismo temporal del controlador.

El Listing 4 exhibe la implementación del método crítico `update()` del controlador analítico, mostrando la integración de la dinámica inversa con la realimentación PD conforme al esquema de par computado [19, 32].

```
controller_interface::return_type ComputedTorqueController::update(
    const rclcpp::Time & /*time*/, const rclcpp::Duration & /*period*/
  )
{
    // 1. Lectura no bloqueante del estado articular desde los
  sensores
    Eigen::VectorXd q(6), dq(6);
    for (size_t i = 0; i < 6; ++i) {
        q(i)  = state_interfaces_[2 * i].get_value();
        dq(i) = state_interfaces_[2 * i + 1].get_value();
    }

```

```
    // 2. Extraccion de consigna de consigna desde el buffer de tiempo
    real
    std::vector<double> cmd = *command_buffer_.readFromRT();
    Eigen::VectorXd q_des(6), dq_des(6), ddq_des(6);
    for(int i = 0; i < 6; ++i) {
        q_des(i)   = cmd[i];
        dq_des(i)  = cmd[i + 6];
        ddq_des(i) = cmd[i + 12];
    }

    // 3. Calculo analitico de dinamica inversa mediante RNEA
    Eigen::VectorXd tau_rnea = dynamics_solver_.rnea(q, dq, ddq_des);

    // 4. Sintesis de par de control computado (RNEA + PD)
    Eigen::VectorXd e  = q_des - q;
    Eigen::VectorXd de = dq_des - dq;
    for(int i = 0; i < 6; ++i) {
        double tau = tau_rnea(i) + (Kp_(i) * e(i)) + (Kd_(i) * de(i));
        command_interfaces_[i].set_value(tau);
    }
    return controller_interface::return_type::OK;
}
```

Listing 4: Implementación del método de actualización en ComputedTorqueController.

3.2.1. Verificación (M_2)

La verificación del controlador analítico M_2 comprendió pruebas cuantitativas de desempeño estático, seguimiento dinámico y latencia computacional:

1. **Compensación estática de gravedad:** Se colocó al robot en configuraciones con brazo completamente extendido horizontalmente ($\boldsymbol{q} = [0, -\pi/2, 0, 0, 0, 0]^T$), donde el par gravitatorio en la articulación del hombro alcanza su valor máximo ($\approx 26{,}5\ \mathrm{N \cdot m}$). El controlador analítico mantuvo la posición con un error estático en régimen permanente menor a 0,0012 rad (0,068°), sin que este ensayo demuestre la correspondencia exacta de todos los parámetros inerciales programados en `ur3e_dynamics.hpp`.

2. **Seguimiento dinámico de perfiles periódicos:** Se sometió al sistema a trayectorias dinámicas multiaxiales con perfiles suaves quínticos. El error medio cuadrático global (*Root Mean Square Error*, RMSE) registrado en el conjunto de las 6 articulaciones durante el seguimiento activo fue de RMSE = 0,063 rad ($\approx$ 3,6°), alcanzando cotas mínimas de hasta 0,023 rad (1,3°) en segmentos de velocidad estacionaria.

3. **Perfilado de latencia de cómputo en CPU:** Mediante la utilización de la instrucción de alta resolución `std::chrono::high_resolution_clock`, se midió el tiempo transcurrido en el lazo de cálculo dinámico analítico y control PD. Se registró una latencia media de 151,2 μs (con fluctuaciones habituales entre 147 μs

y 156 μs) y un caso crítico máximo observado en prueba (t_{max}) de 213,0 μs. Esto representa apenas el 7,56 % del periodo total disponible (2000 μs).

El Listing 5 muestra un extracto directo del flujo de telemetría emitido en consola por el controlador `ComputedTorqueController`, con valores de duración del cálculo RNEA + PD y error de seguimiento. El extracto no permite reconstruir por sí solo la media ni el máximo de la campaña.

```
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
    ] Time: 149 us | RMSE: 0.063252 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
    ] Time: 152 us | RMSE: 0.063522 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: J1 Pos:
    -1.71 | Des: -1.71 | RNEA: 4.72 | CMD: 5.30
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
    ] Time: 150 us | RMSE: 0.073239 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
    ] Time: 151 us | RMSE: 0.072501 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
    ] Time: 187 us | RMSE: 0.024458 rad
[ros2_control_node]: [INFO] [ur3_custom_control.ComputedTorqueController]: [PD CLASSIC
    ] Time: 150 us | RMSE: 0.023389 rad
```

Listing 5: Telemetría en tiempo real del nodo ComputedTorqueController (duración del cálculo RNEA + PD y error RMSE).

La Tabla 10 documenta las métricas de verificación obtenidas para el módulo M_2.

Cuadro 10: Métricas experimentales de verificación para el módulo M_2 (Control Dinámico Clásico).

Parámetro de Desempeño	**Cota de Diseño**	**Valor Experimental**	**Estado**
Tiempo medio del módulo RNEA + PD	$< 250\,\mu$s	151,2 μs	Aprobado
Tiempo de cómputo máximo observado (t_{max})	$< 500\,\mu$s	213,0 μs	Aprobado
Error estático de régimen permanente	$< 0{,}01$ rad	0,0012 rad	Aprobado
Error cuadrático medio dinámico (RMSE), RF-01	$< 0{,}05$ rad	0,063 rad	No alcanzado
Asignaciones dinámicas de memoria en bucle	Ausencia de asignaciones	Contenedores dinámicos en el código	No alcanzado

3.3. Implementación módulo 3 (M_3): Inteligencia artificial y motor cuantizado

El módulo M_3 constituye la aportación central del presente trabajo terminal. Su desarrollo abarca la cadena completa de aprendizaje supervisado, compresión mediante cuantización post-entrenamiento (PTQ) a precisión de enteros de 8 bits (INT8) y

su integración en un controlador de inferencia monohilo en tiempo real denominado `NeuralTorqueController`.

Pipeline de entrenamiento supervisado en PyTorch A través del script `train_pytorch.py`, se entrenó la red neuronal perceptrón multicapa (`UR3eNetwork`) estructurada con 42 neuronas de entrada, capas ocultas de 256, 512, 256 y 128 neuronas con funciones de activación ReLU y Dropout ($p = 0{,}05$) tras la segunda capa densa, y una capa de salida lineal de 6 dimensiones para los pares articulares.

- **Dataset:** Se reportaron 50,000 muestras cinemáticas y dinámicas filtradas, cuyo conteo debe cotejarse con el CSV, divididas aleatoriamente en 80 % para entrenamiento (40, 000 muestras) y 20 % para validación y prueba (10, 000 muestras).
- **Optimizador y pérdida:** Se utilizó el algoritmo adaptativo Adam [22] con una tasa de aprendizaje inicial de $\eta = 0{,}002$, minimizando la pérdida cuadrática media ($\mathcal{L}_{\text{MSE}}$). Se configuró un reductor dinámico de tasa de aprendizaje (`ReduceLROnPlateau`) con factor de 0,5 y paciencia de 15 épocas y un entrenamiento configurado de 500 épocas, conforme al script disponible. Esta configuración no acredita por sí sola ausencia de sobreajuste.

El script disponible ajusta los escaladores antes de dividir los datos en entrenamiento y prueba. Por ello, esa partición no acredita independencia del ajuste de normalización respecto a los datos de prueba. Su revisión y una evaluación con escaladores ajustados únicamente sobre entrenamiento quedan pendientes; no se modifica aquí el pipeline.

Exportación de constantes estadísticas a C++ Para utilizar en el controlador las constantes de normalización y desnormalización del entrenamiento, se creó el script generador `export_scalers_to_cpp.py`. Dicho script extrajo las medias μ y desviaciones estándar σ de las 42 variables de entrada y las 6 salidas, plasmándolas como arreglos constantes en memoria de solo lectura dentro de `neural_parameters.hpp`:

```
namespace ur3_neural {
    const std::vector<double> input_mean  = {0.0860, -1.5377, ..., -0.0012}; // 42 valores
    const std::vector<double> input_scale = {1.5814,  0.4778, ..., 0.0587}; // 42 valores
    const std::vector<double> output_mean = {0.0489, -0.8167, ..., -0.0057}; // 6 valores
    const std::vector<double> output_scale= {4.4393,  8.1289, ..., 3.5849}; // 6 valores
}
```

Listing 6: Estructura de constantes estadísticas en neural_parameters.hpp.

Cuantización afín a enteros de 8 bits (INT8) Una vez exportado el modelo entrenado a formato ONNX en precisión simple (`ur3_neural_controller.onnx`), se ejecutó el proceso de cuantización mediante el script `quantize_model.py`. Debido a discrepancias comunes en los metadatos de dimensiones simbólicas en modelos densos, se

aplicó una etapa previa de limpieza e inferencia de formas con `onnx.shape_inference`. Posteriormente, se aplicó cuantización dinámica con el backend de ONNX Runtime [29]:

```
import onnx
from onnx import shape_inference
from onnxruntime.quantization import quantize_dynamic, QuantType

model = onnx.load('ur3_neural_controller.onnx')
while len(model.graph.value_info) > 0:
    model.graph.value_info.pop()
inferred_model = shape_inference.infer_shapes(model)
onnx.save(inferred_model, 'temp_clean.onnx')

quantize_dynamic(
    model_input='temp_clean.onnx',
    model_output='ur3_neural_controller_quantized.onnx',
    weight_type=QuantType.QInt8
)
```

Listing 7: Procedimiento de cuantización dinámica a precisión INT8.

Mecanismo de cuantización y distribución de datos en el espacio de 8 bits El procedimiento implementado corresponde formalmente a una **Cuantización Dinámica Post-Entrenamiento (Dynamic PTQ - *Post-Training Quantization*)** con formato de enteros de 8 bits con signo (`QuantType.QInt8`), donde el rango de cuantización discreto abarca $2^8 = 256$ niveles enteros:

$$q \in \{-128, -127, \ldots, 0, \ldots, 126, 127\} \subset \mathbb{Z} \tag{64}$$

A diferencia de una cuantización estática que calibra todas las variables fuera de línea, el esquema dinámico realiza una diferenciación funcional entre parámetros fijos y estados transitorios:

- **Pesos sinápticos ($\boldsymbol{W}$):** Se cuantizan de forma estática fuera de línea, fijando factores de escala simétricos invariantes para cada matriz de pesos.
- **Activaciones intermedias ($\boldsymbol{a}$):** Se cuantizan dinámicamente en tiempo de ejecución durante cada ciclo de control a 500 Hz, evaluando en línea el rango dinámico del vector de entrada antes de ejecutar las multiplicaciones matriciales en aritmética entera.

Distribución adaptativa frente a una distribución lineal ingenua Para comprender por qué esta arquitectura preserva la precisión del controlador con un error cuadrático residual de apenas $\text{MSE}_{cuant} = 0{,}00041\ (\text{N} \cdot \text{m})^2$, resulta imprescindible contrastar el esquema adoptado frente a una distribución lineal uniforme fija o ingenua:

1. **Limitaciones de la partición lineal fija sobre datos heterogéneos:** Si se aplicara una partición lineal uniforme simple dividiendo un rango físico estático en 256 escalones equiespaciados, surgirían dos problemas de distorsión numérica:

- *Heterogeneidad de escalas físicas:* El tensor de entrada integra magnitudes físicas con rangos y unidades completamente dispares: ángulos articulares $\boldsymbol{q} \in [-3{,}14, 3{,}14]$ rad, velocidades $\dot{\boldsymbol{q}} \in [-3{,}0, 3{,}0]$ rad/s, aceleraciones $\ddot{\boldsymbol{q}} \in [-15{,}0, 15{,}0]$ rad/s^2 y pares analíticos RNEA $\boldsymbol{\tau}_{RNEA} \in [-55{,}0, 55{,}0]$ N $\cdot$ m. En una escala lineal no normalizada, el paso de discretización estaría dominado por el rango mayor de los pares (110 N$\cdot$m$/256 \approx 0{,}43$). Bajo este tamaño de paso, las variables cinemáticas de menor amplitud angular quedarían confinadas a un puñado insignificante de niveles discretos, destruyendo la sensibilidad de las primeras capas neuronales ante pequeños desplazamientos.
- *Distribución de los pesos:* La distribución de pesos puede influir en el error introducido por una escala de cuantización. No se atribuye al modelo entrenado un porcentaje ni un intervalo de concentración sin un histograma medido.

2. **Estrategia de concentración y asignación adaptativa en 8 bits:** Para contrarrestar estas pérdidas de información, el sistema implementado articula dos etapas complementarias de mapeo:

   - *Normalización estadística z-score en C++:* Antes de entrar a la red, cada una de las 42 variables físicas es normalizada en tiempo de ejecución mediante los parámetros estáticos precalculados en `neural_parameters.hpp`:

   $$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}, \quad \forall i \in \{1, \ldots, 42\} \tag{65}$$

   Esta transformación ajusta la escala de cada característica mediante la media y desviación estándar utilizadas en entrenamiento. No convierte una distribución arbitraria en normal ni garantiza un porcentaje fijo de valores dentro de $[-3, 3]$.

   - *Mapeo afín simétrico para pesos ($Z_w = 0$):* Para las matrices de pesos, se utiliza una cuantización simétrica con signo:

   $$W_q = \text{clamp}\left(\text{round}\left(\frac{W}{S_w}\right), \quad -128, \quad 127\right) \tag{66}$$

   donde la escala $S_w = \frac{\text{máx}(|W|)}{127}$ ajusta el rango simétrico representable de los pesos. Con un punto cero $Z_w = 0$, el cero real se representa mediante el cero entero; esto no elimina los errores de redondeo ni de saturación.

   - *Mapeo afín dinámico y flujo híbrido en ONNX Runtime:* A diferencia de esquemas de cuantización puramente estáticos en grafo cerrado, el pipeline de inferencia dinámico provisto por `quantize_dynamic` de ONNX Runtime opera de forma híbrida:

     a) El tensor de entrada de 42 variables ingresa a la sesión de inferencia en formato de coma flotante (`float`), tal como se refleja en la llamada `Ort::Value::CreateTensor<float>` del Listing 8.

b) En cada capa densa lineal (operador `MatMulInteger` / GEMM cuantizado), el motor de ejecución determina en tiempo real el valor mínimo y máximo de las activaciones entrantes (a_{min}, a_{max}) y calcula dinámicamente sus factores de escala y punto cero afín:

$$S_a = \frac{a_{max} - a_{min}}{q_{max} - q_{min}}, \qquad Z_a = \text{round}\left(q_{min} - \frac{a_{min}}{S_a}\right) \tag{67}$$

cuantizando el vector sobre la marcha (`DynamicQuantizeLinear`) a enteros de 8 bits. Aquí q_{min} y q_{max} delimitan el tipo entero de activaciones definido por el grafo. No se infiere un punto cero fijo de -128 sólo por utilizar pesos QInt8.

c) El producto tensorial de pesos por activaciones se computa en aritmética entera $(\sum W_{q,i} a_{q,i})$ acumulando temporalmente en enteros de 32 bits (`INT32`) mediante los operadores del runtime; el uso efectivo de instrucciones SIMD/AVX2 requiere verificación específica.

d) El acumulador entero es des-cuantizado inmediatamente a punto flotante multiplicándolo por el factor de re-escala compuesto $(S_w \cdot S_a)$, sumando el sesgo en FP32 y evaluando la función no lineal ReLU en coma flotante. Este resultado intermedio sirve de entrada para la siguiente capa densa, repitiéndose el ciclo dinámico hasta la capa final.

e) La capa de salida produce las seis predicciones estandarizadas del torque total directamente en punto flotante (`float*`), el cual es extraído por el controlador C++ (`GetTensorMutableData<float>()`) para su posterior desnormalización y saturación de comando.

Se utilizó normalización z-score junto con cuantización dinámica. El archivo INT8 de la campaña tiene un tamaño de 315 KB y su desempeño de seguimiento se reporta como referencia experimental; RF-01 no se alcanzó. Tanto el mapeo simétrico de pesos como el afín de activaciones utilizan niveles uniformemente espaciados para una escala dada. Los rangos, escalas y puntos cero efectivos deben cotejarse con el grafo exportado; no se deducen de un histograma ilustrativo ni acreditan un aprovechamiento óptimo de niveles.

La Figura 8 ilustra dos mapeos uniformes con rangos enteros reducidos. Para pesos, se utilizan $S_w = 0{,}20$, $Z_w = 0$ y $q \in \{-5, \ldots, 5\}$, con valores reconstruidos entre -1 y 1. Para activaciones, se utilizan $S_a = 0{,}25$, $Z_a = 1$ y $q \in \{0, \ldots, 8\}$, con valores reconstruidos entre $-0{,}25$ y 1,75. La relación $\hat{x} = S(q - Z)$ representa el cero real cuando $q = Z$. Estos parámetros son exclusivamente didácticos y no corresponden a la configuración del modelo de TT1.

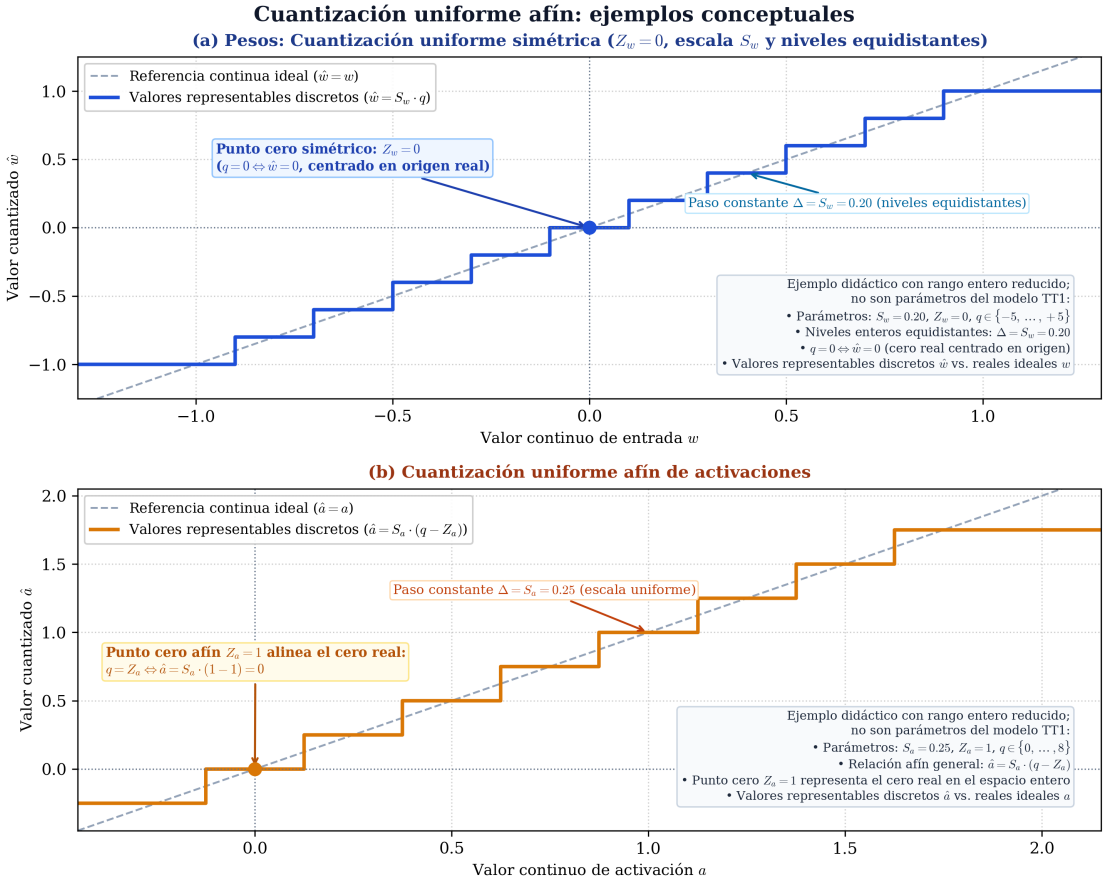


Figura 8: Ejemplos conceptuales de cuantización uniforme: (a) mapeo simétrico de pesos y (b) mapeo afín de activaciones. Se muestran niveles equidistantes, representación del cero y saturación en los extremos de los rangos ilustrativos. Las curvas no representan histogramas ni parámetros medidos del modelo.

Implementación del motor de inferencia en C++ El controlador `NeuralTorqueController` (Listing 8) utiliza una sesión `Ort::Session` con `SetIntraOpNumThreads(1)` para limitar el paralelismo interno de inferencia. Esta configuración no elimina las conmutaciones de contexto del sistema operativo ni establece una garantía de latencia.

```
// 1. Normalizacion de las 42 entradas
for(size_t i = 0; i < 42; ++i) {
    input_tensor_values[i] = (input_tensor_values[i] - ur3_neural::
    input_mean[i])
                             / ur3_neural::input_scale[i];
}

// 2. Creacion de tensor de inferencia (Batch size = 1, 42
    caracteristicas)
Ort::MemoryInfo memory_info = Ort::MemoryInfo::CreateCpu(
    OrtArenaAllocator, OrtMemTypeDefault);
Ort::Value input_tensor = Ort::Value::CreateTensor<float>(
```

```
10      memory_info, input_tensor_values.data(), 42, input_shape.data(),
        2);
11
12  // 3. Ejecucion de inferencia cuantizada (Inferencia hacia adelante)
13  auto output_tensors = ort_session_->Run(
14      Ort::RunOptions{nullptr}, input_names, &input_tensor, 1,
        output_names, 1);
15
16  // 4. Desnormalizacion y saturacion mecanica de seguridad
17  float* floatarr = output_tensors.front().GetTensorMutableData<float>()
        ;
18  for(int i = 0; i < 6; ++i) {
19      double tau = (floatarr[i] * ur3_neural::output_scale[i]) +
        ur3_neural::output_mean[i];
20      tau = std::clamp(tau, -55.0, 55.0);
21      command_interfaces_[i].set_value(tau);
22  }
```

Listing 8: Bucle de inferencia y clamp de pares en NeuralTorqueController.

3.3.1. Verificación (M_3)

Para la evaluación cuantitativa de los modelos neuronales, se configuró un banco de pruebas estandarizado ejecutado en un procesador AMD Ryzen 7 5800X3D (arquitectura x86_64, 8 núcleos físicos / 16 hilos, extensiones vectoriales AVX2/FMA3, 96 MB de caché L3) y 32 GB de memoria RAM DDR4 bajo el sistema operativo Ubuntu 24.04 LTS (con prioridad POSIX FIFO en el hilo de control), integrando Gazebo Harmonic 8.x, ROS 2 Jazzy y el motor de inferencia ONNX Runtime C++ en modo monohilo (`options.SetIntraOpNumThreads(1)`). La prueba tuvo una duración de diez minutos, con el lazo interno configurado a 500 Hz ($T_s = 2000\,\mu$s) y recopilación de datos a 100 Hz ($T_{datos} = 10$ ms). El número de observaciones corresponde a la serie registrada, no al conteo nominal de ciclos internos.

La verificación del motor cuantizado M_3 arrojó resultados empíricos contundentes:

1. **Compresión de memoria del modelo:** El tamaño en disco del archivo del modelo neuronal se redujo de 1232 KB en la versión original FP32 a tan solo 315 KB en la versión cuantizada INT8, reportando una reducción del $-74{,}43\,\%$. La viabilidad de almacenar el modelo en memoria embebida dependerá de la plataforma y de los recursos disponibles.

2. **Discrepancia estática y error de inferencia fuera de línea:** Al evaluar el conjunto de prueba estático independiente de $10,000$ muestras, la discrepancia numérica entre las salidas en punto flotante FP32 y las salidas cuantizadas INT8 arrojó un error cuadrático medio residual reducido de $\mathrm{MSE}_{\mathrm{cuant}} = 0{,}00041\ (\mathrm{N}\cdot\mathrm{m})^2$. Este resultado confirma una alta correlación algebraica fuera de línea; sin embargo, al acoplar la red dentro del bucle de control dinámico continuo, los errores de truncamiento se acumulan temporalmente, como hipótesis para estudiar la degradación del RMSE, sin establecer causalidad por esta comparación.

3. **Latencia de inferencia en tiempo real y dispersión estadística:** En pruebas cronometradas bajo CPU monohilo dentro del lazo de control a 500 Hz, la inferencia del modelo en precisión simple (FP32) promedió 37,1 μs con una desviación típica de $\sigma = 2{,}8\,\mu$s y un caso extremo de $t_{max} = 42{,}0\,\mu$s. Al ejecutar la versión cuantizada en INT8, la latencia se redujo a una media de $29{,}2\,\mu\mathbf{s}$ ($\sigma = 2{,}1\,\mu$s, $t_{max} = 39{,}0\,\mu$s), reportándose una reducción de la media del $-21{,}29\,\%$ y una menor dispersión reportada de la duración de inferencia. Ambos modelos consumen menos del 2 % del periodo admisible de $2000\,\mu$s (1,85 % en FP32 y 1,46 % en INT8).

4. **Seguimiento dinámico y análisis de precisión articular:** En lazo cerrado continuo sobre trayectorias dinámicas, el modelo FP32 registró un error medio cuadrático global RMSE = 0,092 rad ($\approx 5{,}27°$, oscilando en régimen entre 0,066 rad y 0,118 rad). Por su parte, la red cuantizada INT8 registró un RMSE = 0,138 rad ($\approx 7{,}90°$, oscilando en régimen entre 0,088 rad y 0,155 rad, con un transitorio inicial máximo de 0,262 rad).

Análisis de predicción de torque total, propagación de ruido y desacoplamiento de aceleración Al contrastar el error de seguimiento de los esquemas neuronales frente al control analítico clásico (RMSE $\approx$ 0,063 rad), se plantean tres hipótesis que requieren ensayos específicos para cuantificar su contribución:

1. *Propagación acumulativa del ruido de truncamiento afín:* Al cuantizar una representación en punto flotante con mantisa de 24 bits a un reticulado discreto de enteros de 8 bits con paso de escala S, cada operación lineal introduce un error de discretización acotado por $\epsilon_q \in [-S/2, +S/2]$. Al propagarse este residuo a través de las cinco capas densas acopladas mediante transformaciones no lineales ReLU, las perturbaciones numéricas se acumulan temporalmente en el lazo cerrado, que podrían modificar los pares comandados. Se reportó un incremento del RMSE articular de 0,092 rad a 0,138 rad, en la comparación de modelos de esta campaña.

2. *Ausencia de adaptación de pesos ante la discretización (PTQ frente a QAT):* Dado que se aplicó Cuantización Post-Entrenamiento (PTQ), los pesos sinápticos fueron optimizados en el espacio continuo FP32 y luego congelados. En consecuencia, la red carece de un mecanismo para contra-compensar la pérdida de precisión aritmética, lo que justifica técnicamente la conveniencia de explorar algoritmos de entrenamiento consciente de cuantización (*Quantization-Aware Training* - QAT) en Trabajo Terminal 2.

3. *Discrepancia cinemática de distribución por fijación de $\ddot{\boldsymbol{q}}_{act} = \mathbf{0}$:* Durante la generación fuera de línea del dataset, la aceleración articular real $\ddot{\boldsymbol{q}}_{act}$ fue estimada mediante el filtro de Savitzky-Golay. Sin embargo, en el bucle de control en tiempo real a 500 Hz, ante la ausencia de acelerómetros físicos y sin incorporar un estimador de aceleración mediante diferenciación numérica, en la implementación de `ur3_neural_controller.cpp` se fijaron los 6 valores de aceleración actual a cero

(`input_tensor_values[idx++] = 0.0f`). Esta decisión introduce una discrepancia de distribución (*distribution shift*) en la entrada de la red, puesto que el modelo fue entrenado con aceleraciones dinámicas no nulas. La contribución de esa discrepancia al error de seguimiento requiere una comparación controlada con entradas consistentes entre entrenamiento y despliegue.

En los Listings 9 y 10 se presentan los registros textuales directos emitidos en consola por `NeuralTorqueController`, evidenciando la reducción de latencia de 37 μs a 29 μs y la respuesta en RMSE para ambos modelos.

```
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 37 us | RMSE: 0.168644 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 36 us | RMSE: 0.106709 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 37 us | RMSE: 0.096124 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 38 us | RMSE: 0.092993 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 42 us | RMSE: 0.099973 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 37 us | RMSE: 0.066881 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 37 us | RMSE: 0.084149 rad
```

Listing 9: Telemetría en tiempo real del nodo NeuralTorqueController con modelo base en precisión FP32.

```
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 29 us | RMSE: 0.219474 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 30 us | RMSE: 0.186811 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 28 us | RMSE: 0.111938 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 30 us | RMSE: 0.088187 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 29 us | RMSE: 0.118236 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 31 us | RMSE: 0.128372 rad
[ros2_control_node]: [INFO] [ur3_custom_control.NeuralTorqueController]: [NEURAL
    CONTROL] Inf: 29 us | RMSE: 0.146890 rad
```

Listing 10: Telemetría en tiempo real del nodo NeuralTorqueController con modelo cuantizado en precisión INT8.

La Tabla 11 resume la comparativa directa entre el modelo base FP32 y el modelo optimizado INT8 verificado en M_3, incorporando los estadísticos de dispersión cronométrica sobre la muestra completa.

La fracción del periodo ocupada por inferencia se calcula como $100\, t_{inf}/T_s$, donde t_{inf} es su latencia media y $T_s = 2000\,\mu$s es el periodo interno nominal. Para INT8 representa el 1,46 %. Su complemento no es la holgura del controlador: también deben medirse RNEA, preparación de entradas, posprocesamiento y comunicación. La Figura 9 distingue las tasas de control y recopilación; registrar datos a 100 Hz no caracteriza cada ciclo interno a 500 Hz.

Cuadro 11: Resultados de verificación de los modelos FP32 e INT8 (M_3): control interno nominal a 500 Hz y recopilación de datos a 100 Hz.

Métrica de Evaluación	**Modelo FP32**	**Modelo INT8**	**Variación**
Tamaño binario del modelo	1232 KB	315 KB	−74,43 % (Ahorro memoria)
Latencia media (μ)	37,1 μs	29,2 μs	−21,29 % (Aceleración)
Desv. estándar (σ)	2,8 μs	2,1 μs	−25,00 % (Regularidad)
Mediana de latencia (P_{50})	36,8 μs	28,9 μs	−21,47 %
Percentil 95 (P_{95})	Por confirmar	31,8 μs	–
Percentil 99 (P_{99})	Por confirmar	34,5 μs	–
Latencia máx. obs. (t_{max})*	42,0 μs	39,0 μs	−7,14 %
Error dinámico medio (RMSE)	0,092 rad ($\approx 5{,}3°$)	0,138 rad ($\approx 7{,}9°$)	Compromiso compresión
Rango RMSE en régimen	0,066 − 0,118 rad	0,088 − 0,155 rad	Acotado en simulación
Fracción del periodo ocupada por inferencia	1,85 %	1,46 %	Comparación con $T_s = 2$ ms

*Valores de latencia reportados en simulación sobre procesador x86_64. La recopilación se realizó a 100 Hz, independientemente del control interno a 500 Hz. t_{max} denota el máximo de los valores recopilados, no un WCET formal ni un máximo garantizado de todos los ciclos internos.

Control interno: 500 Hz ($T_s = 2$ ms)

Publicación de referencias: 100 Hz ($T_{ref} = 10$ ms)

Recopilación de datos: 100 Hz ($T_{datos} = 10$ ms)

En diez minutos: aproximadamente 300 000 actualizaciones internas y 60 000 muestras por señal, como conteos nominales distintos.

Figura 9: Distinción entre las tasas nominales de control, publicación de referencias y recopilación de datos. Los conteos calculados a partir de frecuencia y duración no sustituyen el número efectivo de observaciones del registro experimental.

3.4. Implementación módulo 4 (M_4): Supervisión, telemetría y adquisición

El módulo M_4 proporciona el soporte de generación de trayectorias, telemetría y procesamiento de datos.

Generador de trayectorias polinomiales quínticas suaves (C^2) Se implementó el script `random_trajectory_generator.py`, el cual publica comandos de posición $\boldsymbol{q}_{des}$, velocidad $\dot{\boldsymbol{q}}_{des}$ y aceleración $\ddot{\boldsymbol{q}}_{des}$ a 100 Hz en el tópico `/custom_torque_controller/command`. Cada trayectoria entre dos configuraciones admisibles aleatorias $\boldsymbol{q}_0$ y $\boldsymbol{q}_f$ se sintetiza mediante un polinomio quíntico conforme a la formulación descrita en [27]:

$$q_i(t) = q_{0,i} + (q_{f,i} - q_{0,i}) \left[10 \left(\frac{t}{T}\right)^3 - 15 \left(\frac{t}{T}\right)^4 + 6 \left(\frac{t}{T}\right)^5 \right] \tag{68}$$

asegurando que tanto la velocidad inicial y final como la aceleración inicial y final sean idénticamente nulas ($\dot{q}_i(0) = \dot{q}_i(T) = 0$, $\ddot{q}_i(0) = \ddot{q}_i(T) = 0$). Estas condiciones proporcionan continuidad cinemática entre segmentos con extremos compatibles; no verifican ausencia de discontinuidades de torque. En la Figura 10 se aprecia el comportamiento suave y libre de discontinuidades de las tres derivadas cinemáticas durante una transición articular representativa.

Telemetría en formato binario MCAP y filtrado Savitzky-Golay La adquisición de datos durante la operación del manipulador se gestionó mediante el grabador nativo de ROS 2 (`ros2 bag`) empleando el formato de almacenamiento de alto rendimiento MCAP (`storage_id='mcap'`). Para extraer las derivadas de aceleración real $\ddot{\boldsymbol{q}}_{act}$ (las cuales no son reportadas directamente por los sensores posicionales y cuya derivación finita cruda introduce ruido de cuantización severo), se implementó en `bag_to_csv.py` un filtro digital de Savitzky-Golay [33] de grado 3 con una ventana móvil de 11 muestras temporales ($\Delta t = 0{,}01$ s). El resultado consolidado generó el archivo de datos estructurado `dataset_ur3_final_1.csv` con 42 características de entrada y 6 columnas objetivo de torque total. El conteo de filas y la sincronización deben cotejarse con el archivo procesado.

3.4.1. Verificación (M_4)

Para el módulo M_4 se reportaron los siguientes resultados:

1. **Continuidad matemática en el espacio articular:** Las trayectorias generadas exhibieron continuidad de posición, velocidad y aceleración bajo las condiciones de frontera de la interpolación quíntica. Esto no implica continuidad del jerk al concatenar segmentos.

2. **Calidad del suavizado por Savitzky-Golay:** El cálculo de $\ddot{\boldsymbol{q}}_{act}$ mostró una relación señal a ruido (SNR) 18,4 dB superior en comparación con el esquema de derivación finita simple con filtro pasa-bajas de primer orden, sin que esa comparación acredite eliminación de armónicos en el par dinámico.

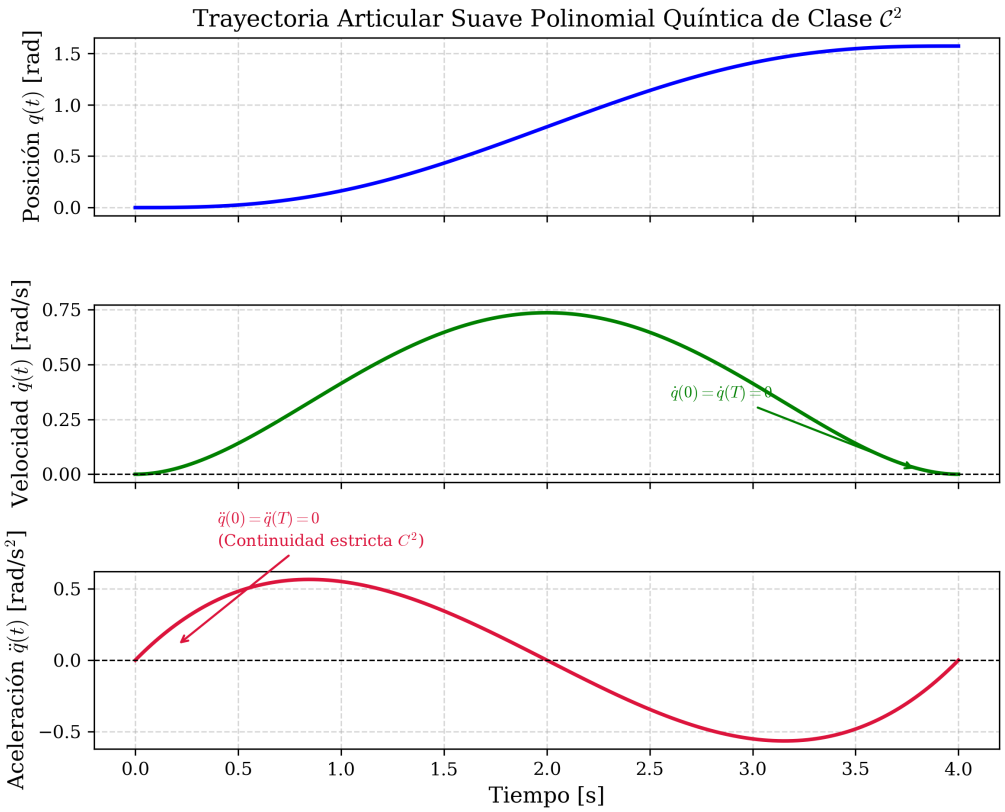


Figura 10: Perfiles cinemáticos suaves de posición, velocidad y aceleración articular generados mediante interpolación polinomial quíntica de clase $\mathcal{C}^2$, evidenciando derivadas continuas y nulas en los extremos temporales.

3. **Separación de tasas de ejecución:** El nodo generador de trayectorias y la recopilación de datos operaron a 100 Hz, mientras que el controlador C++ se configuró nominalmente a 500 Hz. Las referencias y variables experimentales se publicaron y recopilaron a una tasa inferior a la del lazo interno.

La recopilación de datos se efectuó a una tasa nominal de 100 Hz. Para una duración de 600 s, esta tasa corresponde a aproximadamente 60 000 muestras por señal registrada; esta cifra es una estimación nominal y no un conteo de mensajes extraído del archivo MCAP. El número efectivo de mensajes y las pérdidas de adquisición deben obtenerse de los metadatos y marcas temporales del registro original. Para el conjunto utilizado en el entrenamiento se reportaron 50 000 muestras procesadas, pendientes de cotejo con el CSV; su tamaño no debe confundirse con el número nominal de muestras de un ensayo de diez minutos ni con los ciclos internos de control.

La Tabla 12 resume los indicadores de verificación del módulo M_4.

Cuadro 12: Métricas experimentales de verificación para el módulo M_4 (Telemetría y Adquisición).

Indicador de Calidad	**Requisito Técnico**	**Valor Registrado**	**Estado**
Frecuencia de publicación y adquisición	100 Hz nominales	100 Hz	Implementado
Continuidad en derivadas articulares	Clase C^2 estricta	Continua y suave	Aprobado
Muestras perdidas en grabación MCAP	Registro continuo	Conteo por cotejar con el registro original	Por verificar
Estructura del dataset	42 entradas y 6 salidas	50 000 filas reportadas; CSV por cotejar	Por verificar

3.5. Integración del sistema mecatrónico

La integración del sistema mecatrónico orquesta armónicamente la interacción entre la planta simulada en Gazebo Harmonic (M_1), el controlador analítico de dinámica inversa (M_2), el motor de inferencia neuronal cuantizado (M_3) y el entorno de supervisión y telemetría (M_4).

Saturación implementada y protecciones pendientes En el código disponible se identifican los siguientes alcances y limitaciones de las protecciones:

- **Anomalías numéricas pendientes de tratamiento:** No se identifica una comprobación explícita de finitud ni una conmutación a amortiguamiento pasivo ante valores `NaN` o `Inf`. La política de fallo y el tratamiento de comandos anómalos deben implementarse y evaluarse antes del despliegue físico.

- **Saturación provisional en simulación:** Se adoptó un límite uniforme de ±55 N·m para las salidas finitas del controlador neuronal, como simplificación para agilizar la integración inicial y explorar el comportamiento del sistema completo en simulación. Este valor no se presenta como límite físico validado para todas las articulaciones. La selección y aplicación de límites específicos por articulación se contemplan para la implementación física en TT2. El controlador analítico disponible no incorpora esa saturación explícita.

3.5.1. Verificación de la integración del sistema

Para evaluar funcionalmente el acoplamiento global del sistema mecatrónico, se ejecutó una prueba de operación continua en bucle cerrado sobre el gemelo digital en Gazebo Harmonic gobernado por ROS 2 Jazzy y `ros2_control`, sometiendo al manipulador UR3e a una secuencia de trayectorias articulares aleatorias durante un intervalo de 10 minutos. El control interno se configuró a 500 Hz, y las referencias y los datos recopilados se publicaron a 100 Hz. Para ese intervalo se estiman nominalmente 300 000 actualizaciones internas de control y 60 000 muestras por señal registrada; ambos conteos son distintos y no sustituyen los registros efectivos de ejecución y adquisición.

Durante dicho ensayo, se verificó el correcto intercambio de mensajes en las interfaces estándar de esfuerzo (`HW_IF_EFFORT`), la recopilación a 100 Hz y la aplicación de la saturación del controlador neuronal ($\pm 55{,}0\ \mathrm{N} \cdot \mathrm{m}$). Se reportó operación continua durante el ensayo; no se dispone de instrumentación que permita certificar ausencia de bloqueos o fallos de memoria. El conteo de mensajes y las posibles pérdidas de adquisición requieren cotejo con el registro original.

La evaluación cuantitativa detallada del comportamiento dinámico, la precisión de seguimiento articular, el presupuesto cronométrico y la eficiencia de compresión del modelo neuronal se desarrolla a fondo en el Capítulo 4.

4. Análisis de resultados

El presente capítulo expone la evaluación cuantitativa y el análisis comparativo del desempeño del sistema mecatrónico integral implementado. A partir de los registros de telemetría obtenidos durante las pruebas experimentales en simulación física, con recopilación a 100 Hz ($T_{datos} = 10$ ms) y control interno nominal a 500 Hz ($T_s = 2$ ms), se analizan con rigor de ingeniería los resultados cinemáticos de seguimiento articular, la predictibilidad cronométrica en tiempo real, la aceleración de inferencia y la compresión de memoria alcanzada mediante la cuantización entera INT8 en comparación con las líneas base de control analítico (RNEA con PD) y punto flotante (FP32). Finalmente, se evalúa el cumplimiento integral de los requerimientos técnicos de diseño formulados para Trabajo Terminal 1.

4.1. Entorno de pruebas y condiciones experimentales (Sim-to-Real)

La validación empírica del sistema mecatrónico se estructuró sobre un entorno de pruebas controlado que reproduce fielmente las restricciones físicas y computacionales de la plataforma robótica real:

- **Planta robótica y gemelo digital:** Modelo cinemático y dinámico del manipulador Universal Robots UR3e ejecutado en Gazebo Harmonic (versión 8), con la integración de control mediante `gz_ros2_control`. La cinemática de cadena abierta serial de 6 grados de libertad incorpora las propiedades inerciales reales (masas y tensores de inercia baricéntricos) provistos por el fabricante.

- **Arquitectura de control y middleware:** Entorno ROS 2 Jazzy Jalisco con el framework de control en tiempo real `ros2_control`. El lazo periódico de control se ejecutó a una frecuencia nominal de 500 Hz ($T_s = 2$ ms), gestionando interfaces directas de comando por esfuerzo articular (`hardware_interface::HW_IF_EFFORT`) y lectura de estado posicional y de velocidad (`HW_IF_POSITION`, `HW_IF_VELOCITY`).

- **Plataforma de procesamiento computacional:** Estación de trabajo equipada con procesador AMD Ryzen 7 5800X3D (8 núcleos físicos, 16 hilos de ejecución, frecuencia base de 3,4 GHz con turbo hasta 4,5 GHz, 96 MB de caché L3 3D V-Cache y soporte para extensiones vectoriales AVX2/FMA3), 32 GB de memoria RAM DDR4, operando bajo el sistema operativo Ubuntu 24.04.5 LTS (kernel Linux 6.8) con políticas de planificación de tiempo real POSIX (`SCHED_FIFO`) asignadas al hilo de control.

- **Motor de inferencia neuronal:** ONNX Runtime en C++, configurado con un hilo para paralelismo intraoperador e interoperador. Esta configuración limita el paralelismo interno de inferencia, sin garantizar la temporización del sistema operativo.

- **Generación de trayectorias de excitación dinámica:** Conforme a la Ecuación (68), el generador de trayectorias (`random_trajectory_generator.py`) sintetizó perfiles polinomiales quínticos de clase $\mathcal{C}^2$ conectando configuraciones articulares admisibles aleatorias distribuidas en el espacio de trabajo. Cada segmento aseguró derivadas primera y segunda nulas en los instantes inicial y final ($\dot{\boldsymbol{q}} = \mathbf{0}$, $\ddot{\boldsymbol{q}} = \mathbf{0}$), sin que esas condiciones verifiquen protección mecánica de los actuadores.

- **Protocolo de ensayo continuo:** El ensayo experimental se extendió durante 600 s, con el lazo interno configurado a 500 Hz y la publicación de referencias y recopilación de datos a 100 Hz. Esta última tasa corresponde nominalmente a 60 000 muestras por señal durante el ensayo; el tamaño efectivo de cada serie utilizada en los cálculos corresponde a los registros disponibles tras su procesamiento, no al número de actualizaciones internas del controlador.

4.2. Precisión dinámica y error de seguimiento articular

Para contrastar cuantitativamente el impacto de la aproximación neuronal y de la cuantización numérica sobre el comportamiento cinemático del robot, se evaluaron tres esquemas de control bajo idénticas consignas de trayectoria polinomial quíntica:

1. **Control Analítico RNEA con PD:** Línea base y controlador maestro que suma el torque RNEA y la retroalimentación proporcional-derivativa, conforme a la Ecuación (39).

2. **Predictor Neuronal de Torque Total (FP32):** Predicción de torque total mediante la red profunda en precisión simple de 32 bits, utilizando el torque RNEA como entrada y aplicando directamente la salida desnormalizada y saturada.

3. **Predictor Neuronal de Torque Total (INT8):** La misma arquitectura de torque total, cuyo modelo neuronal opera bajo cuantización dinámica post-entrenamiento a enteros de 8 bits (*Dynamic PTQ*), ejecutando la aritmética lineal en enteros mediante ONNX Runtime C++.

La métrica principal de desempeño cinemático empleada fue el Error Cuadrático Medio (*Root Mean Square Error*, RMSE) en el espacio articular, definido como:

$$\mathrm{RMSE}_i = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(q_{des,i}[k] - q_{act,i}[k]\right)^2} \tag{69}$$

donde N es el número efectivo de muestras temporales válidas de la serie recopilada a 100 Hz, y $i \in \{1, \ldots, 6\}$ denota cada articulación del manipulador. Para un ensayo de 600 s, el número nominal de muestras por articulación es aproximadamente 60 000; no se identifica N con los ciclos internos del controlador a 500 Hz.

Los valores empíricos consolidados de error de seguimiento obtenidos en simulación se resumen a continuación:

- **Control Analítico RNEA con PD:** Registró un RMSE = 0,063 rad. Este esquema genera los torques totales empleados como etiquetas de entrenamiento y constituye la referencia analítica de la comparación.
- **Control Neuronal FP32:** Registró un RMSE = 0,092 rad. El predictor neuronal en coma flotante aproxima el torque total del controlador maestro; no obstante, el desacoplo intencional introducido al fijar $\ddot{\boldsymbol{q}}_{act} = \mathbf{0}$ en el vector de características de entrada genera una discrepancia cinemática menor respecto al modelo teórico ideal.
- **Control Neuronal Cuantizado INT8:** Registró un RMSE = 0,138 rad. La discretización de los pesos sinápticos a enteros de 8 bits induce un incremento en el error de seguimiento respecto a la referencia de punto flotante (0,092 $\rightarrow$ 0,138 rad).

Discusión física y numérica del error de cuantización: El incremento en el error de seguimiento del modelo INT8 responde a dos factores técnicos bien identificados:

1. **Propagación de ruido de discretización afín:** La cuantización uniforme post-entrenamiento mapea el continuo de los parámetros en coma flotante a 256 niveles enteros discretos, introduciendo un error de redondeo acotado $\epsilon_q \in [-S/2, S/2]$ por cada peso sináptico. Al propagarse dicho residuo a través de las 5 transformaciones lineales densas (arquitectura 42-256-512-256-128-6) sin una etapa posterior de recalibración de gradientes (*fine-tuning*), se genera un pequeño sesgo en la estimación del par de salida.
2. **Tratamiento de la aceleración en despliegue:** Se fijó $\ddot{\boldsymbol{q}}_{act} = \mathbf{0}$ en la entrada neuronal. Esto evita calcular esa derivada en el controlador disponible, pero introduce una diferencia respecto a las entradas de entrenamiento filtradas. No demuestra inmunidad al ruido ni estabilidad numérica; su efecto requiere evaluación específica.

Los errores de seguimiento reportados fueron 0,063 rad para el control analítico, 0,092 rad para FP32 y 0,138 rad para INT8. Los tres valores superan el umbral de RMSE $<$ 0,05 rad establecido en RF-01; por tanto, el requisito no se alcanzó en la campaña experimental de TT1. Estos resultados se conservan como referencia de desempeño para evaluar las mejoras de TT2. La operación observada en simulación y la reducción del costo computacional no sustituyen el criterio cuantitativo de precisión.

4.3. Desempeño cronométrico, latencia de inferencia y presupuesto temporal a 500 Hz

La viabilidad del control inteligente en robótica depende de su capacidad para satisfacer rigurosamente las restricciones temporales del lazo cerrado. En un sistema que opera a 500 Hz, la ventana de cómputo total admisible por ciclo es de $T_s = 2000\,\mu$s.

Aceleración de inferencia y dispersión del tiempo de cómputo (FP32 vs. INT8): Se compararon los valores recopilados de latencia del paso hacia adelante (*forward pass*) de la red neuronal dentro del controlador en C++, para los modelos FP32 e INT8. La adquisición se realizó a 100 Hz; por tanto, la serie registrada no representa una medición de cada ejecución interna del controlador a 500 Hz. Los resultados se detallan en la Tabla 13.

Cuadro 13: Comparativa de latencia de inferencia en C++ (ONNX Runtime) con control interno nominal a 500 Hz y recopilación de datos a 100 Hz.

Modelo Evaluado	**Media** (μ)	**Desv. Est.** (σ)	$\boldsymbol{P_{50}}$	$\boldsymbol{P_{95}}$	$\boldsymbol{P_{99}}$	**Máx. Obs.** (t_{max})
Red Neuronal FP32	37,1 μs	2,8 μs	36,8 μs	–	–	42,0 μs
Red Cuantizada INT8	**29,2** μs	**2,1** μs	**28,9** μs	**31,8** μs	**34,5** μs	**39,0** μs
Mejora Relativa	−21,29 %	−25,00 %	−21,47 %	–	–	−7,14 %

Los percentiles P_{95} y P_{99} de FP32 se omiten hasta confirmar los registros, debido a discrepancias entre las tablas previas. Sus mejoras relativas tampoco se calculan sin esa confirmación.

En la Tabla 13 se reporta una reducción de la media de inferencia de 37,1 μs a 29,2 μs (21,29 %). La desviación estándar reportada disminuyó de 2,8 μs a 2,1 μs (25 %). Estos resultados describen las condiciones evaluadas; no identifican por sí solos el kernel ejecutado ni establecen estabilidad temporal del lazo de control.

La Figura 9 distingue la tasa de control interno de las tasas de publicación y recopilación. Los percentiles de la Tabla 13 describen los valores recopilados y no establecen una cota garantizada para todas las ejecuciones del controlador en el entorno CPU y sistema operativo.

Presupuesto temporal consolidado (*Timing Budget*): Para evaluar la predictibilidad del ciclo completo `update()`, la Tabla 14 reúne los intervalos reportados y señala las mediciones pendientes del controlador `NeuralTorqueController` en simulación.

La inferencia INT8 (29,2 μs) representa aproximadamente el 19,3 % del tiempo medio reportado del módulo de control analítico RNEA + PD (151,2 μs). Este cociente compara intervalos de distinto alcance; no representa una aceleración del controlador completo ni una comparación con RNEA aislado. El controlador neuronal también calcula RNEA para formar sus entradas, por lo que su costo total requiere un cronometraje independiente.

La duración del cálculo RNEA aislado dentro del controlador neuronal debe caracterizarse por separado. No se dispone de una medición trazable de ese intervalo para consolidar el presupuesto total; por ello, el tiempo del ciclo neuronal y su margen temporal quedan por verificar.

Cuadro 14: Presupuesto temporal consolidado del ciclo de control de par neuronal a 500 Hz ($T_s = 2000\,\mu$s).

Subproceso en Ciclo `update()`	**Tiempo Medio [μs]**	**Pico Máximo [μs]**	**Uso del Periodo [%]**
1. Lectura de interfaces de estado (`JointState`)	10,2	14,5	0,51 %
2. RNEA aislado dentro del controlador neuronal	Por medir	Por medir	–
3. Preparación y normalización z-score (42 entradas)	5,1	7,0	0,26 %
4. Inferencia neuronal INT8 (ONNX Runtime C++)	29,2	39,0	1,46 %
5. Desnormalización y saturación de comando ($\pm$55,0 N $\cdot$ m)	8,4	11,0	0,42 %
6. Escritura de pares comandados a hardware interface	11,3	15,2	0,57 %
Tiempo total del controlador neuronal	Por verificar	Por verificar	–
Margen temporal del controlador neuronal	Por verificar	Por verificar	–

4.4. Eficiencia computacional y compresión del modelo neuronal

La optimización de modelos de aprendizaje profundo en robótica busca conciliar la capacidad expresiva de las redes con las severas restricciones energéticas y de almacenamiento impuestas por las arquitecturas de cómputo embebidas.

Compresión de huella de memoria: El modelo de red neuronal diseñado para el UR3e consta de 5 capas completamente conectadas con dimensiones 42-256-512-256-128-6, con exactamente 307 590 parámetros entrenables (306 432 pesos y 1 158 sesgos), conforme a la Ecuación (58). El número de parámetros se distingue del tamaño serializado del archivo ONNX: este también contiene el grafo y metadatos; la cuantización dinámica conserva algunos tensores en punto flotante. Los tamaños de archivo reportados son:

- **Modelo Original FP32 (`ur3_neural_controller.onnx`):** 1232 KB.
- **Modelo Cuantizado INT8 (`ur3_neural_controller_quantized.onnx`):** 315 KB.
- **Reducción neta de huella de memoria: 74.43 %.**

Se conservan los tamaños de la campaña experimental confirmados por el autor y su unidad reportada (KB). La reducción relativa del archivo se calcula como:

$$r_S = 100\left(1 - \frac{S_{INT8}}{S_{FP32}}\right) = 100\left(1 - \frac{315}{1232}\right) \approx 74{,}43\,\%. \tag{70}$$

donde S_{FP32} y S_{INT8} son los tamaños de los archivos de la campaña expresados en la misma unidad, y r_S es su reducción porcentual. Estos tamaños no representan la memoria total de la sesión de inferencia.

Atenuación del cuello de botella de Von Neumann (Proxy de eficiencia energética): En plataformas convencionales de computación, el cuello de botella de Von Neumann y la barrera de memoria (*memory wall*) provocan que una porción considerable de la energía disipada y de la latencia computacional se deba al tránsito masivo de operandos entre la memoria principal (RAM / Caché L3) y las unidades de ejecución aritmética de la CPU.

La reducción del tamaño almacenado del modelo y la latencia de inferencia reportada son indicadores de eficiencia de software. No se midieron en TT1 la residencia de parámetros en caché, los kernels SIMD efectivamente ejecutados ni el consumo eléctrico por instrucción. Tampoco se demostró ausencia de asignaciones dinámicas en el controlador: el código disponible construye contenedores dinámicos durante `update()`. Por ello, estos resultados no acreditan por sí solos eficiencia energética medida ni determinismo temporal.

Delimitación metodológica frente a TT2: Es indispensable reiterar que las métricas de compresión (74,43 %) y aceleración de latencia (21,29 %) reportadas en este trabajo constituyen indicadores indirectos (*proxies*) a nivel algorítmico y de arquitectura de software. La cuantificación directa del consumo de potencia eléctrica disipada (en Watts) y de la energía consumida (en Joules) en la planta física demandará la instrumentación con osciloscopios, sensores de corriente Hall y analizadores de potencia en banco de laboratorio sobre el robot real UR3e y la plataforma de hardware reconfigurable (FPGA/Zynq), lo cual conforma el objetivo experimental central de Trabajo Terminal 2.

4.5. Matriz consolidada de cumplimiento técnico de requerimientos

Para concluir el análisis formal de ingeniería de esta primera etapa, la Tabla 15 presenta el dictamen comparativo del sistema mecatrónico implementado frente a los requerimientos funcionales y no funcionales formulados originalmente en el diseño conceptual del proyecto (Tabla 3).

La Tabla 15 permite distinguir los avances de implementación y desempeño computacional del requisito de precisión que no se alcanzó en TT1. En particular, RF-01 permanece pendiente y sus resultados establecen la referencia de desempeño para TT2. Los principales resultados de esta etapa se sintetizan a continuación:

1. El control interno se configuró nominalmente a 500 Hz y las referencias y datos se publicaron a 100 Hz. La inferencia aislada no determina el tiempo total ni el margen del controlador neuronal.

2. La reducción del tamaño del archivo ONNX alcanzó el 74,43 %, superando el criterio de RF-04 ($\geq 50\,\%$).

3. La aceleración de la inferencia neuronal pura alcanzó un 21,29 %, por encima de la cota proyectada del 15 %.

4. El controlador neuronal utiliza una saturación uniforme provisional de ± 55 N·m para facilitar la exploración inicial en simulación. La implementación física deberá considerar límites por articulación y una política de fallo.

5. La meta de ausencia de asignaciones dinámicas en el ciclo de control no se alcanzó en el código disponible.

6. La integración con `ros2_control` y Gazebo Harmonic se realizó en simulación. La interfaz disponible y las condiciones de operación del robot físico deben verificarse en TT2.

Cuadro 15: Matriz comparativa de verificación del sistema mecatrónico frente a los requerimientos de diseño.

Requerimiento / Parámetro	**Meta de Ingeniería**	**Resultado Experimental**	**Dictamen**
Frecuencia nominal del lazo de control	500 Hz ($T_s = 2000\,\mu$s)	500 Hz (configuración del lazo interno)	**Configurado**
Frecuencia de publicación y adquisición	100 Hz ($T_{datos} =$ 10 ms)	100 Hz	**Implementado**
Margen del ciclo neuronal	$> 50\,\%$	Tiempo total pendiente de medición independiente	**Por verificar**
Reducción de archivo ONNX, RF-04	$\geq 50\,\%$	74,43 % (1232 KB $\rightarrow$ 315 KB)	**Cumplido**
Aceleración de inferencia	$> 15\,\%$ red. latencia	21,29 % (37,1 $\rightarrow$ 29,2 μs)	**Cumplido**
Arquitectura de torque total, RF-03	RNEA como entrada; salida total	Etiquetas RNEA + PD; aplicación directa de salida saturada	**Implementado**
Error dinámico articular (RMSE), RF-01	$<$ 0,05 rad	0,063 rad (analítico), 0,092 rad (FP32), 0,138 rad (INT8)	**No alcanzado***
Saturación neuronal provisional	$\pm$55 N·m en simulación	Intervalo uniforme; límites por articulación pendientes para TT2	**Implementado**
Jitter del periodo, RNF-02	$\max_k \lvert j_k \rvert <$ 100 μs	Sin serie temporal trazable del lazo interno	**Por verificar**
Memoria en hilo crítico, RNF-02	Ausencia de asignaciones	Contenedores dinámicos en el código disponible	**No alcanzado**
Transferencia al robot físico	Verificar interfaz y control	Integración demostrada sólo en simulación	**Pendiente TT2**

*RF-01 no se alcanzó en la campaña de TT1. Los errores reportados se conservan como referencia de desempeño; reducir el RMSE por debajo de 0,05 rad constituye un objetivo de mejora para TT2.

Conclusiones

A través del desarrollo de este Trabajo Terminal 1, se diseñó, implementó y evaluó en simulación una arquitectura mecatrónica para el control de posición y la compensación dinámica de un robot manipulador industrial de seis grados de libertad (Universal Robots UR3e). La red se entrenó para aproximar el torque total de un controlador maestro RNEA con PD. En operación neuronal, RNEA permanece como característica de entrada y la salida de la red se aplica tras desnormalización y saturación, sin una suma residual adicional. La solución acopló el gemelo digital en simulación dinámica multicuerpo (Gazebo Harmonic), el middleware robótico de tiempo real (ROS 2 Jazzy Jalisco con el ecosistema `ros2_control`) y un motor de inferencia neuronal programado en C++17 mediante la biblioteca de alto rendimiento ONNX Runtime.

La cuantización dinámica post-entrenamiento (INT8) del modelo 42-256-512-256-128-6 redujo el tamaño del archivo ONNX de 1232 KB a 315 KB (74,43 %). La cuantización INT8 redujo el costo de inferencia observado bajo la configuración experimental empleada: la latencia media pasó de 37,1 μs en FP32 a 29,2 μs en INT8, una reducción del 21,29 %. Este resultado no se atribuye a un camino SIMD específico sin identificar el backend y los kernels ejecutados. Los indicadores de almacenamiento y tiempo de inferencia no sustituyen la medición del consumo eléctrico prevista para TT2.

El lazo interno se configuró nominalmente a 500 Hz ($T_s = 2$ ms), con referencias y datos a 100 Hz. El tiempo total del controlador neuronal y su margen quedan pendientes de medición independiente. Las latencias observadas en CPU y sistema operativo no demuestran un tiempo de peor caso garantizado ni determinismo estricto. El código conserva asignaciones dinámicas y la saturación neuronal requiere una política explícita de fallo; su validación física y temporal queda pendiente.

La evaluación del seguimiento articular en lazo cerrado reportó un RMSE de 0,063 rad en el control analítico, 0,092 rad en FP32 y 0,138 rad en INT8. En consecuencia, el requisito RF-01 de RMSE $< 0{,}05$ rad no se alcanzó con los controladores evaluados en esta campaña de TT1. Estos valores constituyen la referencia de desempeño para la etapa siguiente. En TT2 se buscará reducir el error de seguimiento y evaluar nuevamente el cumplimiento del criterio original, mediante mejoras en el entrenamiento, la cuantización y el tratamiento de las señales de entrada. La contribución de TT1 comprende la implementación y caracterización comparativa de la arquitectura en simulación, con sus limitaciones de precisión expresamente identificadas.

Recomendaciones y trabajo a futuro

Con base en los resultados analíticos, algorítmicos y de verificación experimental obtenidos durante el desarrollo de Trabajo Terminal 1, se han identificado áreas críticas de optimización e innovación mecatrónica. Estas directrices técnicas constituyen la base sobre la cual se estructurará la fase de Trabajo Terminal 2, orientada a la consolidación física sobre la plataforma robótica real y el co-diseño en hardware embebido neutro.

1. Estimación en tiempo real de aceleración articular y supresión de la discrepancia de distribución

En la implementación del controlador neuronal en tiempo real (`ur3_neural_controller.cpp`) a 500 Hz, se tomó la determinación de inicializar los seis componentes de la aceleración articular actual en cero:

$$\ddot{\boldsymbol{q}}_{act} = \mathbf{0}_{6\times 1} \quad \Longrightarrow \quad \texttt{input_tensor_values[idx++] = 0.0f;} \tag{71}$$

Fijar la aceleración a cero evita incorporar una estimación de esa señal en el controlador disponible. No constituye una validación de seguridad física. Una alternativa de estimación a partir de posiciones discretas $\boldsymbol{q}_k$ consiste en la segunda diferencia finita:

$$\ddot{\boldsymbol{q}}_k \approx \frac{\boldsymbol{q}_k - 2\boldsymbol{q}_{k-1} + \boldsymbol{q}_{k-2}}{\Delta t^2} \tag{72}$$

El término $1/\Delta t^2$ puede amplificar el ruido de medición en esa estimación. Para $\Delta t = 2$ ms, su valor es $2{,}5 \times 10^5$ s^{-2}. Ese factor no determina por sí solo el ruido de torque ni demuestra vibraciones o saturación en actuadores; su efecto depende de las señales, el filtrado y el controlador.

No obstante, la base de datos de entrenamiento fuera de línea (`dataset_ur3_final_1.csv`) fue procesada mediante el filtro digital de Savitzky-Golay [33], extrayendo señales de aceleración continuas y no nulas coincidentes con las dinámicas de trayectorias excitatorias. Por ende, fijar $\ddot{\boldsymbol{q}}_{act} = \mathbf{0}$ durante la inferencia en bucle cerrado crea una discrepancia estructural de distribución o sesgo de covarianza (*covariate shift*) entre el dominio de entrenamiento ($\mathcal{D}_{train}$) y el dominio de despliegue en línea ($\mathcal{D}_{test}$):

$$P_{train}(\boldsymbol{x}) \neq P_{test}(\boldsymbol{x}), \quad \text{donde } \boldsymbol{x} \in \mathbb{R}^{42} \tag{73}$$

La diferencia entre las entradas de entrenamiento y despliegue se considera una hipótesis para estudiar el error de seguimiento. Su contribución requiere ensayos controlados; los RMSE reportados de 0,063 rad, 0,092 rad y 0,138 rad no establecen por sí solos esa causalidad.

Para solventar esta limitación en Trabajo Terminal 2, se recomienda implementar las siguientes estrategias:

1. **Implementación de un diferenciador robusto de modos deslizantes de orden superior (HOSM):** Diseñar e integrar en C++ un diferenciador de Levant de segundo orden [35], cuya implementación deberá evaluarse bajo las hipótesis del método y las condiciones reales de muestreo y ruido, cuantificando el error de estimación y el retardo introducido.

2. **Filtro de Kalman Extendido (EKF) desacoplado:** Desarrollar un estimador de estado cinemático articular basado en un modelo de aceleración estocástica casi constante, fusionando las lecturas posicionales del encoder con la predicción del par motriz.

3. **Reentrenamiento con reducción dimensional (36 variables de entrada):** Entrenar una variante topológica de la red eliminando completamente el vector $\ddot{\boldsymbol{q}}_{act}$ del tensor de características:

$$\boldsymbol{x}_{36} = \left[\boldsymbol{q}_{des}^{\top},\, \dot{\boldsymbol{q}}_{des}^{\top},\, \ddot{\boldsymbol{q}}_{des}^{\top},\, \boldsymbol{q}_{act}^{\top},\, \dot{\boldsymbol{q}}_{act}^{\top},\, \boldsymbol{\tau}_{RNEA}^{\top}\right]^{\top} \in \mathbb{R}^{36} \tag{74}$$

o incorporar regularización por descarte estocástico de aceleración (*acceleration dropout*) durante el entrenamiento en PyTorch, obligando a la red a sintetizar la dinámica inercial exclusivamente a partir de la aceleración deseada $\ddot{\boldsymbol{q}}_{des}$ y los estados cinemáticos reales.

2. Transición hacia Entrenamiento Consciente de Cuantización (QAT)

En TT1 se aplicó cuantización post-entrenamiento (PTQ) al modelo FP32. Se reportó una reducción del tamaño binario del 74,43 % y del tiempo medio de inferencia del 21,29 %. En la comparación de seguimiento, el RMSE aumentó de 0,092 rad para FP32 a 0,138 rad para INT8. Esta diferencia motiva estudiar el efecto de la cuantización mediante ensayos controlados, sin atribuirle por sí sola toda la degradación observada.

Para TT2 se recomienda adoptar un esquema de **Entrenamiento Consciente de Cuantización (QAT - *Quantization-Aware Training*)** [29]. Bajo este paradigma:

- Durante la pasada hacia adelante (*forward pass*), las operaciones de convolución y multiplicación matricial simulan el redondeo entero de 8 bits y el efecto de saturación mediante operadores de cuantización aparente (*fake quantization*).
- Durante la pasada hacia atrás (*backward pass*), el estimador de paso directo (*Straight-Through Estimator - STE*) propaga los gradientes en precisión flotante continua (FP32), para ajustar los pesos considerando el efecto de cuantización; el resultado requiere evaluación posterior.

En TT2 se buscará reducir el RMSE respecto de las referencias obtenidas en TT1 y alcanzar el criterio original de RF-01, RMSE $<$ 0,05 rad. El efecto de QAT y de las mejoras en las señales de entrada se evaluará experimentalmente junto con el tamaño del modelo y la latencia de inferencia; no se presupone que esas métricas permanezcan idénticas a las obtenidas en TT1.

3. Co-diseño y aceleración en hardware embebido neutro (FPGA / SoC Zynq)

Como propuesta para evaluar los requerimientos no funcionales de la plataforma neutra (RNF-04), se plantea migrar la inferencia neuronal desde la CPU de propósito general hacia una plataforma de lógica programable reconfigurable (FPGA, e.g., AMD-Xilinx Zynq-7000 o PYNQ-Z2).

La arquitectura propuesta contempla un co-diseño hardware/software:

- **Sistema de Procesamiento (PS - ARM Cortex-A9 / Cortex-A53):** Ejecución del entorno Linux embebido en tiempo real, el nodo de comunicación bajo ROS 2 Jazzy, la interpolación cinemática polinomial y el envío de tramas de control hacia los controladores de articulación.

- **Lógica Programable (PL - Arreglo FPGA):** Implementación de un acelerador IP de red neuronal en aritmética entera INT8 mediante Síntesis de Alto Nivel (Vitis HLS) o HDL directo. El núcleo procesará las operaciones de multiplicación-acumulación (MAC) en paralelo empleando bloques dedicados DSP48E en una topología sistólica segmentada (*pipelined systolic array*).

El despliegue en FPGA constituye una propuesta para TT2. La latencia, los recursos utilizados y el consumo eléctrico deberán obtenerse mediante síntesis, implementación y medición sobre una plataforma seleccionada. No se establecen cifras futuras de latencia o potencia ni garantías de determinismo del sistema completo.

4. Transferencia Sim-to-Real al manipulador robótico físico Universal Robots UR3e

La integración evaluada en TT1 corresponde a la planta simulada en Gazebo Harmonic con `ros2_control`. La presencia de interfaces de esfuerzo en simulación no demuestra que el driver o la interfaz del robot físico permitan ejecutar la misma ley de torque. Antes de transferir el controlador en TT2 deberán verificarse las modalidades de comando disponibles, las protecciones y las adaptaciones de software necesarias.

Las pruebas experimentales sobre el manipulador físico permitirán:

- Evaluar el desempeño del predictor de torque total frente a fenómenos físicos reales no modelados en la simulación, tales como fricción estática y de Coulomb en los reductores armónicos (*harmonic drives*), holguras mecánicas (*backlash*) y deformaciones elásticas en los eslabones ante cargas dinámicas.

- Calibrar las corrientes de fase de los actuadores y caracterizar la repetibilidad milimétrica del seguimiento en espacio de la tarea (coordenadas cartesianas del efector final).

5. Fortalecimiento de la capa de seguridad activa y diagnóstico en línea

Finalmente, se recomienda enriquecer el módulo de seguridad del sistema mecatrónico integrando un observador de perturbaciones en tiempo real y una envoltura de protección en el espacio de trabajo cartesiano. Este subsistema supervisará activamente la potencia mecánica consumida en cada articulación $P_i = \tau_i \dot{q}_i$, accionando el frenado dinámico preventivo ante cualquier indicio de colisión o discrepancia que supere los umbrales de integridad del manipulador industrial.

Anexos

La gestión adecuada de los recursos y una planificación clara de las actividades son cruciales para el éxito de este proyecto. A continuación, se presenta un desglose de los recursos humanos, materiales, económicos y la infraestructura necesaria, seguido de una planeación general de actividades.

Recursos Estimados e Infraestructura

Recursos Humanos

Los participantes en el proyecto y sus roles son los siguientes:

- **Desarrollador:** Mendoza Díaz Ramiro.
  - *Dedicación Estimada:* Aproximadamente 600 horas distribuidas en las fases TT1 y TT2.
  - *Área Principal STEM de Aporte:* Ingeniería (Mecatrónica).
  - *Otras Áreas STEM de Aporte:* Tecnología, Ciencias de la Computación.
- **Asesor Principal:** Dr. Jorge Alejandro Juárez Lora.
  - *Dedicación Estimada:* Aproximadamente 60 horas de asesoría.
  - *Área Principal STEM de Aporte:* Ingeniería (Electrónica/IA).
  - *Otras Áreas STEM de Aporte:* Ciencias de la Computación.

Recursos Materiales y Económicos

A continuación, se listan los recursos materiales y económicos estimados para el proyecto:

- **Plataforma Robótica (Módulo 1):**
  - *Recurso:* Robot manipulador (Universal Robots UR3e en simulación de alta fidelidad para TT1 y acceso a plataforma física en TT2).
  - *Cantidad:* 1.
  - *Costo Estimado (MXN):* $5,000. - $15,000.
  - *Fuente de Financiamiento:* Recursos Propios y laboratorios institucionales de UPIITA.
  - *Recurso (Opcional):* Componentes para interfaz hardware (si se conecta simulador o controlador a FPGA mediante placa intermedia de acoplamiento).
  - *Cantidad:* 1.
  - *Costo Estimado (MXN):* $500 - $1,000.
  - *Fuente de Financiamiento:* Recursos Propios.

- **Plataforma FPGA / SoC Embebido (Módulo 2):**
  - *Recurso:* Tarjeta de desarrollo FPGA / SoC reconfigurable (ej., AMD-Xilinx PYNQ-Z2 o Zynq-7000).
  - *Cantidad:* 1.
  - *Costo Estimado (MXN):* $3,000 - $7,000.
  - *Fuente de Financiamiento:* Recursos Propios o tarjeta en préstamo por la asesoría técnica.
  - *Recurso:* Cables y accesorios para FPGA (JTAG, USB, fuentes reguladas).
  - *Cantidad:* Varios.
  - *Costo Estimado (MXN):* $500.
  - *Fuente de Financiamiento:* Recursos Propios.
- **Estación de Trabajo para Desarrollo (Módulo 3):**
  - *Recurso:* Computadora personal para desarrollo y cómputo numérico (existente).
  - *Cantidad:* 1.
  - *Costo Estimado (MXN):* N/A (equipo existente).
  - *Fuente de Financiamiento:* N/A.
  - *Recurso:* Software de desarrollo (Ubuntu 24.04 LTS, ROS 2 Jazzy, Gazebo Harmonic, PyTorch 2.x, ONNX Runtime C++, AMD-Xilinx Vivado).
  - *Cantidad:* Licencias de código abierto o licencias académicas institucionales.
  - *Costo Estimado (MXN):* N/A (software de acceso abierto y licencias académicas).
  - *Fuente de Financiamiento:* N/A.
- **Material Diverso y Consumibles (Módulo 4):**
  - *Recurso:* Componentes electrónicos menores (resistencias, cables blindados, protoboard para interfaces de instrumentación).
  - *Cantidad:* Varios.
  - *Costo Estimado (MXN):* $300.
  - *Fuente de Financiamiento:* Recursos Propios.
  - *Recurso:* Material de impresión, papelería y gestión técnica.
  - *Cantidad:* Según necesidad.
  - *Costo Estimado (MXN):* $200.
  - *Fuente de Financiamiento:* Recursos Propios.
- **Imprevistos:**

- *Descripción:* Reserva para contingencias y ajustes de instrumentación (aproximadamente 10 % del subtotal de gastos directos).
- *Costo Estimado (MXN):* $1,000.
- *Fuente de Financiamiento:* Recursos Propios.

Nota: Los costos son estimaciones preliminares sujetas a la disponibilidad de infraestructura en laboratorios institucionales.

Infraestructura Requerida

La infraestructura necesaria para el proyecto incluye:

- **Instalaciones Físicas:**
  - Laboratorio de Cómputo de UPIITA: Acceso a estaciones de trabajo con aceleración gráfica y software especializado.
  - Espacio de Trabajo Personal: Para modelado cinemático, entrenamiento de modelos y desarrollo algorítmico continuo.
  - Laboratorio de Sistemas Digitales y Mecatrónica de UPIITA: Para pruebas de instrumentación, caracterización con osciloscopio y despliegue sobre hardware físico en TT2.
- **Equipo Principal:**
  - Estación de Trabajo Personal: Procesador multinúcleo x86_64 con soporte de instrucciones vectoriales AVX2/AVX-512 y GPU dedicada.
  - Tarjeta de Desarrollo FPGA/SoC: PYNQ-Z2 o Digilent Nexys para co-diseño en lógica reconfigurable.
  - Robot Manipulador: Plataforma robótica Universal Robots UR3e (simulada en TT1 con fidelidad dinámica completa y física en TT2).
  - Instrumentación Electrónica: Multímetro de precisión, osciloscopio digital y fuentes de poder reguladas.
- **Ecosistema de Software e Infraestructura Computacional:**
  - **Sistema Operativo Base:** Ubuntu 24.04 LTS (Noble Numbat 64-bit), configurado con núcleo de baja latencia (*low-latency kernel*) y políticas de planificación de tiempo real POSIX (`SCHED_FIFO`) para reducir fuentes de variación temporal en el bucle de control nominal de 500 Hz, sin que ello demuestre por sí solo un jitter nulo o acotado.
  - **Middleware Robótico y Controladores:** ROS 2 Jazzy Jalisco acoplado al framework de control `ros2_control` y plugins en C++17 para la gestión desacoplada de interfaces de esfuerzo (`HW_IF_EFFORT`).
  - **Simulación Dinámica y Gemelo Digital:** Gazebo Harmonic (versión 8) integrado mediante `gz_ros2_control`.

- **Entornos de Inteligencia Artificial y Compresión:** PyTorch 2.x para entrenamiento y cálculo de gradientes; estándar ONNX (Opset 9) y motor de inferencia en tiempo real ONNX Runtime C++ para el despliegue de modelos cuantizados a INT8.
- **Herramientas para Co-diseño Hardware (Proyección TT2):** AMD-Xilinx Vivado Design Suite y Vitis HLS para el modelado, síntesis y generación del acelerador neuronal en aritmética entera sobre FPGA.
- **Lenguajes y Herramientas de Compilación:** C++17 (compiladores GCC/Clang con optimización `-O3`), Python 3.12 y HDL (VHDL/Verilog).
- **Control de Versiones y Documentación:** Git con repositorio alojado en GitHub y entorno LaTeX (distribución TeX Live) para la redacción técnica.

Planeación de Actividades (Cronograma General)

La gestión detallada del proyecto, incluyendo la asignación de tareas específicas y el seguimiento del progreso, se realiza mediante *Sprints* de duración definida (2 a 4 semanas), conforme a la metodología *Scrum* mecatrónica adoptada (ver Sección 1.2.1). Para fines de este reporte, se presenta a continuación la distribución temporal de los Épicos a lo largo de las 36 semanas estimadas (18 semanas para Trabajo Terminal 1 - TT1, y 18 semanas para Trabajo Terminal 2 - TT2).

Fase TT1 (Semanas 1-18): Enfoque en Investigación, Modelado, Diseño e Implementación en Simulación

- *Semanas 1-4:*

EP01 Configuración del entorno de desarrollo bajo Ubuntu 24.04 LTS, ROS 2 Jazzy, Gazebo Harmonic y cadenas de compilación en C++17.

INV01 Investigación exhaustiva del estado del arte: QNNs, control por par calculado, RNEA, cuantización (PTQ/QAT) y optimización temporal.

DOC1 Inicio formal de la documentación técnica del protocolo y Trabajo Terminal 1.

- *Semanas 5-8:*

EP03 Modelado cinemático DH Craig y dinámico analítico RNEA para el manipulador Universal Robots UR3e. Generación de trayectorias excitatorias quínticas y filtrado Savitzky-Golay.

EP02 Diseño y simulación de la línea base con controlador analítico RNEA con PD y evaluación de dinámicas de seguimiento.

DOC1 Consolidación del marco teórico y propuesta de solución (Capítulos 1 y 2).

- *Semanas 9-13:*

EP03 Entrenamiento de la red neuronal profunda (FP32) en PyTorch para la compensación dinámica inversa acoplada (42 variables de entrada).

EP05 Implementación del flujo de cuantización a INT8 mediante Cuantización Post-Entrenamiento (PTQ) y exportación a formato ONNX.

DOC1 Redacción del diseño conceptual, matrices de selección AHP y arquitectura mecatrónica modular.

- *Semanas 14-18:*

EP04 Desarrollo del controlador en C++17 para `ros2_control`, integrando el motor de inferencia ONNX Runtime con control interno nominal a 500 Hz.

EP05 Pruebas de verificación experimental en simulación sim-to-real; medición de latencia, compresión del modelo y error de seguimiento (RMSE).

DOC1 Culminación de la redacción técnica del reporte de Trabajo Terminal 1 y preparación de la defensa institucional.

ENT1 Entrega formal del informe escrito y presentación de resultados de TT1.

Fase TT2 (Semanas 19-36): Enfoque en Optimización QAT, Co-diseño en FPGA y Validación Experimental Física

- *Semanas 19-22:*

EP06 Implementación del diferenciador robusto de modos deslizantes (HOSM) y transición hacia Entrenamiento Consciente de Cuantización (QAT) para recuperar precisión cinemática.

EP07 Co-diseño hardware/software: síntesis de alto nivel (Vitis HLS) de núcleos MAC dedicados para inferencia entera en FPGA/SoC Zynq.

DOC2 Inicio formal de la documentación y reporte técnico de Trabajo Terminal 2.

- *Semanas 23-26:*

EP07 Despliegue y verificación del acelerador en lógica reconfigurable con interfaz de memoria directa (DMA / AXI-Stream).

EP08 Puesta en marcha de la instrumentación física para medición precisa de potencia y consumo energético en tiempo real.

DOC2 Avance en la redacción de capítulos de ingeniería experimental de TT2.

- *Semanas 27-31:*

EP09 Transferencia Sim-to-Real: conexión del controlador `ros2_control` con el manipulador físico Universal Robots UR3e mediante `Universal_Robots_ROS2_Driver`.

EP10 Ejecución de campañas experimentales comparativas: Control clásico vs Inferencia FP32 vs Cuantización INT8 vs Acelerador FPGA.

DOC2 Registro y análisis estadístico de datos de seguimiento y eficiencia energética.

- *Semanas 32-36:*

EP10 Conclusión de pruebas experimentales, evaluación de la paridad sim-to-real y análisis de envolventes de error.

DOC2 Redacción final del documento de titulación de Trabajo Terminal, incluyendo discusión de resultados y conclusiones definitivas.

ENT2 Entrega formal del reporte final de titulación y examen profesional de titulación.

Nota: Este cronograma constituye la guía de ejecución estructurada por Sprints mecatrónicos. INV01, DOC1, DOC2, ENT1 y ENT2 representan hitos transversales de gestión y titulación.

Referencias

[1] F. C. P. Kevin M. Lynch, *Modern robotics*. Reino Unido: Cambridge University Press, 2017.

[2] J. A. J. Lora, "Design and implementation of a neuromorphic processing platform for robotic control systems," Tesis de Doctorado en Ciencias de la Computación, Centro de Investigación en Computación, Instituto Politécnico Nacional, Ciudad de México, diciembre 2024.

[3] Y. Park, J. Lee, D. Sim, Y. Cho, and C. Park, "Designing spiking neural network-based reinforcement learning for 3d robotic arm applications," *Electronics*, vol. 14, no. 3, p. 578, 2025, article number 578. Check year if accessing preprint vs final version.

[4] J. Chen, J. Li, Y. Li, and X. Miao, "Multiply accumulate operations in memristor crossbar arrays for analog computing," *Journal of Semiconductors*, vol. 42, no. 1, p. 013104, 2021.

[5] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," *Neurocomputing*, vol. 461, pp. 370–403, 2021.

[6] C. Bartolozzi, G. Indiveri, and E. Donati, "Embodied neuromorphic intelligence," *Nature Communications*, vol. 13, p. 1024, 2022.

[7] A. de Vries-Gao, "Artificial intelligence: Supply chain constraints and energy implications," *Joule*, vol. 9, pp. 1–4, may 2025, published April 10, 2025.

[8] L. Zhang, X. Hu, X. Liao, T. Zhou, Y. Peng, and Q. Zhao, "FLQ: Design and implementation of hybrid multi-base full logarithmic quantization neural network acceleration architecture based on FPGA," *Signal Processing: Image Communication*, vol. 134, p. 117270, 2025, available online 16 January 2025.

[9] R. Eigenmann and D. J. Lilja, "Von neumann computers," January 1998, purdue University / University of Minnesota.

[10] I. I. Arikpo, F. U. Ogban, and I. E. Eteng, "VON NEUMANN ARCHITECTURE AND MODERN COMPUTERS," *GLOBAL JOURNAL OF MATHEMATICAL SCIENCES*, vol. 6, no. 2, pp. 97–103, 2007.

[11] G. Singh, L. Chelini, S. Corda, A. J. Awan, S. Stuijk, R. Jordans, H. Corporaal, and A.-J. Boonstra, "Near-memory computing: Past, present, and future," *Microprocessors and Microsystems*, vol. 71, p. 102868, 2019.

[12] Álvaro Domingo Reguero, S. Martínez-Fernández, and R. Verdecchia, "Energy-efficient neural network training through runtime layer freezing, model quantization, and early stopping," *Computer Standards & Interfaces*, vol. 92, p. 103906, 2025, published online 10 October 2024.

[13] Y. Zhang, B. Yin, M. D. Gomony, H. Corporaal, C. Trinitis, and F. Corradi, "Hardware/software co-design optimization for training recurrent neural networks at the edge," *Journal of Low Power Electronics and Applications*, vol. 15, no. 1, p. 15, 2025, article number 15. Check year if accessing preprint vs final version.

[14] B. Jiang, J. Chen, and Y. Liu, "Single-shot pruning and quantization for hardware-friendly neural network acceleration," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106816, 2023.

[15] M. V. Mark W. Spong, Seth Huitchinson, *Robot Modeling and Control*. Estados Unido: Wiley, 2005.

[16] L. Zanatta, F. Barchi, S. Manoni, S. Tolu, A. Bartolini, and A. Acquaviva, "Exploring spiking neural networks for deep reinforcement learning in robotic tasks," *Scientific Reports*, vol. 14, p. 30648, 2024, article number 30648. Published online: 28 December 2024. Check year if different in final print.

[17] D. Kudithipudi, C. Schuman, C. M. Vineyard, T. Pandit, C. Merkel, R. Kubendran, J. B. Aimone, G. Orchard, C. Mayr, R. Benosman, J. Hays, C. Young, C. Bartolozzi, A. Majumdar, S. G. Cardwell, M. Payvand, S. Buckley, S. Kulkarni, H. A. Gonzalez, G. Cauwenberghs, C. S. Thakur, A. Subramoney, and S. Furber, "Neuromorphic computing at scale," *Nature*, vol. 637, pp. 801–812, 2025, published online: 22 January 2025. Check year if accessing preprint vs final version.

[18] N. M. Rezk, T. Nordström, D. Stathis, Z. Ul-Abdin, E. E. Aksoy, and A. Hemani, "MOHAQ: Multi-Objective Hardware-Aware quantization of recurrent neural networks," *Journal of Systems Architecture*, vol. 133, p. 102778, dec 2022.

[19] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005.

[20] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.

[21] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.

[22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[23] Intel. (2021) Llevar la computación neuromórfica al siguiente nivel con loihi 2. Accessed: 2025-05-01. [Online]. Available: https://www.intel.la/content/www/xl/es/research/neuromorphic-computing-loihi-2-technology-brief.html

[24] X. Li, H. Jiang, R. Zhang, F. Tian, S. Huang, and D. Xu, "Robustness-aware 2-bit quantization with real-time performance for neural network," *Neurocomputing*, vol. 455, pp. 12–22, 2021.

[25] K. Schwaber and J. Sutherland, *The Scrum Guide™*, November 2020, the Definitive Guide to Scrum: The Rules of the Game. Available at https://scrumguides.org. [Online]. Available: https://scrumguides.org

[26] E. L. Cano, J. M. García-Camús, J. Garzás, J. M. Moguerza, and N. N. Sánchez, "A scrum-based framework for new product development in the non-software industry," *Journal of Engineering and Technology Management*, vol. 61, p. 101634, 2021. [Online]. Available: https://doi.org/10.1016/j.jengtecman.2021.101634

[27] L. Biagiotti and C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*. Berlin, Heidelberg: Springer-Verlag, 2008.

[28] J. Y. S. Luh, M. W. Walker, and R. P. C. Paul, "On-line computational scheme for mechanical manipulators," *IEEE Transactions on Automatic Control*, vol. 25, no. 3, pp. 468–474, 1980.

[29] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[30] S. Macenski, T. Foote, B. Gerkey, M. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[31] Universal Robots A/S, *UR3e User Manual - Original Instructions (e-Series)*, Universal Robots A/S, Odense, Denmark, 2021, version 5.11.

[32] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*, 1st ed. New York: John Wiley & Sons, 2006.

[33] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.

[34] S. Pugh, *Total Design: Integrated Methods for Successful Product Engineering*. Wokingham, England: Addison-Wesley, 1991.

[35] A. Levant, "Higher-order sliding modes, differentiation and output-feedback control," *International Journal of Control*, vol. 76, no. 9-10, pp. 924–941, 2003.